Learn Python Programming

programiz.com/python-programming

About Python Programming

- **Free and open-source** - You can freely use and distribute Python, even for commercial use.
- **Easy to learn** - Python has a very simple and elegant syntax. It's much easier to read and write Python programs compared to other languages like C++, Java, C#.
- **Portable** - You can move Python programs from one platform to another, and run it without any changes.

Why Learn Python?

- Python is easy to learn. Its syntax is easy and code is very readable.
- Python has a lot of applications. It's used for developing web applications, data science, rapid application development, and so on.
- Python allows you to write programs in fewer lines of code than most of the programming languages.
- The popularity of Python is growing rapidly. Now it's one of the most popular programming languages.

How to learn Python?

- **Python tutorial from Programiz** - We provide step by step Python tutorials, examples, and references. Get started with Python.
- **Official Python tutorial** - Might be hard to follow and understand for beginners. Visit the official Python tutorial.
- **Get Learn Python App** - The beginner-friendly app contains byte-size lessons and an integrated Python interpreter. To learn more, visit: Learn Python app
- **Write a lot of Python code**- The only way you can learn programming is by writing a lot of code.

Python Resources

How to Get Started With Python?

programiz.com/python-programming/first-program

In this tutorial, you will learn to install and run Python on your computer. Once we do that, we will also write our first Python program.

Python is a cross-platform programming language, which means that it can run on multiple platforms like Windows, macOS, Linux, and has even been ported to the Java and .NET virtual machines. It is free and open-source.

Even though most of today's Linux and Mac have Python pre-installed in it, the version might be out-of-date. So, it is always a good idea to install the most current version.

The Easiest Way to Run Python

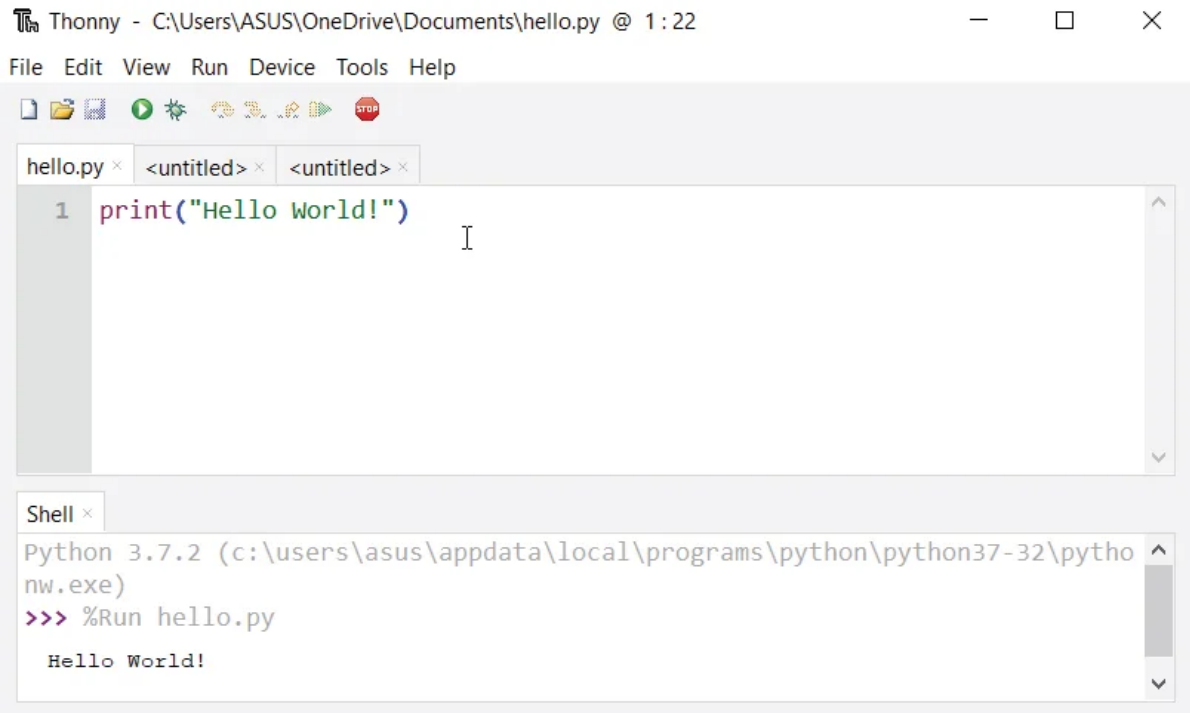
The easiest way to run Python is by using **Thonny IDE**.

The Thonny IDE comes with the latest version of Python bundled in it. So you don't have to install Python separately.

Follow the following steps to run Python on your computer.

1. Download Thonny IDE.
2. Run the installer to install **Thonny** on your computer.
3. Go to: **File** > **New**. Then save the file with `.py` extension. For example, `hello.py`, `example.py`, etc.
   You can give any name to the file. However, the file name should end with **.py**

4. Write Python code in the file and save it.



Running Python using Thonny IDE

5. Then Go to **Run** > **Run current script** or simply click **F5** to run it.

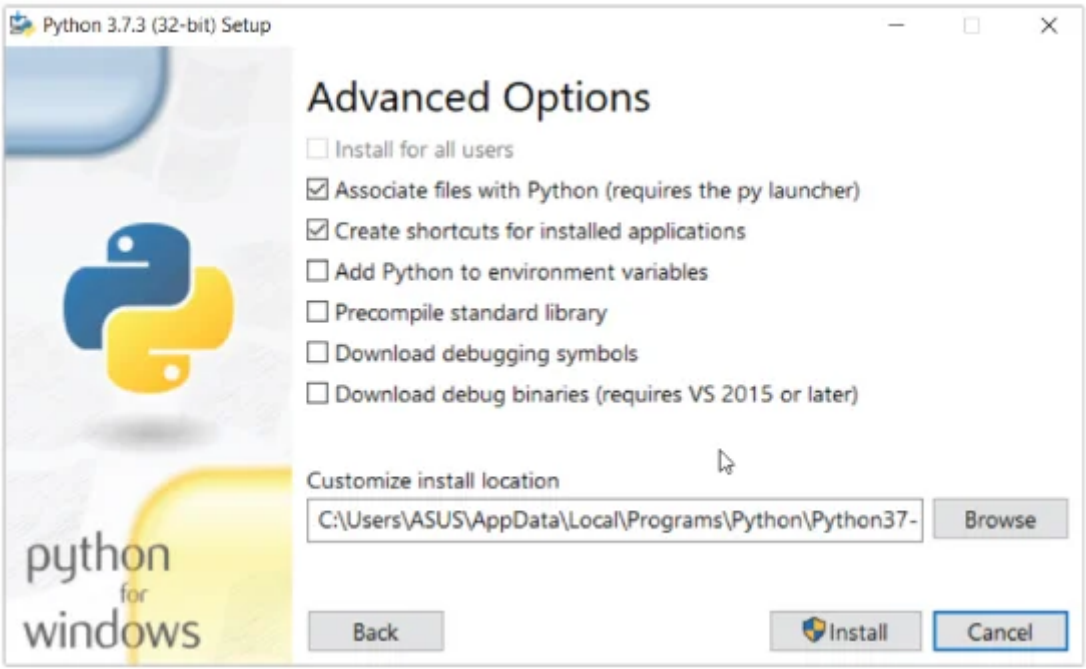
Install Python Separately

If you don't want to use Thonny, here's how you can install and run Python on your computer.

1. Download the latest version of Python.

2. Run the installer file and follow the steps to install Python
   During the install process, check **Add Python to environment variables**. This will add Python to environment variables, and you can run Python from any part of the computer.

   Also, you can choose the path where Python is installed.



Installing Python on the computer

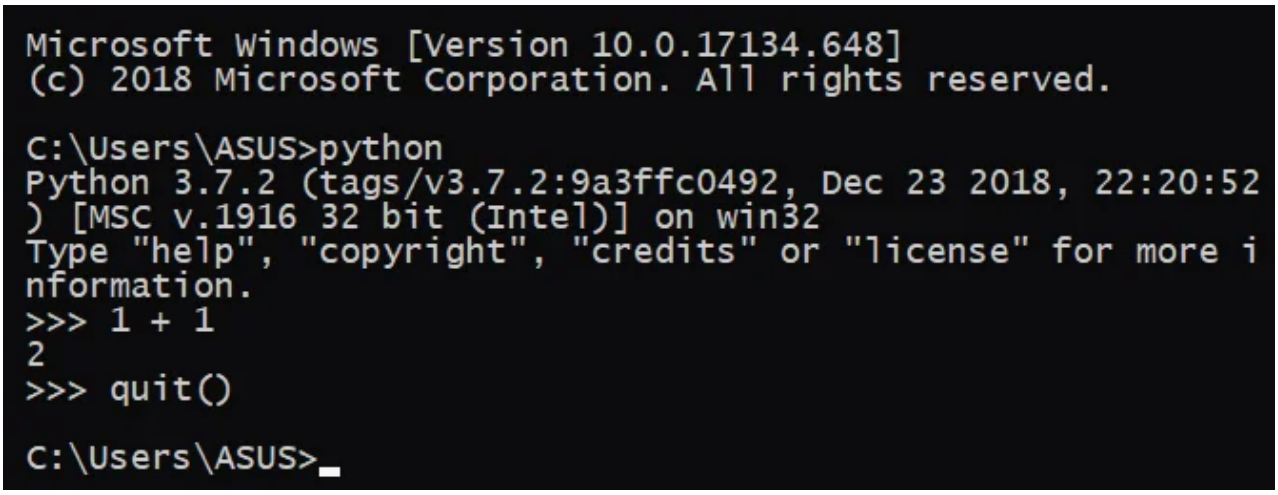
Once you finish the installation process, you can run Python.

---

1. Run Python in Immediate mode

Once Python is installed, typing `python` in the command line will invoke the interpreter in immediate mode. We can directly type in Python code, and press Enter to get the output.

Try typing in `1 + 1` and press enter. We get `2` as the output. This prompt can be used as a calculator. To exit this mode, type `quit()` and press enter.



Running Python on the Command Line

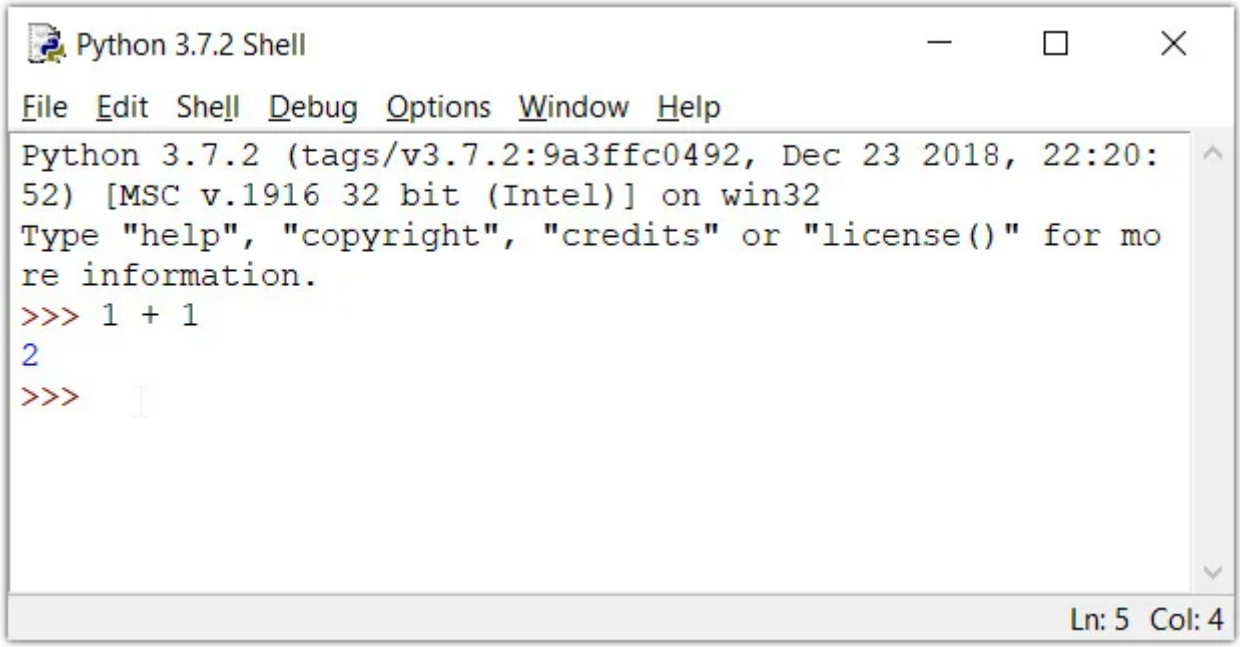
2. Run Python in the Integrated Development Environment (IDE)

We can use any text editing software to write a Python script file.

We just need to save it with the `.py` extension. But using an IDE can make our life a lot easier. IDE is a piece of software that provides useful features like code hinting, syntax highlighting and checking, file explorers, etc. to the programmer for application development.

By the way, when you install Python, an IDE named **IDLE** is also installed. You can use it to run Python on your computer. It's a decent IDE for beginners.

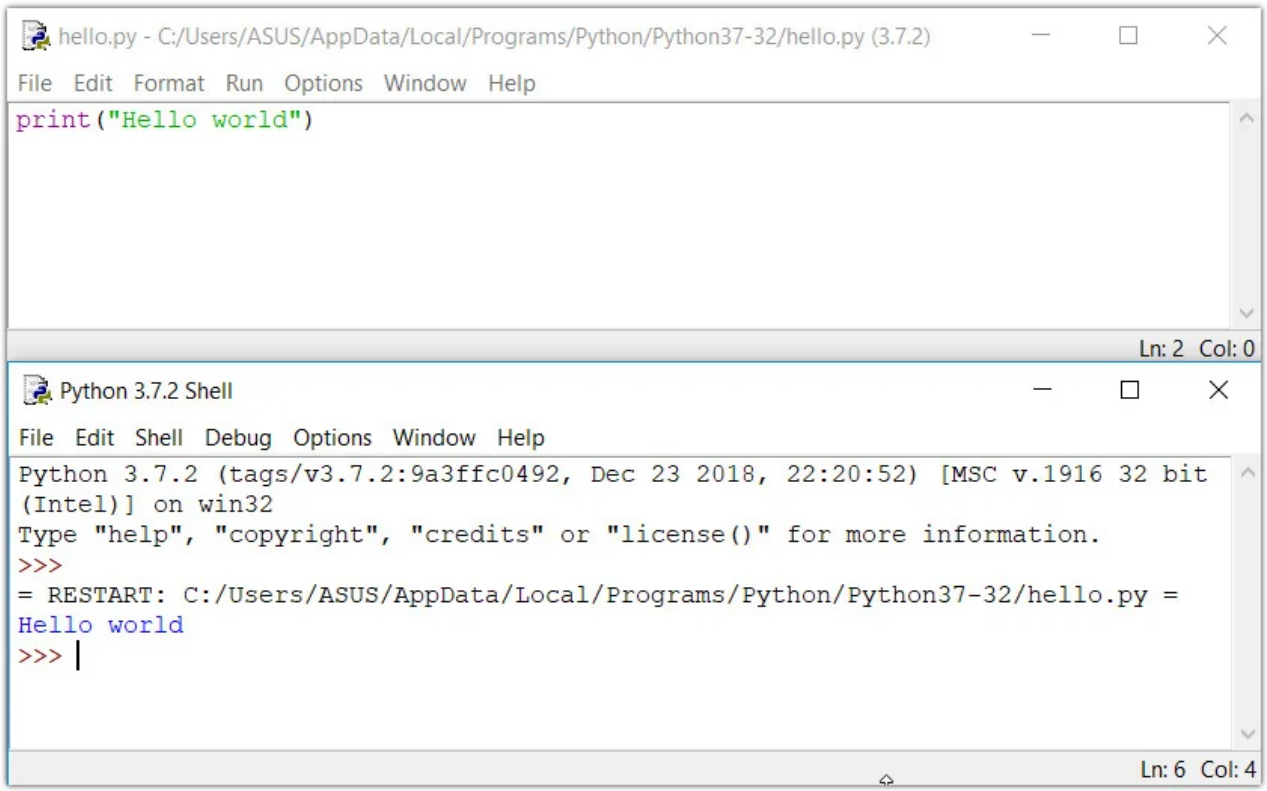
When you open IDLE, an interactive Python Shell is opened.



Python IDLE

Now you can create a new file and save it with **.py** extension. For example, **hello.py**

Write Python code in the file and save it. To run the file, go to **Run** > **Run Module** or simply click **F5**.



Running a Python program in IDLE

Your first Python Program

Now that we have Python up and running, we can write our first Python program.

Let's create a very simple program called `Hello World`. A **"Hello, World!"** is a simple program that outputs `Hello, World!` on the screen. Since it's a very simple program, it's often used to introduce a new programming language to beginners.

Type the following code in any text editor or an IDE and save it as `hello_world.py`

```
print("Hello, world!")
```

Then, run the file. You will get the following output.

```
Hello, world!
```

Congratulations! You just wrote your first program in Python.

As you can see, this was a pretty easy task. This is the beauty of the Python programming language.

Python Keywords and Identifiers

programiz.com/python-programming/keywords-identifier

In this tutorial, you will learn about keywords (reserved words in Python) and identifiers (names given to variables, functions, etc.).

Python Keywords

Keywords are the reserved words in Python.

We cannot use a keyword as a variable name, function name or any other identifier. They are used to define the syntax and structure of the Python language.

In Python, keywords are case sensitive.

There are 33 keywords in Python 3.7. This number can vary slightly over the course of time.

All the keywords except `True`, `False` and `None` are in lowercase and they must be written as they are. The list of all the keywords is given below.

`False`	`await`	`else`	`import`	`pass`
`None`	`break`	`except`	`in`	`raise`
`True`	`class`	`finally`	`is`	`return`
`and`	`continue`	`for`	`lambda`	`try`
`as`	`def`	`from`	`nonlocal`	`while`
`assert`	`del`	`global`	`not`	`with`
`async`	`elif`	`if`	`or`	`yield`

Looking at all the keywords at once and trying to figure out what they mean might be overwhelming.

If you want to have an overview, here is the complete list of all the keywords with examples.

Python Identifiers

An identifier is a name given to entities like class, functions, variables, etc. It helps to differentiate one entity from another.

Rules for writing identifiers

1. Identifiers can be a combination of letters in lowercase **(a to z)** or uppercase **(A to Z)** or digits **(0 to 9)** or an underscore `_` . Names like `myClass` , `var_1` and `print_this_to_screen` , all are valid example.
2. An identifier cannot start with a digit. `1variable` is invalid, but `variable1` is a valid name.
3. Keywords cannot be used as identifiers.

```
global = 1
```

Output

```
  File "<interactive input>", line 1
    global = 1
           ^
SyntaxError: invalid syntax
```

4. We cannot use special symbols like **!**, **@**, **#**, **$**, **%** etc. in our identifier.

```
a@ = 0
```

Output

```
  File "<interactive input>", line 1
    a@ = 0
     ^
SyntaxError: invalid syntax
```

5. An identifier can be of any length.

Things to Remember

Python is a case-sensitive language. This means, `Variable` and `variable` are not the same.

Always give the identifiers a name that makes sense. While `c = 10` is a valid name, writing `count = 10` would make more sense, and it would be easier to figure out what it represents when you look at your code after a long gap.

Multiple words can be separated using an underscore, like `this_is_a_long_variable` .

Python Statement, Indentation and Comments

programiz.com/python-programming/statement-indentation-comments

In this tutorial, you will learn about Python statements, why indentation is important and use of comments in programming.

Python Statement

Instructions that a Python interpreter can execute are called statements. For example, `a = 1` is an assignment statement. `if` statement, `for` statement, `while` statement, etc. are other kinds of statements which will be discussed later.

Multi-line statement

In Python, the end of a statement is marked by a newline character. But we can make a statement extend over multiple lines with the line continuation character (\). For example:

```
a = 1 + 2 + 3 + \
    4 + 5 + 6 + \
    7 + 8 + 9
```

This is an explicit line continuation. In Python, line continuation is implied inside parentheses *( )*, brackets *[ ]*, and braces *{ }*. For instance, we can implement the above multi-line statement as:

```
a = (1 + 2 + 3 +
    4 + 5 + 6 +
    7 + 8 + 9)
```

Here, the surrounding parentheses *( )* do the line continuation implicitly. Same is the case with *[ ]* and *{ }*. For example:

```
colors = ['red',
          'blue',
          'green']
```

We can also put multiple statements in a single line using semicolons, as follows:

```
a = 1; b = 2; c = 3
```

Python Indentation

Most of the programming languages like C, C++, and Java use braces *{ }* to define a block of code. Python, however, uses indentation.

A code block (body of a function, loop, etc.) starts with indentation and ends with the first unindented line. The amount of indentation is up to you, but it must be consistent throughout that block.

Generally, four whitespaces are used for indentation and are preferred over tabs. Here is an example.

```
for i in range(1,11):
    print(i)
    if i == 5:
        break
```

The enforcement of indentation in Python makes the code look neat and clean. This results in Python programs that look similar and consistent.

Indentation can be ignored in line continuation, but it's always a good idea to indent. It makes the code more readable. For example:

```
if True:
    print('Hello')
    a = 5
```

and

```
if True: print('Hello'); a = 5
```

both are valid and do the same thing, but the former style is clearer.

Incorrect indentation will result in `IndentationError`.

Python Comments

Comments are very important while writing a program. They describe what is going on inside a program, so that a person looking at the source code does not have a hard time figuring it out.

You might forget the key details of the program you just wrote in a month's time. So taking the time to explain these concepts in the form of comments is always fruitful.

In Python, we use the hash (#) symbol to start writing a comment.

It extends up to the newline character. Comments are for programmers to better understand a program. Python Interpreter ignores comments.

```
#This is a comment
#print out Hello
print('Hello')
```

Multi-line comments

We can have comments that extend up to multiple lines. One way is to use the hash(#) symbol at the beginning of each line. For example:

```
#This is a long comment
#and it extends
#to multiple lines
```

Another way of doing this is to use triple quotes, either ''' or """.

These triple quotes are generally used for multi-line strings. But they can be used as a multi-line comment as well. Unless they are not docstrings, they do not generate any extra code.

```
"""This is also a
perfect example of
multi-line comments"""
```

To learn more about comments, visit Python Comments.

Docstrings in Python

A docstring is short for documentation string.

Python docstrings (documentation strings) are the string literals that appear right after the definition of a function, method, class, or module.

Triple quotes are used while writing docstrings. For example:

```
def double(num):
    """Function to double the value"""
    return 2*num
```

Docstrings appear right after the definition of a function, class, or a module. This separates docstrings from multiline comments using triple quotes.

The docstrings are associated with the object as their `__doc__` attribute.

So, we can access the docstrings of the above function with the following lines of code:

```
def double(num):
    """Function to double the value"""
    return 2*num
print(double.__doc__)
```

Output

```
Function to double the value
```

To learn more about docstrings in Python, visit Python Docstrings.

Python Variables, Constants and Literals

programiz.com/python-programming/variables-constants-literals

In this tutorial, you will learn about Python variables, constants, literals and their use cases.

Python Variables

A variable is a named location used to store data in the memory. It is helpful to think of variables as a container that holds data that can be changed later in the program. For example,

```
number = 10
```

Here, we have created a variable named *number*. We have assigned the value `10` to the variable.

You can think of variables as a bag to store books in it and that book can be replaced at any time.

```
number = 10
number = 1.1
```

Initially, the value of *number* was `10`. Later, it was changed to `1.1`.

Note: In Python, we don't actually assign values to the variables. Instead, Python gives the reference of the object(value) to the variable.

Assigning values to Variables in Python

As you can see from the above example, you can use the assignment operator `=` to assign a value to a variable.

Example 1: Declaring and assigning value to a variable

```
website = "apple.com"
print(website)
```

Output

```
apple.com
```

In the above program, we assigned a value `apple.com` to the variable *website*. Then, we printed out the value assigned to *website* i.e. `apple.com`

Note: Python is a type-inferred language, so you don't have to explicitly define the variable type. It automatically knows that `apple.com` is a string and declares the *website* variable as a string.

Example 2: Changing the value of a variable

```
website = "apple.com"
print(website)

# assigning a new value to website
website = "programiz.com"

print(website)
```

Output

```
apple.com
programiz.com
```

In the above program, we have assigned `apple.com` to the *website* variable initially. Then, the value is changed to `programiz.com`.

Example 3: Assigning multiple values to multiple variables

```
a, b, c = 5, 3.2, "Hello"

print (a)
print (b)
print (c)
```

If we want to assign the same value to multiple variables at once, we can do this as:

```
x = y = z = "same"

print (x)
print (y)
print (z)
```

The second program assigns the `same` string to all the three variables *x*, *y* and *z*.

Constants

A constant is a type of variable whose value cannot be changed. It is helpful to think of constants as containers that hold information which cannot be changed later.

You can think of constants as a bag to store some books which cannot be replaced once placed inside the bag.

Assigning value to constant in Python

In Python, constants are usually declared and assigned in a module. Here, the module is a new file containing variables, functions, etc which is imported to the main file. Inside the module, constants are written in all capital letters and underscores separating the words.

Example 3: Declaring and assigning value to a constant

Create a **constant.py**:

```
PI = 3.14
GRAVITY = 9.8
```

Create a **main.py**:

```
import constant

print(constant.PI)
print(constant.GRAVITY)
```

Output

```
3.14
9.8
```

In the above program, we create a **constant.py** module file. Then, we assign the constant value to *PI* and *GRAVITY*. After that, we create a **main.py** file and import the `constant` module. Finally, we print the constant value.

Note: In reality, we don't use constants in Python. Naming them in all capital letters is a convention to separate them from variables, however, it does not actually prevent reassignment.

Rules and Naming Convention for Variables and constants

1. Constant and variable names should have a combination of letters in lowercase (a to z) or uppercase (**A to Z**) or digits (**0 to 9**) or an underscore (_). For example:

   ```
   snake_case
   MACRO_CASE
   camelCase
   CapWords
   ```

2. Create a name that makes sense. For example, *vowel* makes more sense than *v*.
3. If you want to create a variable name having two words, use underscore to separate them. For example:

   ```
   my_name
   current_salary
   ```

4. Use capital letters possible to declare a constant. For example:

```
PI
G
MASS
SPEED_OF_LIGHT
TEMP
```

5. Never use special symbols like !, @, #, $, %, etc.
6. Don't start a variable name with a digit.

Literals

Literal is a raw data given in a variable or constant. In Python, there are various types of literals they are as follows:

Numeric Literals

Numeric Literals are immutable (unchangeable). Numeric literals can belong to 3 different numerical types: `Integer`, `Float`, and `Complex`.

Example 4: How to use Numeric literals in Python?

```
a = 0b1010 #Binary Literals
b = 100 #Decimal Literal
c = 0o310 #Octal Literal
d = 0x12c #Hexadecimal Literal

#Float Literal
float_1 = 10.5
float_2 = 1.5e2

#Complex Literal
x = 3.14j

print(a, b, c, d)
print(float_1, float_2)
print(x, x.imag, x.real)
```

Output

```
10 100 200 300
10.5 150.0
3.14j 3.14 0.0
```

In the above program,

- We assigned integer literals into different variables. Here, *a* is binary literal, *b* is a decimal literal, *c* is an octal literal and *d* is a hexadecimal literal.
- When we print the variables, all the literals are converted into decimal values.
- `10.5` and `1.5e2` are floating-point literals. `1.5e2` is expressed with exponential and is equivalent to `1.5 * 10`^{2}.

- We assigned a complex literal i.e `3.14j` in variable *x*. Then we use **imaginary** literal (x.imag) and **real** literal (x.real) to create imaginary and real parts of complex numbers.

To learn more about Numeric Literals, refer to Python Numbers.

String literals

A string literal is a sequence of characters surrounded by quotes. We can use both single, double, or triple quotes for a string. And, a character literal is a single character surrounded by single or double quotes.

Example 7: How to use string literals in Python?

```
strings = "This is Python"
char = "C"
multiline_str = """This is a multiline string with more than one line code."""
unicode = u"\u00dcnic\u00f6de"
raw_str = r"raw \n string"

print(strings)
print(char)
print(multiline_str)
print(unicode)
print(raw_str)
```

Output

```
This is Python
C
This is a multiline string with more than one line code.
Ünicöde
raw \n string
```

In the above program, `This is Python` is a string literal and `C` is a character literal.

The value in triple-quotes `"""` assigned to the *multiline_str* is a multi-line string literal.

The string `u"\u00dcnic\u00f6de"` is a Unicode literal which supports characters other than English. In this case, `\u00dc` represents `Ü` and `\u00f6` represents `ö`.

`r"raw \n string"` is a raw string literal.

Boolean literals

A Boolean literal can have any of the two values: `True` or `False`.

Example 8: How to use boolean literals in Python?

```
x = (1 == True)
y = (1 == False)
a = True + 4
b = False + 10

print("x is", x)
print("y is", y)
print("a:", a)
print("b:", b)
```

Output

```
x is True
y is False
a: 5
b: 10
```

In the above program, we use boolean literal `True` and `False`. In Python, `True` represents the value as `1` and `False` as `0`. The value of *x* is `True` because `1` is equal to `True`. And, the value of *y* is `False` because `1` is not equal to `False`.

Similarly, we can use the `True` and `False` in numeric expressions as the value. The value of *a* is `5` because we add `True` which has a value of `1` with `4`. Similarly, *b* is `10` because we add the `False` having value of `0` with `10`.

Special literals

Python contains one special literal i.e. `None`. We use it to specify that the field has not been created.

Example 9: How to use special literals in Python?

```
drink = "Available"
food = None

def menu(x):
    if x == drink:
        print(drink)
    else:
        print(food)

menu(drink)
menu(food)
```

Output

```
Available
None
```

In the above program, we define a `menu` function. Inside `menu`, when we set the argument as `drink` then, it displays `Available`. And, when the argument is `food`, it displays `None`.

Literal Collections

There are four different literal collections List literals, Tuple literals, Dict literals, and Set literals.

Example 10: How to use literals collections in Python?

```
fruits = ["apple", "mango", "orange"] #list
numbers = (1, 2, 3) #tuple
alphabets = {'a':'apple', 'b':'ball', 'c':'cat'} #dictionary
vowels = {'a', 'e', 'i' , 'o', 'u'} #set

print(fruits)
print(numbers)
print(alphabets)
print(vowels)
```

Output

```
['apple', 'mango', 'orange']
(1, 2, 3)
{'a': 'apple', 'b': 'ball', 'c': 'cat'}
{'e', 'a', 'o', 'i', 'u'}
```

In the above program, we created a list of *fruits*, a tuple of *numbers*, a dictionary *dict* having values with keys designated to each value and a set of *vowels*.

To learn more about literal collections, refer to Python Data Types.

Python Data Types

programiz.com/python-programming/variables-datatypes

In this tutorial, you will learn about different data types you can use in Python.

Data types in Python

Every value in Python has a datatype. Since everything is an object in Python programming, data types are actually classes and variables are instance (object) of these classes.

There are various data types in Python. Some of the important types are listed below.

Python Numbers

Integers, floating point numbers and complex numbers fall under Python numbers category. They are defined as `int`, `float` and `complex` classes in Python.

We can use the `type()` function to know which class a variable or a value belongs to. Similarly, the `isinstance()` function is used to check if an object belongs to a particular class.

```
a = 5
print(a, "is of type", type(a))

a = 2.0
print(a, "is of type", type(a))

a = 1+2j
print(a, "is complex number?", isinstance(1+2j,complex))
```

Output

```
5 is of type <class 'int'>
2.0 is of type <class 'float'>
(1+2j) is complex number? True
```

Integers can be of any length, it is only limited by the memory available.

A floating-point number is accurate up to 15 decimal places. Integer and floating points are separated by decimal points. `1` is an integer, `1.0` is a floating-point number.

Complex numbers are written in the form, `x + yj`, where x is the real part and y is the imaginary part. Here are some examples.

```
>>> a = 1234567890123456789
>>> a
1234567890123456789
>>> b = 0.1234567890123456789
>>> b
0.12345678901234568
>>> c = 1+2j
>>> c
(1+2j)
```

Notice that the `float` variable *b* got truncated.

Python List

List is an ordered sequence of items. It is one of the most used datatype in Python and is very flexible. All the items in a list do not need to be of the same type.

Declaring a list is pretty straight forward. Items separated by commas are enclosed within brackets `[ ]`.

```
a = [1, 2.2, 'python']
```

We can use the slicing operator `[ ]` to extract an item or a range of items from a list. The index starts from 0 in Python.

```
a = [5,10,15,20,25,30,35,40]

# a[2] = 15
print("a[2] = ", a[2])

# a[0:3] = [5, 10, 15]
print("a[0:3] = ", a[0:3])

# a[5:] = [30, 35, 40]
print("a[5:] = ", a[5:])
```

Output

```
a[2] =  15
a[0:3] =  [5, 10, 15]
a[5:] =  [30, 35, 40]
```

Lists are mutable, meaning, the value of elements of a list can be altered.

```
a = [1, 2, 3]
a[2] = 4
print(a)
```

Output

```
[1, 2, 4]
```

Python Tuple

Tuple is an ordered sequence of items same as a list. The only difference is that tuples are immutable. Tuples once created cannot be modified.

Tuples are used to write-protect data and are usually faster than lists as they cannot change dynamically.

It is defined within parentheses `()` where items are separated by commas.

```
t = (5,'program', 1+3j)
```

We can use the slicing operator `[]` to extract items but we cannot change its value.

```
t = (5,'program', 1+3j)

# t[1] = 'program'
print("t[1] = ", t[1])

# t[0:3] = (5, 'program', (1+3j))
print("t[0:3] = ", t[0:3])

# Generates error
# Tuples are immutable
t[0] = 10
```

Output

```
t[1] =  program
t[0:3] =  (5, 'program', (1+3j))
Traceback (most recent call last):
  File "test.py", line 11, in <module>
    t[0] = 10
TypeError: 'tuple' object does not support item assignment
```

Python Strings

String is sequence of Unicode characters. We can use single quotes or double quotes to represent strings. Multi-line strings can be denoted using triple quotes, `'''` or `"""`.

```
s = "This is a string"
print(s)
s = '''A multiline
string'''
print(s)
```

Output

```
This is a string
A multiline
string
```

Just like a list and tuple, the slicing operator `[ ]` can be used with strings. Strings, however, are immutable.

```
s = 'Hello world!'

# s[4] = 'o'
print("s[4] = ", s[4])

# s[6:11] = 'world'
print("s[6:11] = ", s[6:11])

# Generates error
# Strings are immutable in Python
s[5] ='d'
```

Output

```
s[4] =  o
s[6:11] =  world
Traceback (most recent call last):
  File "<string>", line 11, in <module>
TypeError: 'str' object does not support item assignment
```

Python Set

Set is an unordered collection of unique items. Set is defined by values separated by comma inside braces `{ }`. Items in a set are not ordered.

```
a = {5,2,3,1,4}

# printing set variable
print("a = ", a)

# data type of variable a
print(type(a))
```

Output

```
a =  {1, 2, 3, 4, 5}
<class 'set'>
```

We can perform set operations like union, intersection on two sets. Sets have unique values. They eliminate duplicates.

```
a = {1,2,2,3,3,3}
print(a)
```

Output

```
{1, 2, 3}
```

Since, set are unordered collection, indexing has no meaning. Hence, the slicing operator `[]` does not work.

```
>>> a = {1,2,3}
>>> a[1]
Traceback (most recent call last):
  File "<string>", line 301, in runcode
  File "<interactive input>", line 1, in <module>
TypeError: 'set' object does not support indexing
```

Python Dictionary

Dictionary is an unordered collection of key-value pairs.

It is generally used when we have a huge amount of data. Dictionaries are optimized for retrieving data. We must know the key to retrieve the value.

In Python, dictionaries are defined within braces `{}` with each item being a pair in the form `key:value`. Key and value can be of any type.

```
>>> d = {1:'value','key':2}
>>> type(d)
<class 'dict'>
```

We use key to retrieve the respective value. But not the other way around.

```
d = {1:'value','key':2}
print(type(d))

print("d[1] = ", d[1])

print("d['key'] = ", d['key'])

# Generates error
print("d[2] = ", d[2])
```

Output

```
<class 'dict'>
d[1] =  value
d['key'] =  2
Traceback (most recent call last):
  File "<string>", line 9, in <module>
KeyError: 2
```

Conversion between data types

We can convert between different data types by using different type conversion functions like `int()`, `float()`, `str()`, etc.

```
>>> float(5)
5.0
```

Conversion from float to int will truncate the value (make it closer to zero).

```
>>> int(10.6)
10
>>> int(-10.6)
-10
```

Conversion to and from string must contain compatible values.

```
>>> float('2.5')
2.5
>>> str(25)
'25'
>>> int('1p')
Traceback (most recent call last):
  File "<string>", line 301, in runcode
  File "<interactive input>", line 1, in <module>
ValueError: invalid literal for int() with base 10: '1p'
```

We can even convert one sequence to another.

```
>>> set([1,2,3])
{1, 2, 3}
>>> tuple({5,6,7})
(5, 6, 7)
>>> list('hello')
['h', 'e', 'l', 'l', 'o']
```

To convert to dictionary, each element must be a pair:

```
>>> dict([[1,2],[3,4]])
{1: 2, 3: 4}
>>> dict([(3,26),(4,44)])
{3: 26, 4: 44}
```

Python Type Conversion and Type Casting

programiz.com/python-programming/type-conversion-and-casting

In this article, you will learn about the Type conversion and uses of type conversion.

Before learning Type Conversion in Python, you should have knowledge about Python Data Types.

Type Conversion

The process of converting the value of one data type (integer, string, float, etc.) to another data type is called type conversion. Python has two types of type conversion.

1. Implicit Type Conversion
2. Explicit Type Conversion

Implicit Type Conversion

In Implicit type conversion, Python automatically converts one data type to another data type. This process doesn't need any user involvement.

Let's see an example where Python promotes the conversion of the lower data type (integer) to the higher data type (float) to avoid data loss.

Example 1: Converting integer to float

```
num_int = 123
num_flo = 1.23

num_new = num_int + num_flo

print("datatype of num_int:",type(num_int))
print("datatype of num_flo:",type(num_flo))

print("Value of num_new:",num_new)
print("datatype of num_new:",type(num_new))
```

When we run the above program, the output will be:

```
datatype of num_int: <class 'int'>
datatype of num_flo: <class 'float'>

Value of num_new: 124.23
datatype of num_new: <class 'float'>
```

In the above program,

- We add two variables *num_int* and *num_flo*, storing the value in *num_new*.

- We will look at the data type of all three objects respectively.
- In the output, we can see the data type of *num_int* is an `integer` while the data type of *num_flo* is a `float`.
- Also, we can see the *num_new* has a `float` data type because Python always converts smaller data types to larger data types to avoid the loss of data.

Now, let's try adding a string and an integer, and see how Python deals with it.

Example 2: Addition of string(higher) data type and integer(lower) datatype

```
num_int = 123
num_str = "456"

print("Data type of num_int:",type(num_int))
print("Data type of num_str:",type(num_str))

print(num_int+num_str)
```

When we run the above program, the output will be:

```
Data type of num_int: <class 'int'>
Data type of num_str: <class 'str'>

Traceback (most recent call last):
  File "python", line 7, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

In the above program,

- We add two variables *num_int* and *num_str*.
- As we can see from the output, we got `TypeError`. Python is not able to use Implicit Conversion in such conditions.
- However, Python has a solution for these types of situations which is known as Explicit Conversion.

Explicit Type Conversion

In Explicit Type Conversion, users convert the data type of an object to required data type. We use the predefined functions like `int()`, `float()`, `str()`, etc to perform explicit type conversion.

This type of conversion is also called typecasting because the user casts (changes) the data type of the objects.

Syntax :

```
<required_datatype>(expression)
```

Typecasting can be done by assigning the required data type function to the expression.

Example 3: Addition of string and integer using explicit conversion

```
num_int = 123
num_str = "456"

print("Data type of num_int:",type(num_int))
print("Data type of num_str before Type Casting:",type(num_str))

num_str = int(num_str)
print("Data type of num_str after Type Casting:",type(num_str))

num_sum = num_int + num_str

print("Sum of num_int and num_str:",num_sum)
print("Data type of the sum:",type(num_sum))
```

When we run the above program, the output will be:

```
Data type of num_int: <class 'int'>
Data type of num_str before Type Casting: <class 'str'>

Data type of num_str after Type Casting: <class 'int'>

Sum of num_int and num_str: 579
Data type of the sum: <class 'int'>
```

In the above program,

- We add *num_str* and *num_int* variable.
- We converted *num_str* from string(higher) to integer(lower) type using `int()` function to perform the addition.
- After converting *num_str* to an integer value, Python is able to add these two variables.
- We got the *num_sum* value and data type to be an integer.

Key Points to Remember

1. Type Conversion is the conversion of object from one data type to another data type.
2. Implicit Type Conversion is automatically performed by the Python interpreter.
3. Python avoids the loss of data in Implicit Type Conversion.
4. Explicit Type Conversion is also called Type Casting, the data types of objects are converted using predefined functions by the user.
5. In Type Casting, loss of data may occur as we enforce the object to a specific data type.

Python Input, Output and Import

programiz.com/python-programming/input-output-import

This tutorial focuses on two built-in functions print() and input() to perform I/O task in Python. Also, you will learn to import modules and use them in your program.

Python provides numerous built-in functions that are readily available to us at the Python prompt.

Some of the functions like `input()` and `print()` are widely used for standard input and output operations respectively. Let us see the output section first.

Python Output Using print() function

We use the `print()` function to output data to the standard output device (screen). We can also output data to a file, but this will be discussed later.

An example of its use is given below.

```
print('This sentence is output to the screen')
```

Output

```
This sentence is output to the screen
```

Another example is given below:

```
a = 5
print('The value of a is', a)
```

Output

```
The value of a is 5
```

In the second `print()` statement, we can notice that space was added between the string and the value of variable *a*. This is by default, but we can change it.

The actual syntax of the `print()` function is:

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

Here, `objects` is the value(s) to be printed.

The `sep` separator is used between the values. It defaults into a space character.

After all values are printed, `end` is printed. It defaults into a new line.

The `file` is the object where the values are printed and its default value is `sys.stdout` (screen). Here is an example to illustrate this.

```
print(1, 2, 3, 4)
print(1, 2, 3, 4, sep='*')
print(1, 2, 3, 4, sep='#', end='&')
```

Output

```
1 2 3 4
1*2*3*4
1#2#3#4&
```

Output formatting

Sometimes we would like to format our output to make it look attractive. This can be done by using the `str.format()` method. This method is visible to any string object.

```
>>> x = 5; y = 10
>>> print('The value of x is {} and y is {}'.format(x,y))
The value of x is 5 and y is 10
```

Here, the curly braces `{}` are used as placeholders. We can specify the order in which they are printed by using numbers (tuple index).

```
print('I love {0} and {1}'.format('bread','butter'))
print('I love {1} and {0}'.format('bread','butter'))
```

Output

```
I love bread and butter
I love butter and bread
```

We can even use keyword arguments to format the string.

```
>>> print('Hello {name}, {greeting}'.format(greeting = 'Goodmorning', name =
'John'))
Hello John, Goodmorning
```

We can also format strings like the old `sprintf()` style used in C programming language. We use the `%` operator to accomplish this.

```
>>> x = 12.3456789
>>> print('The value of x is %3.2f' %x)
The value of x is 12.35
>>> print('The value of x is %3.4f' %x)
The value of x is 12.3457
```

Python Input

Up until now, our programs were static. The value of variables was defined or hard coded into the source code.

To allow flexibility, we might want to take the input from the user. In Python, we have the `input()` function to allow this. The syntax for `input()` is:

```
input([prompt])
```

where `prompt` is the string we wish to display on the screen. It is optional.

```
>>> num = input('Enter a number: ')
Enter a number: 10
>>> num
'10'
```

Here, we can see that the entered value `10` is a string, not a number. To convert this into a number we can use `int()` or `float()` functions.

```
>>> int('10')
10
>>> float('10')
10.0
```

This same operation can be performed using the `eval()` function. But `eval` takes it further. It can evaluate even expressions, provided the input is a string

```
>>> int('2+3')
Traceback (most recent call last):
  File "<string>", line 301, in runcode
  File "<interactive input>", line 1, in <module>
ValueError: invalid literal for int() with base 10: '2+3'
>>> eval('2+3')
5
```

Python Import

When our program grows bigger, it is a good idea to break it into different modules.

A module is a file containing Python definitions and statements. Python modules have a filename and end with the extension `.py` .

Definitions inside a module can be imported to another module or the interactive interpreter in Python. We use the `import` keyword to do this.

For example, we can import the `math` module by typing the following line:

```
import math
```

We can use the module in the following ways:

```
import math
print(math.pi)
```

Output

```
3.141592653589793
```

Now all the definitions inside `math` module are available in our scope. We can also import some specific attributes and functions only, using the `from` keyword. For example:

```
>>> from math import pi
>>> pi
3.141592653589793
```

While importing a module, Python looks at several places defined in `sys.path`. It is a list of directory locations.

```
>>> import sys
>>> sys.path
['',
 'C:\\Python33\\Lib\\idlelib',
 'C:\\Windows\\system32\\python33.zip',
 'C:\\Python33\\DLLs',
 'C:\\Python33\\lib',
 'C:\\Python33',
 'C:\\Python33\\lib\\site-packages']
```

We can also add our own location to this list.

Python Operators

programiz.com/python-programming/operators

In this tutorial, you'll learn everything about different types of operators in Python, their syntax and how to use them with examples.

What are operators in python?

Operators are special symbols in Python that carry out arithmetic or logical computation. The value that the operator operates on is called the operand.

For example:

```
>>> 2+3
5
```

Here, `+` is the operator that performs addition. `2` and `3` are the operands and `5` is the output of the operation.

Arithmetic operators

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, etc.

Operator	Meaning	Example
+	Add two operands or unary plus	x + y+ 2
-	Subtract right operand from the left or unary minus	x - y- 2
*	Multiply two operands	x * y
/	Divide left operand by the right one (always results into float)	x / y
%	Modulus - remainder of the division of left operand by the right	x % y (remainder of x/y)
//	Floor division - division that results into whole number adjusted to the left in the number line	x // y
**	Exponent - left operand raised to the power of right	x**y (x to the power y)

Example 1: Arithmetic operators in Python

```
x = 15
y = 4

# Output: x + y = 19
print('x + y =',x+y)

# Output: x - y = 11
print('x - y =',x-y)

# Output: x * y = 60
print('x * y =',x*y)

# Output: x / y = 3.75
print('x / y =',x/y)

# Output: x // y = 3
print('x // y =',x//y)

# Output: x ** y = 50625
print('x ** y =',x**y)
```

Output

```
x + y = 19
x - y = 11
x * y = 60
x / y = 3.75
x // y = 3
x ** y = 50625
```

Comparison operators

Comparison operators are used to compare values. It returns either `True` or `False` according to the condition.

Operator	Meaning	Example
>	Greater than - True if left operand is greater than the right	x > y
<	Less than - True if left operand is less than the right	x < y
==	Equal to - True if both operands are equal	x == y
!=	Not equal to - True if operands are not equal	x != y
>=	Greater than or equal to - True if left operand is greater than or equal to the right	x >= y
<=	Less than or equal to - True if left operand is less than or equal to the right	x <= y

Example 2: Comparison operators in Python

```
x = 10
y = 12

# Output: x > y is False
print('x > y is',x>y)

# Output: x < y is True
print('x < y is',x<y)

# Output: x == y is False
print('x == y is',x==y)

# Output: x != y is True
print('x != y is',x!=y)

# Output: x >= y is False
print('x >= y is',x>=y)

# Output: x <= y is True
print('x <= y is',x<=y)
```

Output

```
x > y is False
x < y is True
x == y is False
x != y is True
x >= y is False
x <= y is True
```

Logical operators

Logical operators are the `and`, `or`, `not` operators.

Operator	Meaning	Example
and	True if both the operands are true	x and y
or	True if either of the operands is true	x or y
not	True if operand is false (complements the operand)	not x

Example 3: Logical Operators in Python

```
x = True
y = False

print('x and y is',x and y)

print('x or y is',x or y)

print('not x is',not x)
```

Output

```
x and y is False
x or y is True
not x is False
```

Here is the truth table for these operators.

Bitwise operators

Bitwise operators act on operands as if they were strings of binary digits. They operate bit by bit, hence the name.

For example, 2 is `10` in binary and 7 is `111`.

In the table below: Let *x* = 10 (`0000 1010` in binary) and *y* = 4 (`0000 0100` in binary)

Operator	Meaning	Example
&	Bitwise AND	x & y = 0 (`0000 0000`)
|	Bitwise OR	x | y = 14 (`0000 1110`)
~	Bitwise NOT	~x = -11 (`1111 0101`)
^	Bitwise XOR	x ^ y = 14 (`0000 1110`)
>>	Bitwise right shift	x >> 2 = 2 (`0000 0010`)
<<	Bitwise left shift	x << 2 = 40 (`0010 1000`)

Assignment operators

Assignment operators are used in Python to assign values to variables.

`a = 5` is a simple assignment operator that assigns the value 5 on the right to the variable *a* on the left.

There are various compound operators in Python like `a += 5` that adds to the variable and later assigns the same. It is equivalent to `a = a + 5`.

Operator	Example	Equivalent to
=	x = 5	x = 5
+=	x += 5	x = x + 5

-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5
//=	x //= 5	x = x // 5
**=	x **= 5	x = x ** 5
&=	x &= 5	x = x & 5
|=	x |= 5	x = x | 5
^=	x ^= 5	x = x ^ 5
>>=	x >>= 5	x = x >> 5
<<=	x <<= 5	x = x << 5

Special operators

Python language offers some special types of operators like the identity operator or the membership operator. They are described below with examples.

Identity operators

`is` and `is not` are the identity operators in Python. They are used to check if two values (or variables) are located on the same part of the memory. Two variables that are equal does not imply that they are identical.

Operator	Meaning	Example
is	True if the operands are identical (refer to the same object)	x is True
is not	True if the operands are not identical (do not refer to the same object)	x is not True

Example 4: Identity operators in Python

```
x1 = 5
y1 = 5
x2 = 'Hello'
y2 = 'Hello'
x3 = [1,2,3]
y3 = [1,2,3]

# Output: False
print(x1 is not y1)

# Output: True
print(x2 is y2)

# Output: False
print(x3 is y3)
```

Output

```
False
True
False
```

Here, we see that *x1* and *y1* are integers of the same values, so they are equal as well as identical. Same is the case with *x2* and *y2* (strings).

But *x3* and *y3* are lists. They are equal but not identical. It is because the interpreter locates them separately in memory although they are equal.

Membership operators

`in` and `not in` are the membership operators in Python. They are used to test whether a value or variable is found in a sequence (string, list, tuple, set and dictionary).

In a dictionary we can only test for presence of key, not the value.

Operator	Meaning	Example
in	True if value/variable is found in the sequence	5 in x
not in	True if value/variable is not found in the sequence	5 not in x

Example #5: Membership operators in Python

```
x = 'Hello world'
y = {1:'a',2:'b'}

# Output: True
print('H' in x)

# Output: True
print('hello' not in x)

# Output: True
print(1 in y)

# Output: False
print('a' in y)
```

Output

```
True
True
True
False
```

Here, `'H'` is in *x* but `'hello'` is not present in *x* (remember, Python is case sensitive). Similarly, `1` is key and `'a'` is the value in dictionary *y*. Hence, `'a' in y` returns `False`.

Python if...else Statement

programiz.com/python-programming/if-elif-else

In this article, you will learn to create decisions in a Python program using different forms of if..else statement.

What is if...else statement in Python?

Decision making is required when we want to execute a code only if a certain condition is satisfied.

The `if…elif…else` statement is used in Python for decision making.

Python if Statement Syntax

```
if test expression:
    statement(s)
```

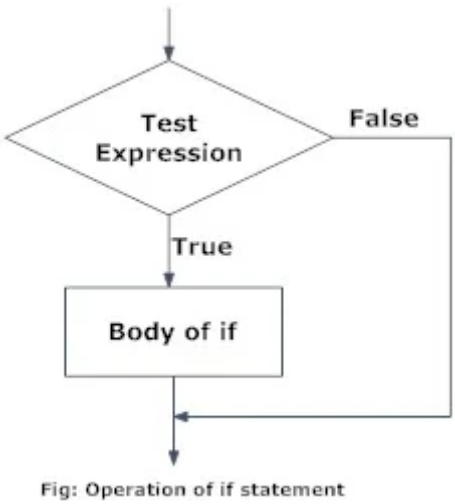
Here, the program evaluates the `test expression` and will execute statement(s) only if the test expression is `True`.

If the test expression is `False`, the statement(s) is not executed.

In Python, the body of the `if` statement is indicated by the indentation. The body starts with an indentation and the first unindented line marks the end.

Python interprets non-zero values as `True`. `None` and `0` are interpreted as `False`.

Python if Statement Flowchart



Flowchart of if statement in Python programming

Example: Python if Statement

```
# If the number is positive, we print an appropriate message

num = 3
if num > 0:
    print(num, "is a positive number.")
print("This is always printed.")

num = -1
if num > 0:
    print(num, "is a positive number.")
print("This is also always printed.")
```

When you run the program, the output will be:

```
3 is a positive number
This is always printed
This is also always printed.
```

In the above example, `num > 0` is the test expression.

The body of `if` is executed only if this evaluates to `True`.

When the variable *num* is equal to 3, test expression is true and statements inside the body of `if` are executed.

If the variable *num* is equal to -1, test expression is false and statements inside the body of `if` are skipped.

The `print()` statement falls outside of the `if` block (unindented). Hence, it is executed regardless of the test expression.

Syntax of if...else

```
if test expression:
    Body of if
else:
    Body of else
```

The `if..else` statement evaluates `test expression` and will execute the body of `if` only when the test condition is `True`.

If the condition is `False`, the body of `else` is executed. Indentation is used to separate the blocks.

Python if..else Flowchart

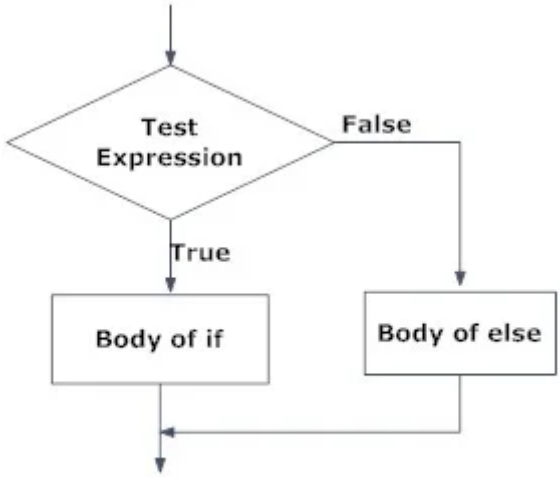


Fig: Operation of if...else statement

Flowchart of if...else statement in Python

Example of if...else

```
# Program checks if the number is positive or negative
# And displays an appropriate message

num = 3

# Try these two variations as well.
# num = -5
# num = 0

if num >= 0:
    print("Positive or Zero")
else:
    print("Negative number")
```

Output

```
Positive or Zero
```

In the above example, when *num* is equal to 3, the test expression is true and the body of `if` is executed and the `body` of else is skipped.

If *num* is equal to -5, the test expression is false and the body of `else` is executed and the body of `if` is skipped.

If *num* is equal to 0, the test expression is true and body of `if` is executed and `body` of else is skipped.

Python if...elif...else Statement

Syntax of if...elif...else

```
if test expression:
    Body of if
elif test expression:
    Body of elif
else:
    Body of else
```

The `elif` is short for else if. It allows us to check for multiple expressions.

If the condition for `if` is `False`, it checks the condition of the next `elif` block and so on.

If all the conditions are `False`, the body of else is executed.

Only one block among the several `if...elif...else` blocks is executed according to the condition.

The `if` block can have only one `else` block. But it can have multiple `elif` blocks.

Flowchart of if...elif...else

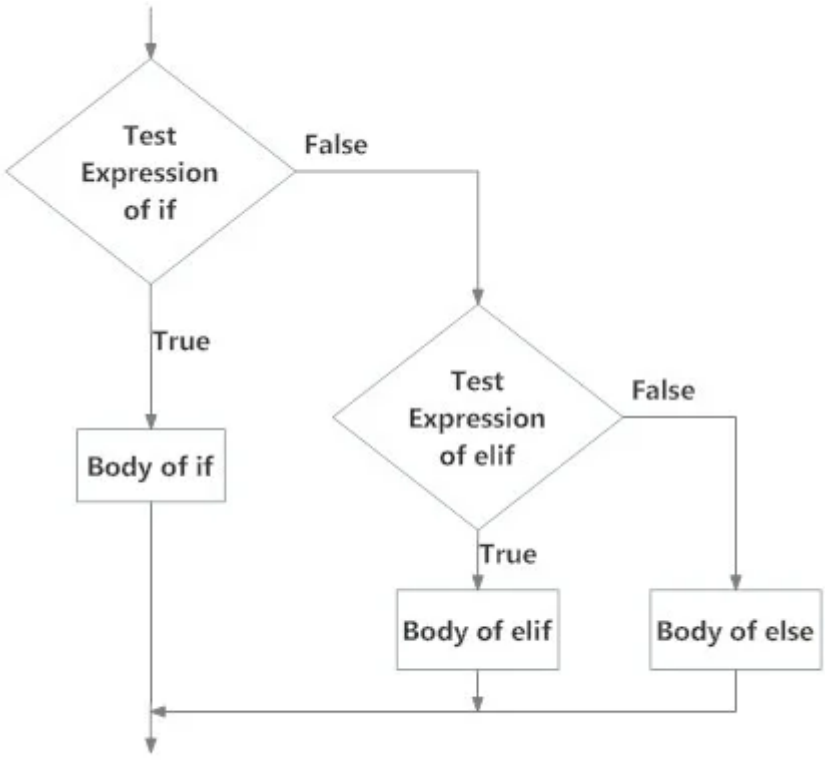


Fig: Operation of if...elif...else statement

Flowchart of if...elif....else statement in Python

Example of if...elif...else

```
'''In this program,
we check if the number is positive or
negative or zero and
display an appropriate message'''

num = 3.4

# Try these two variations as well:
# num = 0
# num = -4.5

if num > 0:
    print("Positive number")
elif num == 0:
    print("Zero")
else:
    print("Negative number")
```

When variable *num* is positive, `Positive number` is printed.

If *num* is equal to 0, `Zero` is printed.

If *num* is negative, `Negative number` is printed.

Python Nested if statements

We can have a `if...elif...else` statement inside another `if...elif...else` statement. This is called nesting in computer programming.

Any number of these statements can be nested inside one another. Indentation is the only way to figure out the level of nesting. They can get confusing, so they must be avoided unless necessary.

Python Nested if Example

```
'''In this program, we input a number
check if the number is positive or
negative or zero and display
an appropriate message
This time we use nested if statement'''

num = float(input("Enter a number: "))
if num >= 0:
    if num == 0:
        print("Zero")
    else:
        print("Positive number")
else:
    print("Negative number")
```

Output 1

```
Enter a number: 5
Positive number
```

Output 2

```
Enter a number: -1
Negative number
```

Output 3

```
Enter a number: 0
Zero
```

Python for Loop

programiz.com/python-programming/for-loop

In this article, you'll learn to iterate over a sequence of elements using the different variations of for loop.

What is for loop in Python?

The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects. Iterating over a sequence is called traversal.

Syntax of for Loop

```
for val in sequence:
    loop body
```

Here, `val` is the variable that takes the value of the item inside the sequence on each iteration.

Loop continues until we reach the last item in the sequence. The body of for loop is separated from the rest of the code using indentation.

Flowchart of for Loop

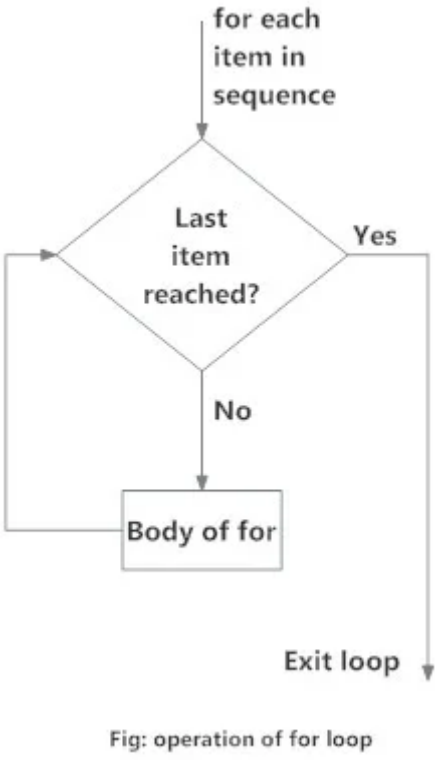


Fig: operation of for loop

Flowchart of for Loop in Python

Example: Python for Loop

```
# Program to find the sum of all numbers stored in a list

# List of numbers
numbers = [6, 5, 3, 8, 4, 2, 5, 4, 11]

# variable to store the sum
sum = 0

# iterate over the list
for val in numbers:
    sum = sum+val

print("The sum is", sum)
```

When you run the program, the output will be:

```
The sum is 48
```

The range() function

We can generate a sequence of numbers using `range()` function. `range(10)` will generate numbers from 0 to 9 (10 numbers).

We can also define the start, stop and step size as `range(start, stop,step_size)`. step_size defaults to 1 if not provided.

The `range` object is "lazy" in a sense because it doesn't generate every number that it "contains" when we create it. However, it is not an iterator since it supports `in`, `len` and `__getitem__` operations.

This function does not store all the values in memory; it would be inefficient. So it remembers the start, stop, step size and generates the next number on the go.

To force this function to output all the items, we can use the function `list()`.

The following example will clarify this.

```
print(range(10))

print(list(range(10)))

print(list(range(2, 8)))

print(list(range(2, 20, 3)))
```

Output

```
range(0, 10)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[2, 3, 4, 5, 6, 7]
[2, 5, 8, 11, 14, 17]
```

We can use the `range()` function in `for` loops to iterate through a sequence of numbers. It can be combined with the `len()` function to iterate through a sequence using indexing. Here is an example.

```
# Program to iterate through a list using indexing

genre = ['pop', 'rock', 'jazz']

# iterate over the list using index
for i in range(len(genre)):
    print("I like", genre[i])
```

Output

```
I like pop
I like rock
I like jazz
```

---

for loop with else

A `for` loop can have an optional `else` block as well. The `else` part is executed if the items in the sequence used in for loop exhausts.

The break keyword can be used to stop a for loop. In such cases, the else part is ignored.

Hence, a for loop's else part runs if no break occurs.

Here is an example to illustrate this.

```
digits = [0, 1, 5]

for i in digits:
    print(i)
else:
    print("No items left.")
```

When you run the program, the output will be:

```
0
1
5
No items left.
```

Here, the for loop prints items of the list until the loop exhausts. When the for loop exhausts, it executes the block of code in the `else` and prints `No items left.`

This `for...else` statement can be used with the `break` keyword to run the `else` block only when the `break` keyword was not executed. Let's take an example:

```
# program to display student's marks from record
student_name = 'Soyuj'

marks = {'James': 90, 'Jules': 55, 'Arthur': 77}

for student in marks:
    if student == student_name:
        print(marks[student])
        break
else:
    print('No entry with that name found.')
```

Output

```
No entry with that name found.
```

Python while Loop

programiz.com/python-programming/while-loop

Loops are used in programming to repeat a specific block of code. In this article, you will learn to create a while loop in Python.

What is while loop in Python?

The while loop in Python is used to iterate over a block of code as long as the test expression (condition) is true.

We generally use this loop when we don't know the number of times to iterate beforehand.

Syntax of while Loop in Python

```
while test_expression:
    Body of while
```

In the while loop, test expression is checked first. The body of the loop is entered only if the `test_expression` evaluates to `True`. After one iteration, the test expression is checked again. This process continues until the `test_expression` evaluates to `False`.

In Python, the body of the while loop is determined through indentation.

The body starts with indentation and the first unindented line marks the end.

Python interprets any non-zero value as `True`. `None` and `0` are interpreted as `False`.

Flowchart of while Loop

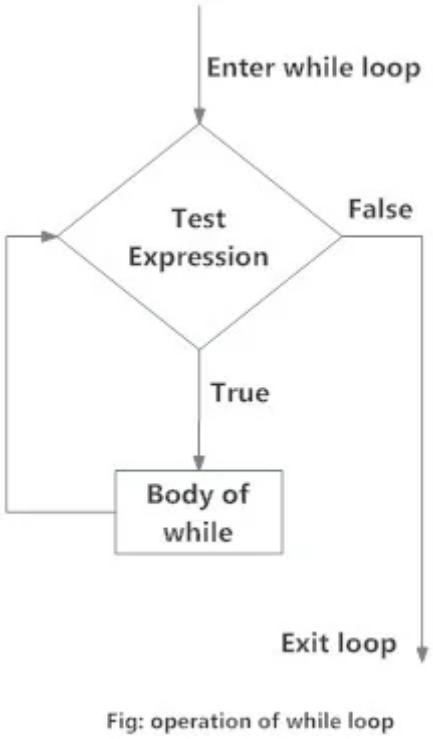


Fig: operation of while loop

Flowchart for while loop in Python

Example: Python while Loop

```
# Program to add natural
# numbers up to
# sum = 1+2+3+...+n

# To take input from the user,
# n = int(input("Enter n: "))

n = 10

# initialize sum and counter
sum = 0
i = 1

while i <= n:
    sum = sum + i
    i = i+1    # update counter

# print the sum
print("The sum is", sum)
```

When you run the program, the output will be:

```
Enter n: 10
The sum is 55
```

In the above program, the test expression will be `True` as long as our counter variable *i* is less than or equal to *n* (10 in our program).

We need to increase the value of the counter variable in the body of the loop. This is very important (and mostly forgotten). Failing to do so will result in an infinite loop (never-ending loop).

Finally, the result is displayed.

While loop with else

Same as with for loops, while loops can also have an optional `else` block.

The `else` part is executed if the condition in the while loop evaluates to `False`.

The while loop can be terminated with a break statement. In such cases, the `else` part is ignored. Hence, a while loop's `else` part runs if no break occurs and the condition is false.

Here is an example to illustrate this.

```
'''Example to illustrate
the use of else statement
with the while loop'''

counter = 0

while counter < 3:
    print("Inside loop")
    counter = counter + 1
else:
    print("Inside else")
```

Output

```
Inside loop
Inside loop
Inside loop
Inside else
```

Here, we use a counter variable to print the string `Inside loop` three times.

On the fourth iteration, the condition in `while` becomes `False`. Hence, the `else` part is executed.

Python break and continue

programiz.com/python-programming/break-continue

In this article, you will learn to use break and continue statements to alter the flow of a loop.

What is the use of break and continue in Python?

In Python, `break` and `continue` statements can alter the flow of a normal loop.

Loops iterate over a block of code until the test expression is false, but sometimes we wish to terminate the current iteration or even the whole loop without checking test expression.

The `break` and `continue` statements are used in these cases.

Python break statement

The `break` statement terminates the loop containing it. Control of the program flows to the statement immediately after the body of the loop.

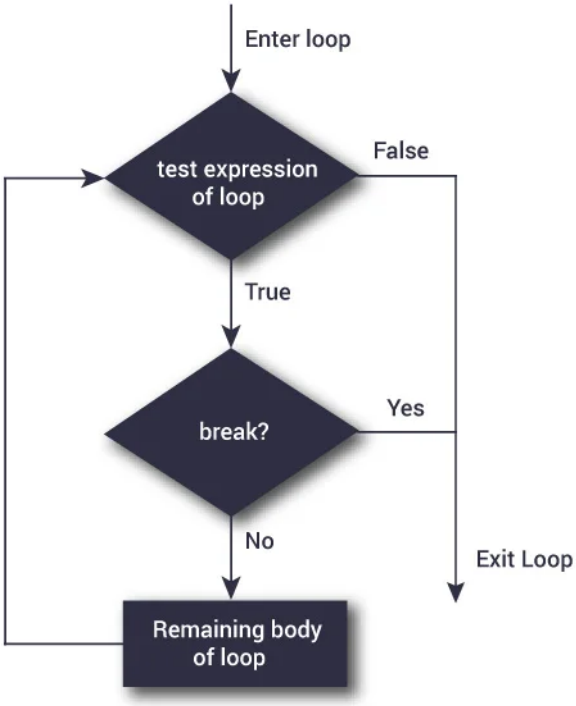
If the `break` statement is inside a nested loop (loop inside another loop), the `break` statement will terminate the innermost loop.

Syntax of break

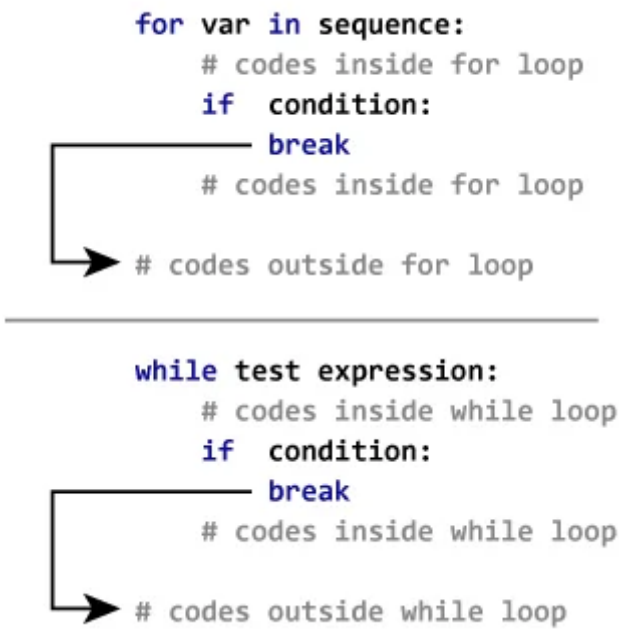
```
break
```

Flowchart of break

The working of break statement in for loop and while loop is shown below.



Flowchart of break statement in Python



Working of the break statement

Example: Python break

```
# Use of break statement inside the loop

for val in "string":
    if val == "i":
        break
    print(val)

print("The end")
```

Output

```
s
t
r
The end
```

In this program, we iterate through the `"string"` sequence. We check if the letter is `i`, upon which we break from the loop. Hence, we see in our output that all the letters up till `i` gets printed. After that, the loop terminates.

Python continue statement

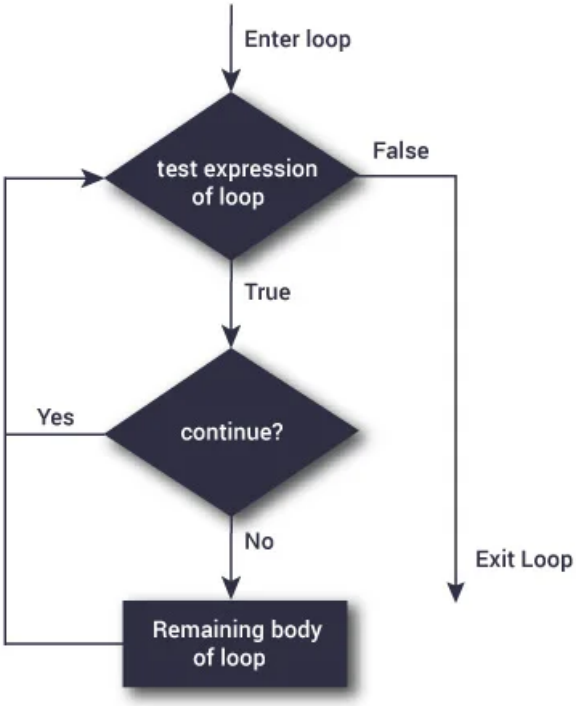
The `continue` statement is used to skip the rest of the code inside a loop for the current iteration only. Loop does not terminate but continues on with the next iteration.

Syntax of Continue

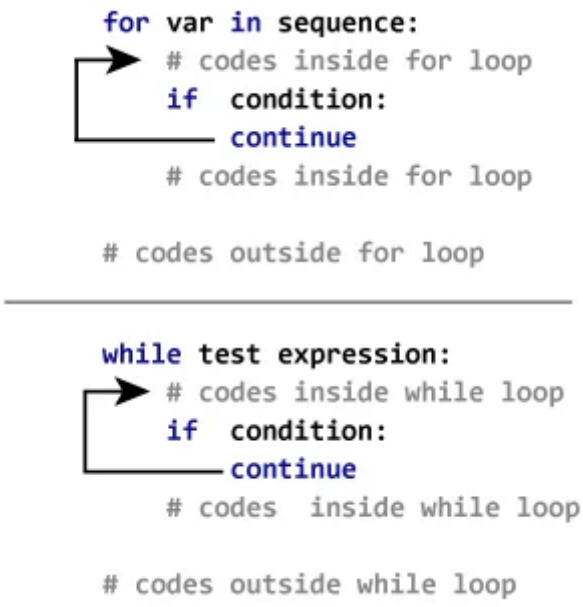
```
continue
```

Flowchart of continue

The working of the `continue` statement in for and while loop is shown below.



Flowchart of continue statement in Python



How continue statement works in python

Example: Python continue

```
# Program to show the use of continue statement inside loops

for val in "string":
    if val == "i":
        continue
    print(val)

print("The end")
```

Output

```
s
t
r
n
g
The end
```

This program is same as the above example except the `break` statement has been replaced with `continue`.

We continue with the loop, if the string is `i`, not executing the rest of the block. Hence, we see in our output that all the letters except `i` gets printed.

Python pass statement

programiz.com/python-programming/pass-statement

In this article, you'll learn about pass statement. It is used as a placeholder for future implementation of functions, loops, etc.

What is pass statement in Python?

In Python programming, the `pass` statement is a null statement. The difference between a comment and a `pass` statement in Python is that while the interpreter ignores a comment entirely, `pass` is not ignored.

However, nothing happens when the pass is executed. It results in no operation (NOP).

Syntax of pass

```
pass
```

We generally use it as a placeholder.

Suppose we have a loop or a function that is not implemented yet, but we want to implement it in the future. They cannot have an empty body. The interpreter would give an error. So, we use the `pass` statement to construct a body that does nothing.

Example: pass Statement

```
'''pass is just a placeholder for
functionality to be added later.'''
sequence = {'p', 'a', 's', 's'}
for val in sequence:
    pass
```

We can do the same thing in an empty function or class as well.

```
def function(args):
    pass

class Example:
    pass
```

Python Functions

programiz.com/python-programming/function

In this article, you'll learn about functions, what a function is, the syntax, components, and types of functions. Also, you'll learn to create a function in Python.

What is a function in Python?

In Python, a function is a group of related statements that performs a specific task.

Functions help break our program into smaller and modular chunks. As our program grows larger and larger, functions make it more organized and manageable.

Furthermore, it avoids repetition and makes the code reusable.

Syntax of Function

```
def function_name(parameters):
        """docstring"""
        statement(s)
```

Above shown is a function definition that consists of the following components.

1. Keyword `def` that marks the start of the function header.
2. A function name to uniquely identify the function. Function naming follows the same rules of writing identifiers in Python.
3. Parameters (arguments) through which we pass values to a function. They are optional.
4. A colon (:) to mark the end of the function header.
5. Optional documentation string (docstring) to describe what the function does.
6. One or more valid python statements that make up the function body. Statements must have the same indentation level (usually 4 spaces).
7. An optional `return` statement to return a value from the function.

Example of a function

```
def greet(name):
    """
    This function greets to
    the person passed in as
    a parameter
    """
    print("Hello, " + name + ". Good morning!")
```

How to call a function in python?

Once we have defined a function, we can call it from another function, program, or even the Python prompt. To call a function we simply type the function name with appropriate parameters.

```
>>> greet('Paul')
Hello, Paul. Good morning!
```

Try running the above code in the Python program with the function definition to see the output.

```
def greet(name):
    """
    This function greets to
    the person passed in as
    a parameter
    """
    print("Hello, " + name + ". Good morning!")

greet('Paul')
```

Note: In python, the function definition should always be present before the function call. Otherwise, we will get an error. For example,

```
# function call
greet('Paul')

# function definition
def greet(name):
    """
    This function greets to
    the person passed in as
    a parameter
    """
    print("Hello, " + name + ". Good morning!")

# Erro: name 'greet' is not defined
```

Docstrings

The first string after the function header is called the docstring and is short for documentation string. It is briefly used to explain what a function does.

Although optional, documentation is a good programming practice. Unless you can remember what you had for dinner last week, always document your code.

In the above example, we have a docstring immediately below the function header. We generally use triple quotes so that docstring can extend up to multiple lines. This string is available to us as the `__doc__` attribute of the function.

For example:

Try running the following into the Python shell to see the output.

```
>>> print(greet.__doc__)

    This function greets to
    the person passed in as
    a parameter
```

To learn more about docstrings in Python, visit Python Docstrings.

The return statement

The `return` statement is used to exit a function and go back to the place from where it was called.

Syntax of return

```
return [expression_list]
```

This statement can contain an expression that gets evaluated and the value is returned. If there is no expression in the statement or the `return` statement itself is not present inside a function, then the function will return the `None` object.

For example:

```
>>> print(greet("May"))
Hello, May. Good morning!
None
```

Here, `None` is the returned value since `greet()` directly prints the name and no `return` statement is used.

Example of return

```
def absolute_value(num):
    """This function returns the absolute
    value of the entered number"""

    if num >= 0:
        return num
    else:
        return -num

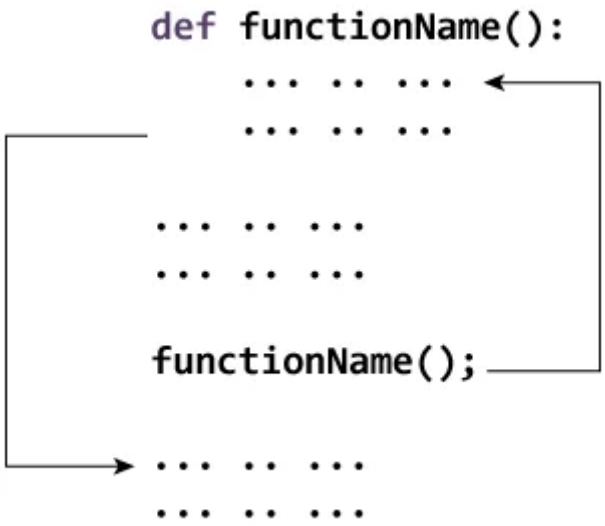

print(absolute_value(2))

print(absolute_value(-4))
```

Output

```
2
4
```

How Function works in Python?



Working of functions in Python

Scope and Lifetime of variables

Scope of a variable is the portion of a program where the variable is recognized. Parameters and variables defined inside a function are not visible from outside the function. Hence, they have a local scope.

The lifetime of a variable is the period throughout which the variable exists in the memory. The lifetime of variables inside a function is as long as the function executes.

They are destroyed once we return from the function. Hence, a function does not remember the value of a variable from its previous calls.

Here is an example to illustrate the scope of a variable inside a function.

```
def my_func():
        x = 10
        print("Value inside function:",x)

x = 20
my_func()
print("Value outside function:",x)
```

Output

```
Value inside function: 10
Value outside function: 20
```

Here, we can see that the value of *x* is 20 initially. Even though the function `my_func()` changed the value of *x* to 10, it did not affect the value outside the function.

This is because the variable *x* inside the function is different (local to the function) from the one outside. Although they have the same names, they are two different variables with different scopes.

On the other hand, variables outside of the function are visible from inside. They have a global scope.

We can read these values from inside the function but cannot change (write) them. In order to modify the value of variables outside the function, they must be declared as global variables using the keyword `global`.

Types of Functions

Basically, we can divide functions into the following two types:

1. Built-in functions - Functions that are built into Python.
2. User-defined functions - Functions defined by the users themselves.

Python Function Arguments

programiz.com/python-programming/function-argument

In Python, you can define a function that takes variable number of arguments. In this article, you will learn to define such functions using default, keyword and arbitrary arguments.

Arguments

In the user-defined function topic, we learned about defining a function and calling it. Otherwise, the function call will result in an error. Here is an example.

```
def greet(name, msg):
    """This function greets to
    the person with the provided message"""
    print("Hello", name + ', ' + msg)

greet("Monica", "Good morning!")
```

Output

```
Hello Monica, Good morning!
```

Here, the function `greet()` has two parameters.

Since we have called this function with two arguments, it runs smoothly and we do not get any error.

If we call it with a different number of arguments, the interpreter will show an error message. Below is a call to this function with one and no arguments along with their respective error messages.

```
>>> greet("Monica")    # only one argument
TypeError: greet() missing 1 required positional argument: 'msg'

>>> greet()    # no arguments
TypeError: greet() missing 2 required positional arguments: 'name' and 'msg'
```

Variable Function Arguments

Up until now, functions had a fixed number of arguments. In Python, there are other ways to define a function that can take variable number of arguments.

Three different forms of this type are described below.

Python Default Arguments

Function arguments can have default values in Python.

We can provide a default value to an argument by using the assignment operator (=). Here is an example.

```
def greet(name, msg="Good morning!"):
    """
    This function greets to
    the person with the
    provided message.

    If the message is not provided,
    it defaults to "Good
    morning!"
    """

    print("Hello", name + ', ' + msg)


greet("Kate")
greet("Bruce", "How do you do?")
```

Output

```
Hello Kate, Good morning!
Hello Bruce, How do you do?
```

In this function, the parameter `name` does not have a default value and is required (mandatory) during a call.

On the other hand, the parameter `msg` has a default value of `"Good morning!"`. So, it is optional during a call. If a value is provided, it will overwrite the default value.

Any number of arguments in a function can have a default value. But once we have a default argument, all the arguments to its right must also have default values.

This means to say, non-default arguments cannot follow default arguments. For example, if we had defined the function header above as:

```
def greet(msg = "Good morning!", name):
```

We would get an error as:

```
SyntaxError: non-default argument follows default argument
```

Python Keyword Arguments

When we call a function with some values, these values get assigned to the arguments according to their position.

For example, in the above function `greet()`, when we called it as `greet("Bruce", "How do you do?")`, the value `"Bruce"` gets assigned to the argument *name* and similarly `"How do you do?"` to *msg*.

Python allows functions to be called using keyword arguments. When we call functions in this way, the order (position) of the arguments can be changed. Following calls to the above function are all valid and produce the same result.

```
# 2 keyword arguments
greet(name = "Bruce",msg = "How do you do?")

# 2 keyword arguments (out of order)
greet(msg = "How do you do?",name = "Bruce")

1 positional, 1 keyword argument
greet("Bruce", msg = "How do you do?")
```

As we can see, we can mix positional arguments with keyword arguments during a function call. But we must keep in mind that keyword arguments must follow positional arguments.

Having a positional argument after keyword arguments will result in errors. For example, the function call as follows:

```
greet(name="Bruce","How do you do?")
```

Will result in an error:

```
SyntaxError: non-keyword arg after keyword arg
```

Python Arbitrary Arguments

Sometimes, we do not know in advance the number of arguments that will be passed into a function. Python allows us to handle this kind of situation through function calls with an arbitrary number of arguments.

In the function definition, we use an asterisk (*) before the parameter name to denote this kind of argument. Here is an example.

```
def greet(*names):
    """This function greets all
    the person in the names tuple."""

    # names is a tuple with arguments
    for name in names:
        print("Hello", name)


greet("Monica", "Luke", "Steve", "John")
```

Output

```
Hello Monica
Hello Luke
Hello Steve
Hello John
```

Here, we have called the function with multiple arguments. These arguments get wrapped up into a tuple before being passed into the function. Inside the function, we use a `for` loop to retrieve all the arguments back.

Python Recursion

programiz.com/python-programming/recursion

In this tutorial, you will learn to create a recursive function (a function that calls itself).

What is recursion?

Recursion is the process of defining something in terms of itself.

A physical world example would be to place two parallel mirrors facing each other. Any object in between them would be reflected recursively.

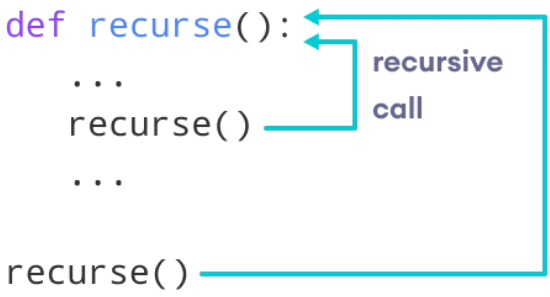
Python Recursive Function

In Python, we know that a function can call other functions. It is even possible for the function to call itself. These types of construct are termed as recursive functions.

The following image shows the working of a recursive function called `recurse`.

Following is an example of a recursive function to find the factorial of an integer.

Factorial of a number is the product of all the integers from 1 to that number. For example, the factorial of 6 (denoted as 6!) is `1*2*3*4*5*6 = 720`.



Recursive Function in Python

Example of a recursive function

```
def factorial(x):
    """This is a recursive function
    to find the factorial of an integer"""

    if x == 1:
        return 1
    else:
        return (x * factorial(x-1))


num = 3
print("The factorial of", num, "is", factorial(num))
```

Output

```
The factorial of 3 is 6
```

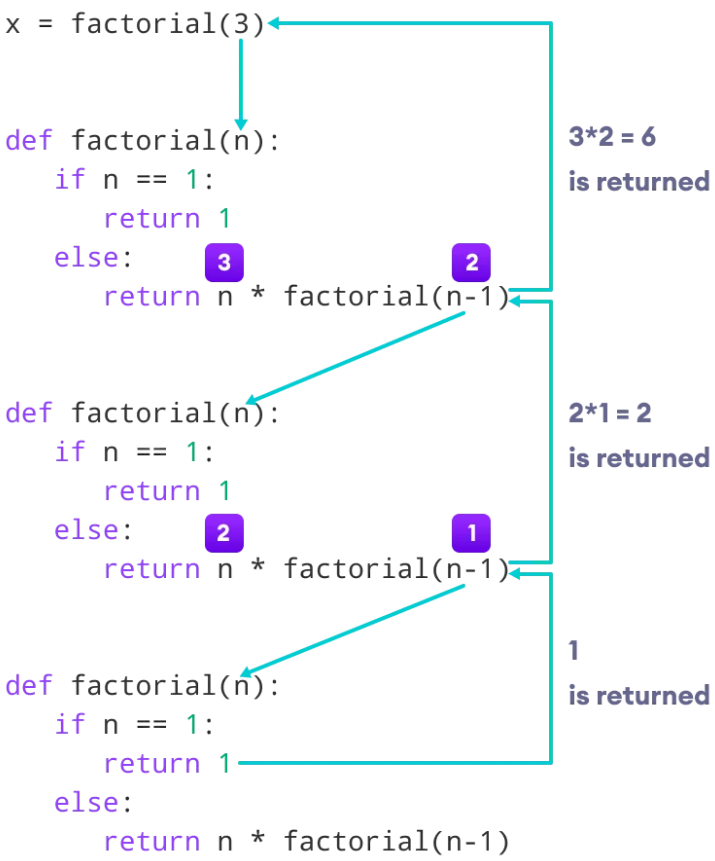
In the above example, `factorial()` is a recursive function as it calls itself.

When we call this function with a positive integer, it will recursively call itself by decreasing the number.

Each function multiplies the number with the factorial of the number below it until it is equal to one. This recursive call can be explained in the following steps.

```
factorial(3)          # 1st call with 3
3 * factorial(2)      # 2nd call with 2
3 * 2 * factorial(1)  # 3rd call with 1
3 * 2 * 1             # return from 3rd call as number=1
3 * 2                 # return from 2nd call
6                     # return from 1st call
```

Let's look at an image that shows a step-by-step process of what is going on:



Working of a recursive factorial function

Our recursion ends when the number reduces to 1. This is called the base condition.

Every recursive function must have a base condition that stops the recursion or else the function calls itself infinitely.

The Python interpreter limits the depths of recursion to help avoid infinite recursions, resulting in stack overflows.

By default, the maximum depth of recursion is `1000`. If the limit is crossed, it results in `RecursionError`. Let's look at one such condition.

```
def recursor():
    recursor()
recursor()
```

Output

```
Traceback (most recent call last):
  File "<string>", line 3, in <module>
  File "<string>", line 2, in a
  File "<string>", line 2, in a
  File "<string>", line 2, in a
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

Advantages of Recursion

1. Recursive functions make the code look clean and elegant.
2. A complex task can be broken down into simpler sub-problems using recursion.
3. Sequence generation is easier with recursion than using some nested iteration.

Disadvantages of Recursion

1. Sometimes the logic behind recursion is hard to follow through.
2. Recursive calls are expensive (inefficient) as they take up a lot of memory and time.
3. Recursive functions are hard to debug.

Python Anonymous/Lambda Function

programiz.com/python-programming/anonymous-function

In this article, you'll learn about the anonymous function, also known as lambda functions. You'll learn what they are, their syntax and how to use them (with examples).

What are lambda functions in Python?

In Python, an anonymous function is a function that is defined without a name.

While normal functions are defined using the `def` keyword in Python, anonymous functions are defined using the `lambda` keyword.

Hence, anonymous functions are also called lambda functions.

How to use lambda Functions in Python?

A lambda function in python has the following syntax.

Syntax of Lambda Function in python

```
lambda arguments: expression
```

Lambda functions can have any number of arguments but only one expression. The expression is evaluated and returned. Lambda functions can be used wherever function objects are required.

Example of Lambda Function in python

Here is an example of lambda function that doubles the input value.

```
# Program to show the use of lambda functions
double = lambda x: x * 2

print(double(5))
```

Output

```
10
```

In the above program, `lambda x: x * 2` is the lambda function. Here *x* is the argument and `x * 2` is the expression that gets evaluated and returned.

This function has no name. It returns a function object which is assigned to the identifier `double`. We can now call it as a normal function. The statement

```
double = lambda x: x * 2
```

is nearly the same as:

```
def double(x):
   return x * 2
```

Use of Lambda Function in python

We use lambda functions when we require a nameless function for a short period of time.

In Python, we generally use it as an argument to a higher-order function (a function that takes in other functions as arguments). Lambda functions are used along with built-in functions like `filter()`, `map()` etc.

Example use with filter()

The `filter()` function in Python takes in a function and a list as arguments.

The function is called with all the items in the list and a new list is returned which contains items for which the function evaluates to `True`.

Here is an example use of `filter()` function to filter out only even numbers from a list.

```
# Program to filter out only the even items from a list
my_list = [1, 5, 4, 6, 8, 11, 3, 12]

new_list = list(filter(lambda x: (x%2 == 0) , my_list))

print(new_list)
```

Output

```
[4, 6, 8, 12]
```

Example use with map()

The `map()` function in Python takes in a function and a list.

The function is called with all the items in the list and a new list is returned which contains items returned by that function for each item.

Here is an example use of `map()` function to double all the items in a list.

```
# Program to double each item in a list using map()

my_list = [1, 5, 4, 6, 8, 11, 3, 12]

new_list = list(map(lambda x: x * 2 , my_list))

print(new_list)
```

Output

```
[2, 10, 8, 12, 16, 22, 6, 24]
```

Python Global, Local and Nonlocal variables

programiz.com/python-programming/global-local-nonlocal-variables

In this tutorial, you'll learn about Python Global variables, Local variables, Nonlocal variables and where to use them.

Global Variables

In Python, a variable declared outside of the function or in global scope is known as a global variable. This means that a global variable can be accessed inside or outside of the function.

Let's see an example of how a global variable is created in Python.

Example 1: Create a Global Variable

```
x = "global"

def foo():
    print("x inside:", x)


foo()
print("x outside:", x)
```

Output

```
x inside: global
x outside: global
```

In the above code, we created *x* as a global variable and defined a `foo()` to print the global variable *x*. Finally, we call the `foo()` which will print the value of *x*.

What if you want to change the value of *x* inside a function?

```
x = "global"

def foo():
    x = x * 2
    print(x)

foo()
```

Output

```
UnboundLocalError: local variable 'x' referenced before assignment
```

The output shows an error because Python treats *x* as a local variable and *x* is also not defined inside `foo()`.

To make this work, we use the `global` keyword. Visit Python Global Keyword to learn more.

Local Variables

A variable declared inside the function's body or in the local scope is known as a local variable.

Example 2: Accessing local variable outside the scope

```
def foo():
    y = "local"


foo()
print(y)
```

Output

```
NameError: name 'y' is not defined
```

The output shows an error because we are trying to access a local variable *y* in a global scope whereas the local variable only works inside `foo()` or local scope.

Let's see an example on how a local variable is created in Python.

Example 3: Create a Local Variable

Normally, we declare a variable inside the function to create a local variable.

```
def foo():
    y = "local"
    print(y)

foo()
```

Output

```
local
```

Let's take a look at the earlier problem where *x* was a global variable and we wanted to modify *x* inside `foo()`.

Global and local variables

Here, we will show how to use global variables and local variables in the same code.

Example 4: Using Global and Local variables in the same code

```
x = "global "

def foo():
    global x
    y = "local"
    x = x * 2
    print(x)
    print(y)

foo()
```

Output

```
global global
local
```

In the above code, we declare *x* as a global and *y* as a local variable in the `foo()`. Then, we use multiplication operator `*` to modify the global variable *x* and we print both *x* and *y*.

After calling the `foo()`, the value of *x* becomes `global global` because we used the `x * 2` to print two times `global`. After that, we print the value of local variable *y* i.e `local`.

Example 5: Global variable and Local variable with same name

```
x = 5

def foo():
    x = 10
    print("local x:", x)


foo()
print("global x:", x)
```

Output

```
local x: 10
global x: 5
```

In the above code, we used the same name *x* for both global variable and local variable. We get a different result when we print the same variable because the variable is declared in both scopes, i.e. the local scope inside `foo()` and global scope outside `foo()`.

When we print the variable inside `foo()` it outputs `local x: 10`. This is called the local scope of the variable.

Similarly, when we print the variable outside the `foo()`, it outputs `global x: 5`. This is called the global scope of the variable.

Nonlocal Variables

Nonlocal variables are used in nested functions whose local scope is not defined. This means that the variable can be neither in the local nor the global scope.

Let's see an example of how a nonlocal variable is used in Python.

We use `nonlocal` keywords to create nonlocal variables.

Example 6: Create a nonlocal variable

```
def outer():
    x = "local"

    def inner():
        nonlocal x
        x = "nonlocal"
        print("inner:", x)

    inner()
    print("outer:", x)


outer()
```

Output

```
inner: nonlocal
outer: nonlocal
```

In the above code, there is a nested `inner()` function. We use `nonlocal` keywords to create a nonlocal variable. The `inner()` function is defined in the scope of another function `outer()`.

Note : If we change the value of a nonlocal variable, the changes appear in the local variable.

Python Global Keyword

programiz.com/python-programming/global-keyword

In this article, you'll learn about the global keyword, global variable and when to use global keywords.

Before reading this article, make sure you have got some basics of Python Global, Local and Nonlocal Variables.

What is the global keyword

In Python, `global` keyword allows you to modify the variable outside of the current scope. It is used to create a global variable and make changes to the variable in a local context.

Rules of global Keyword

The basic rules for `global` keyword in Python are:

- When we create a variable inside a function, it is local by default.
- When we define a variable outside of a function, it is global by default. You don't have to use `global` keyword.
- We use `global` keyword to read and write a global variable inside a function.
- Use of `global` keyword outside a function has no effect.

Use of global Keyword

Let's take an example.

Example 1: Accessing global Variable From Inside a Function

```
c = 1 # global variable

def add():
    print(c)

add()
```

When we run the above program, the output will be:

```
1
```

However, we may have some scenarios where we need to modify the global variable from inside a function.

Example 2: Modifying Global Variable From Inside the Function

```
c = 1 # global variable

def add():
    c = c + 2 # increment c by 2
    print(c)

add()
```

When we run the above program, the output shows an error:

```
UnboundLocalError: local variable 'c' referenced before assignment
```

This is because we can only access the global variable but cannot modify it from inside the function.

The solution for this is to use the `global` keyword.

Example 3: Changing Global Variable From Inside a Function using global

```
c = 0 # global variable

def add():
    global c
    c = c + 2 # increment by 2
    print("Inside add():", c)

add()
print("In main:", c)
```

When we run the above program, the output will be:

```
Inside add(): 2
In main: 2
```

In the above program, we define *c* as a global keyword inside the `add()` function.

Then, we increment the variable *c* by **2**, i.e `c = c + 2`. After that, we call the `add()` function. Finally, we print the global variable *c*.

As we can see, change also occurred on the global variable outside the function, `c = 2`.

Global Variables Across Python Modules

In Python, we create a single module `config.py` to hold global variables and share information across Python modules within the same program.

Here is how we can share global variables across the python modules.

Example 4: Share a global Variable Across Python Modules

Create a `config.py` file, to store global variables

```
a = 0
b = "empty"
```

Create a `update.py` file, to change global variables

```
import config

config.a = 10
config.b = "alphabet"
```

Create a `main.py` file, to test changes in value

```
import config
import update

print(config.a)
print(config.b)
```

When we run the `main.py` file, the output will be

```
10
alphabet
```

In the above, we have created three files: `config.py`, `update.py`, and `main.py`.

The module `config.py` stores global variables of *a* and *b*. In the `update.py` file, we import the `config.py` module and modify the values of *a* and *b*. Similarly, in the `main.py` file, we import both `config.py` and `update.py` module. Finally, we print and test the values of global variables whether they are changed or not.

Global in Nested Functions

Here is how you can use a global variable in nested function.

Example 5: Using a Global Variable in Nested Function

```
def foo():
    x = 20

    def bar():
        global x
        x = 25

    print("Before calling bar: ", x)
    print("Calling bar now")
    bar()
    print("After calling bar: ", x)

foo()
print("x in main: ", x)
```

The output is :

```
Before calling bar: 20
Calling bar now
After calling bar: 20
x in main: 25
```

In the above program, we declared a global variable inside the nested function `bar()`. Inside `foo()` function, *x* has no effect of the global keyword.

Before and after calling `bar()`, the variable *x* takes the value of local variable i.e `x = 20`. Outside of the `foo()` function, the variable *x* will take value defined in the `bar()` function i.e `x = 25`. This is because we have used `global` keyword in *x* to create global variable inside the `bar()` function (local scope).

If we make any changes inside the `bar()` function, the changes appear outside the local scope, i.e. `foo()`.

Python Modules

programiz.com/python-programming/modules

In this article, you will learn to create and import custom modules in Python. Also, you will find different techniques to import and use custom and built-in modules in Python.

What are modules in Python?

Modules refer to a file containing Python statements and definitions.

A file containing Python code, for example: `example.py`, is called a module, and its module name would be `example`.

We use modules to break down large programs into small manageable and organized files. Furthermore, modules provide reusability of code.

We can define our most used functions in a module and import it, instead of copying their definitions into different programs.

Let us create a module. Type the following and save it as `example.py`.

```
# Python Module example

def add(a, b):
   """This program adds two
   numbers and return the result"""

   result = a + b
   return result
```

Here, we have defined a function `add()` inside a module named `example`. The function takes in two numbers and returns their sum.

How to import modules in Python?

We can import the definitions inside a module to another module or the interactive interpreter in Python.

We use the `import` keyword to do this. To import our previously defined module `example`, we type the following in the Python prompt.

```
>>> import example
```

This does not import the names of the functions defined in `example` directly in the current symbol table. It only imports the module name `example` there.

Using the module name we can access the function using the dot `.` operator. For example:

```
>>> example.add(4,5.5)
9.5
```

Python has tons of standard modules. You can check out the full list of Python standard modules and their use cases. These files are in the Lib directory inside the location where you installed Python.

Standard modules can be imported the same way as we import our user-defined modules.

There are various ways to import modules. They are listed below..

Python import statement

We can import a module using the `import` statement and access the definitions inside it using the dot operator as described above. Here is an example.

```
# import statement example
# to import standard module math

import math
print("The value of pi is", math.pi)
```

When you run the program, the output will be:

```
The value of pi is 3.141592653589793
```

Import with renaming

We can import a module by renaming it as follows:

```
# import module by renaming it

import math as m
print("The value of pi is", m.pi)
```

We have renamed the `math` module as `m`. This can save us typing time in some cases.

Note that the name `math` is not recognized in our scope. Hence, `math.pi` is invalid, and `m.pi` is the correct implementation.

Python from...import statement

We can import specific names from a module without importing the module as a whole. Here is an example.

```
# import only pi from math module

from math import pi
print("The value of pi is", pi)
```

Here, we imported only the `pi` attribute from the `math` module.

In such cases, we don't use the dot operator. We can also import multiple attributes as follows:

```
>>> from math import pi, e
>>> pi
3.141592653589793
>>> e
2.718281828459045
```

Import all names

We can import all names(definitions) from a module using the following construct:

```
# import all names from the standard module math

from math import *
print("The value of pi is", pi)
```

Here, we have imported all the definitions from the math module. This includes all names visible in our scope except those beginning with an underscore(private definitions).

Importing everything with the asterisk (*) symbol is not a good programming practice. This can lead to duplicate definitions for an identifier. It also hampers the readability of our code.

Python Module Search Path

While importing a module, Python looks at several places. Interpreter first looks for a built-in module. Then(if built-in module not found), Python looks into a list of directories defined in `sys.path`. The search is in this order.

- The current directory.
- `PYTHONPATH` (an environment variable with a list of directories).
- The installation-dependent default directory.

```
>>> import sys
>>> sys.path
['',
'C:\\Python33\\Lib\\idlelib',
'C:\\Windows\\system32\\python33.zip',
'C:\\Python33\\DLLs',
'C:\\Python33\\lib',
'C:\\Python33',
'C:\\Python33\\lib\\site-packages']
```

We can add and modify this list to add our own path.

Reloading a module

The Python interpreter imports a module only once during a session. This makes things more efficient. Here is an example to show how this works.

Suppose we have the following code in a module named `my_module`.

```
# This module shows the effect of
#  multiple imports and reload

print("This code got executed")
```

Now we see the effect of multiple imports.

```
>>> import my_module
This code got executed
>>> import my_module
>>> import my_module
```

We can see that our code got executed only once. This goes to say that our module was imported only once.

Now if our module changed during the course of the program, we would have to reload it.One way to do this is to restart the interpreter. But this does not help much.

Python provides a more efficient way of doing this. We can use the `reload()` function inside the `imp` module to reload a module. We can do it in the following ways:

```
>>> import imp
>>> import my_module
This code got executed
>>> import my_module
>>> imp.reload(my_module)
This code got executed
<module 'my_module' from '.\\my_module.py'>
```

The dir() built-in function

We can use the `dir()` function to find out names that are defined inside a module.

For example, we have defined a function `add()` in the module `example` that we had in the beginning.

We can use `dir` in `example` module in the following way:

```
>>> dir(example)
['__builtins__',
'__cached__',
'__doc__',
'__file__',
'__initializing__',
'__loader__',
'__name__',
'__package__',
'add']
```

Here, we can see a sorted list of names (along with `add`). All other names that begin with an underscore are default Python attributes associated with the module (not user-defined).

For example, the `__name__` attribute contains the name of the module.

```
>>> import example
>>> example.__name__
'example'
```

All the names defined in our current namespace can be found out using the `dir()` function without any arguments.

```
>>> a = 1
>>> b = "hello"
>>> import math
>>> dir()
['__builtins__', '__doc__', '__name__', 'a', 'b', 'math', 'pyscripter']
```

Python Package

programiz.com/python-programming/package

In this article, you'll learn to divide your code base into clean, efficient modules using Python packages. Also, you'll learn to import and use your own or third party packages in a Python program.

What are packages?

We don't usually store all of our files on our computer in the same location. We use a well-organized hierarchy of directories for easier access.

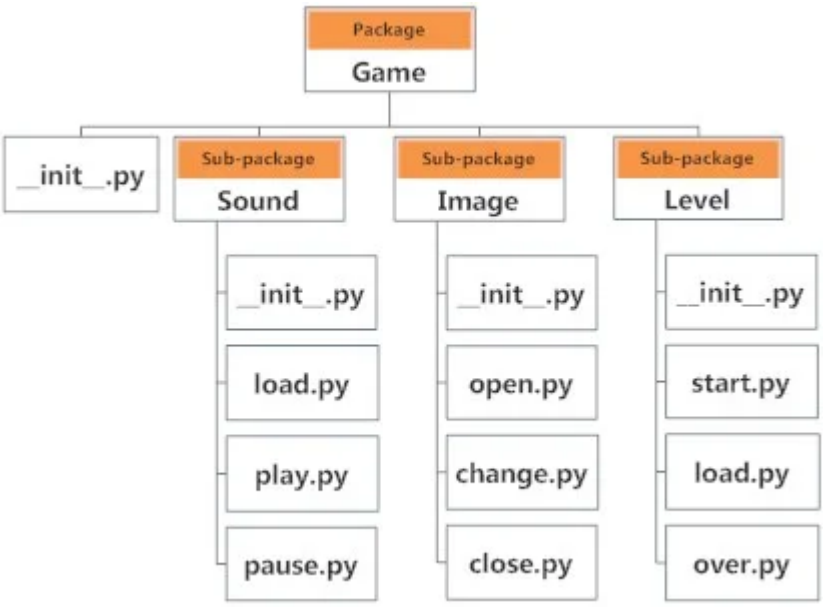
Similar files are kept in the same directory, for example, we may keep all the songs in the "**music**" directory. Analogous to this, Python has packages for directories and modules for files.

As our application program grows larger in size with a lot of modules, we place similar modules in one package and different modules in different packages. This makes a project (program) easy to manage and conceptually clear.

Similarly, as a directory can contain subdirectories and files, a Python package can have sub-packages and modules.

A directory must contain a file named `__init__.py` in order for Python to consider it as a package. This file can be left empty but we generally place the initialization code for that package in this file.

Here is an example. Suppose we are developing a game. One possible organization of packages and modules could be as shown in the figure below.



Package Module Structure in Python Programming

Importing module from a package

We can import modules from packages using the dot (.) operator.

For example, if we want to import the `start` module in the above example, it can be done as follows:

```
import Game.Level.start
```

Now, if this module contains a function named `select_difficulty()`, we must use the full name to reference it.

```
Game.Level.start.select_difficulty(2)
```

If this construct seems lengthy, we can import the module without the package prefix as follows:

```
from Game.Level import start
```

We can now call the function simply as follows:

```
start.select_difficulty(2)
```

Another way of importing just the required function (or class or variable) from a module within a package would be as follows:

```
from Game.Level.start import select_difficulty
```

Now we can directly call this function.

```
select_difficulty(2)
```

Although easier, this method is not recommended. Using the full namespace avoids confusion and prevents two same identifier names from colliding.

While importing packages, Python looks in the list of directories defined in `sys.path`, similar as for module search path.

Python Numbers, Type Conversion and Mathematics

programiz.com/python-programming/numbers

In this article, you'll learn about the different numbers used in Python, how to convert from one data type to the other, and the mathematical operations supported in Python.

Number Data Type in Python

Python supports integers, floating-point numbers and complex numbers. They are defined as `int`, `float`, and `complex` classes in Python.

Integers and floating points are separated by the presence or absence of a decimal point. For instance, 5 is an integer whereas 5.0 is a floating-point number.

Complex numbers are written in the form, `x + yj`, where *x* is the real part and *y* is the imaginary part.

We can use the `type()` function to know which class a variable or a value belongs to and `isinstance()` function to check if it belongs to a particular class.

Let's look at an example:

```
a = 5

print(type(a))

print(type(5.0))

c = 5 + 3j
print(c + 3)

print(isinstance(c, complex))
```

When we run the above program, we get the following output:

```
<class 'int'>
<class 'float'>
(8+3j)
True
```

While integers can be of any length, a floating-point number is accurate only up to 15 decimal places (the 16th place is inaccurate).

The numbers we deal with every day are of the decimal (base 10) number system. But computer programmers (generally embedded programmers) need to work with binary (base 2), hexadecimal (base 16) and octal (base 8) number systems.

In Python, we can represent these numbers by appropriately placing a prefix before that number. The following table lists these prefixes.

Number System	Prefix
Binary	'0b' or '0B'
Octal	'0o' or '0O'
Hexadecimal	'0x' or '0X'

Here are some examples

```
# Output: 107
print(0b1101011)

# Output: 253 (251 + 2)
print(0xFB + 0b10)

# Output: 13
print(0o15)
```

When you run the program, the output will be:

```
107
253
13
```

Type Conversion

We can convert one type of number into another. This is also known as coercion.

Operations like addition, subtraction coerce integer to float implicitly (automatically), if one of the operands is float.

```
>>> 1 + 2.0
3.0
```

We can see above that 1 (integer) is coerced into 1.0 (float) for addition and the result is also a floating point number.

We can also use built-in functions like `int()`, `float()` and `complex()` to convert between types explicitly. These functions can even convert from strings.

```
>>> int(2.3)
2
>>> int(-2.8)
-2
>>> float(5)
5.0
>>> complex('3+5j')
(3+5j)
```

When converting from float to integer, the number gets truncated (decimal parts are removed).

Python Decimal

Python built-in class float performs some calculations that might amaze us. We all know that the sum of 1.1 and 2.2 is 3.3, but Python seems to disagree.

```
>>> (1.1 + 2.2) == 3.3
False
```

What is going on?

It turns out that floating-point numbers are implemented in computer hardware as binary fractions as the computer only understands binary (0 and 1). Due to this reason, most of the decimal fractions we know, cannot be accurately stored in our computer.

Let's take an example. We cannot represent the fraction 1/3 as a decimal number. This will give 0.33333333... which is infinitely long, and we can only approximate it.

It turns out that the decimal fraction 0.1 will result in an infinitely long binary fraction of 0.0001100110011001100110011... and our computer only stores a finite number of it.

This will only approximate 0.1 but never be equal. Hence, it is the limitation of our computer hardware and not an error in Python.

```
>>> 1.1 + 2.2
3.3000000000000003
```

To overcome this issue, we can use the decimal module that comes with Python. While floating-point numbers have precision up to 15 decimal places, the decimal module has user-settable precision.

Let's see the difference:

```
import decimal

print(0.1)

print(decimal.Decimal(0.1))
```

Output

```
0.1
0.1000000000000000055511151231257827021181583404541015625
```

This module is used when we want to carry out decimal calculations as we learned in school.

It also preserves significance. We know 25.50 kg is more accurate than 25.5 kg as it has two significant decimal places compared to one.

```
from decimal import Decimal as D

print(D('1.1') + D('2.2'))

print(D('1.2') * D('2.50'))
```

Output

```
3.3
3.000
```

Notice the trailing zeroes in the above example.

We might ask, why not implement `Decimal` every time, instead of float? The main reason is efficiency. Floating point operations are carried out must faster than `Decimal` operations.

When to use Decimal instead of float?

We generally use Decimal in the following cases.

- When we are making financial applications that need exact decimal representation.
- When we want to control the level of precision required.
- When we want to implement the notion of significant decimal places.

Python Fractions

Python provides operations involving fractional numbers through its `fractions` module.

A fraction has a numerator and a denominator, both of which are integers. This module has support for rational number arithmetic.

We can create Fraction objects in various ways. Let's have a look at them.

```
import fractions

print(fractions.Fraction(1.5))

print(fractions.Fraction(5))

print(fractions.Fraction(1,3))
```

Output

```
3/2
5
1/3
```

While creating `Fraction` from `float`, we might get some unusual results. This is due to the imperfect binary floating point number representation as discussed in the previous section.

Fortunately, `Fraction` allows us to instantiate with string as well. This is the preferred option when using decimal numbers.

```
import fractions

# As float
# Output: 2476979795053773/2251799813685248
print(fractions.Fraction(1.1))

# As string
# Output: 11/10
print(fractions.Fraction('1.1'))
```

Output

```
2476979795053773/2251799813685248
11/10
```

This data type supports all basic operations. Here are a few examples.

```
from fractions import Fraction as F

print(F(1, 3) + F(1, 3))

print(1 / F(5, 6))

print(F(-3, 10) > 0)

print(F(-3, 10) < 0)
```

Output

```
2/3
6/5
False
True
```

Python Mathematics

Python offers modules like `math` and `random` to carry out different mathematics like trigonometry, logarithms, probability and statistics, etc.

```
import math

print(math.pi)

print(math.cos(math.pi))

print(math.exp(10))

print(math.log10(1000))

print(math.sinh(1))

print(math.factorial(6))
```

Output

```
3.141592653589793
-1.0
22026.465794806718
3.0
1.1752011936438014
720
```

Here is the full list of functions and attributes available in the Python math module.

```
import random

print(random.randrange(10, 20))

x = ['a', 'b', 'c', 'd', 'e']

# Get random choice
print(random.choice(x))

# Shuffle x
random.shuffle(x)

# Print the shuffled x
print(x)

# Print random element
print(random.random())
```

When we run the above program we get the output as follows.(Values may be different due to the random behavior)

```
18
e
['c', 'e', 'd', 'b', 'a']
0.5682821194654443
```

Here is the full list of functions and attributes available in the Python random module.

Python List

programiz.com/python-programming/list

In this tutorial, we'll learn everything about Python lists: creating lists, changing list elements, removing elements, and other list operations with the help of examples.

Python lists are one of the most versatile data types that allow us to work with multiple elements at once. For example,

```
# a list of programming languages
['Python', 'C++', 'JavaScript']
```

Create Python Lists

In Python, a list is created by placing elements inside square brackets `[]`, separated by commas.

```
# list of integers
my_list = [1, 2, 3]
```

A list can have any number of items and they may be of different types (integer, float, string, etc.).

```
# empty list
my_list = []

# list with mixed data types
my_list = [1, "Hello", 3.4]
```

A list can also have another list as an item. This is called a nested list.

```
# nested list
my_list = ["mouse", [8, 4, 6], ['a']]
```

Access List Elements

There are various ways in which we can access the elements of a list.

List Index

We can use the index operator `[]` to access an item in a list. In Python, indices start at 0. So, a list having 5 elements will have an index from 0 to 4.

Trying to access indexes other than these will raise an `IndexError`. The index must be an integer. We can't use float or other types, this will result in `TypeError`.

Nested lists are accessed using nested indexing.

```
my_list = ['p', 'r', 'o', 'b', 'e']

# first item
print(my_list[0])  # p

# third item
print(my_list[2])  # o

# fifth item
print(my_list[4])  # e

# Nested List
n_list = ["Happy", [2, 0, 1, 5]]

# Nested indexing
print(n_list[0][1])

print(n_list[1][3])

# Error! Only integer can be used for indexing
print(my_list[4.0])
```

Output

```
p
o
e
a
5
Traceback (most recent call last):
  File "<string>", line 21, in <module>
TypeError: list indices must be integers or slices, not float
```

Negative indexing

Python allows negative indexing for its sequences. The index of -1 refers to the last item, -2 to the second last item and so on.

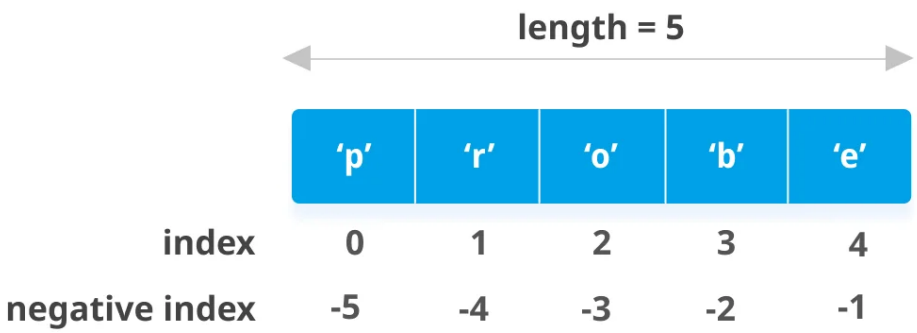
```
# Negative indexing in lists
my_list = ['p','r','o','b','e']

# last item
print(my_list[-1])

# fifth last item
print(my_list[-5])
```

Output

```
e
p
```



List indexing in Python

List Slicing in Python

We can access a range of items in a list by using the slicing operator `:`.

```
# List slicing in Python

my_list = ['p','r','o','g','r','a','m','i','z']

# elements from index 2 to index 4
print(my_list[2:5])

# elements from index 5 to end
print(my_list[5:])

# elements beginning to end
print(my_list[:])
```

Output

```
['o', 'g', 'r']
['a', 'm', 'i', 'z']
['p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z']
```

Note: When we slice lists, the start index is inclusive but the end index is exclusive. For example, `my_list[2: 5]` returns a list with elements at index 2, 3 and 4, but not 5.

Add/Change List Elements

Lists are mutable, meaning their elements can be changed unlike string or tuple.

We can use the assignment operator `=` to change an item or a range of items.

```
# Correcting mistake values in a list
odd = [2, 4, 6, 8]

# change the 1st item
odd[0] = 1

print(odd)

# change 2nd to 4th items
odd[1:4] = [3, 5, 7]

print(odd)
```

Output

```
[1, 4, 6, 8]
[1, 3, 5, 7]
```

We can add one item to a list using the `append()` method or add several items using the `extend()` method.

```
# Appending and Extending lists in Python
odd = [1, 3, 5]

odd.append(7)

print(odd)

odd.extend([9, 11, 13])

print(odd)
```

Output

```
[1, 3, 5, 7]
[1, 3, 5, 7, 9, 11, 13]
```

We can also use `+` operator to combine two lists. This is also called concatenation.

The `*` operator repeats a list for the given number of times.

```
# Concatenating and repeating lists
odd = [1, 3, 5]

print(odd + [9, 7, 5])

print(["re"] * 3)
```

Output

```
[1, 3, 5, 9, 7, 5]
['re', 're', 're']
```

Furthermore, we can insert one item at a desired location by using the method `insert()` or insert multiple items by squeezing it into an empty slice of a list.

```
# Demonstration of list insert() method
odd = [1, 9]
odd.insert(1,3)

print(odd)

odd[2:2] = [5, 7]

print(odd)
```

Output

```
[1, 3, 9]
[1, 3, 5, 7, 9]
```

Delete List Elements

We can delete one or more items from a list using the Python del statement. It can even delete the list entirely.

```
# Deleting list items
my_list = ['p', 'r', 'o', 'b', 'l', 'e', 'm']

# delete one item
del my_list[2]

print(my_list)

# delete multiple items
del my_list[1:5]

print(my_list)

# delete the entire list
del my_list

# Error: List not defined
print(my_list)
```

Output

```
['p', 'r', 'b', 'l', 'e', 'm']
['p', 'm']
Traceback (most recent call last):
  File "<string>", line 18, in <module>
NameError: name 'my_list' is not defined
```

We can use `remove()` to remove the given item or `pop()` to remove an item at the given index.

The `pop()` method removes and returns the last item if the index is not provided. This helps us implement lists as stacks (first in, last out data structure).

And, if we have to empty the whole list, we can use the `clear()` method.

```
my_list = ['p','r','o','b','l','e','m']
my_list.remove('p')

# Output: ['r', 'o', 'b', 'l', 'e', 'm']
print(my_list)

# Output: 'o'
print(my_list.pop(1))

# Output: ['r', 'b', 'l', 'e', 'm']
print(my_list)

# Output: 'm'
print(my_list.pop())

# Output: ['r', 'b', 'l', 'e']
print(my_list)

my_list.clear()

# Output: []
print(my_list)
```

Output

```
['r', 'o', 'b', 'l', 'e', 'm']
o
['r', 'b', 'l', 'e', 'm']
m
['r', 'b', 'l', 'e']
[]
```

Finally, we can also delete items in a list by assigning an empty list to a slice of elements.

```
>>> my_list = ['p','r','o','b','l','e','m']
>>> my_list[2:3] = []
>>> my_list
['p', 'r', 'b', 'l', 'e', 'm']
>>> my_list[2:5] = []
>>> my_list
['p', 'r', 'm']
```

Python List Methods

Python has many useful list methods that makes it really easy to work with lists. Here are some of the commonly used list methods.

Methods	Descriptions
append()	adds an element to the end of the list
extend()	adds all elements of a list to another list
insert()	inserts an item at the defined index

remove()	removes an item from the list
pop()	returns and removes an element at the given index
clear()	removes all items from the list
index()	returns the index of the first matched item
count()	returns the count of the number of items passed as an argument
sort()	sort items in a list in ascending order
reverse()	reverse the order of items in the list
copy()	returns a shallow copy of the list

```
# Example on Python list methods

my_list = [3, 8, 1, 6, 8, 8, 4]

# Add 'a' to the end
my_list.append('a')

# Output: [3, 8, 1, 6, 0, 8, 4, 'a']
print(my_list)

# Index of first occurrence of 8
print(my_list.index(8))   # Output: 1

# Count of 8 in the list
print(my_list.count(8))  # Output: 3
```

List Comprehension: Elegant way to create Lists

List comprehension is an elegant and concise way to create a new list from an existing list in Python.

A list comprehension consists of an expression followed by for statement inside square brackets.

Here is an example to make a list with each item being increasing power of 2.

```
pow2 = [2 ** x for x in range(10)]
print(pow2)
```

Output

```
[1, 2, 4, 8, 16, 32, 64, 128, 256, 512]
```

This code is equivalent to:

```
pow2 = []
for x in range(10):
   pow2.append(2 ** x)
```

A list comprehension can optionally contain more `for` or if statements. An optional `if` statement can filter out items for the new list. Here are some examples.

```
>>> pow2 = [2 ** x for x in range(10) if x > 5]
>>> pow2
[64, 128, 256, 512]
>>> odd = [x for x in range(20) if x % 2 == 1]
>>> odd
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
>>> [x+y for x in ['Python ','C '] for y in ['Language','Programming']]
['Python Language', 'Python Programming', 'C Language', 'C Programming']
```

Visit Python list comprehension to learn more.

Other List Operations in Python

List Membership Test

We can test if an item exists in a list or not, using the keyword `in` .

```
my_list = ['p', 'r', 'o', 'b', 'l', 'e', 'm']

# Output: True
print('p' in my_list)

# Output: False
print('a' in my_list)

# Output: True
print('c' not in my_list)
```

Output

```
True
False
True
```

Iterating Through a List

Using a `for` loop we can iterate through each item in a list.

```
for fruit in ['apple','banana','mango']:
    print("I like",fruit)
```

Output

```
I like apple
I like banana
I like mango
```

Python Tuple

programiz.com/python-programming/tuple

In this article, you'll learn everything about Python tuples. More specifically, what are tuples, how to create them, when to use them and various methods you should be familiar with.

A tuple in Python is similar to a list. The difference between the two is that we cannot change the elements of a tuple once it is assigned whereas we can change the elements of a list.

Creating a Tuple

A tuple is created by placing all the items (elements) inside parentheses `()`, separated by commas. The parentheses are optional, however, it is a good practice to use them.

A tuple can have any number of items and they may be of different types (integer, float, list, string, etc.).

```
# Different types of tuples

# Empty tuple
my_tuple = ()
print(my_tuple)

# Tuple having integers
my_tuple = (1, 2, 3)
print(my_tuple)

# tuple with mixed datatypes
my_tuple = (1, "Hello", 3.4)
print(my_tuple)

# nested tuple
my_tuple = ("mouse", [8, 4, 6], (1, 2, 3))
print(my_tuple)
```

Output

```
()
(1, 2, 3)
(1, 'Hello', 3.4)
('mouse', [8, 4, 6], (1, 2, 3))
```

A tuple can also be created without using parentheses. This is known as tuple packing.

```
my_tuple = 3, 4.6, "dog"
print(my_tuple)

# tuple unpacking is also possible
a, b, c = my_tuple

print(a)      # 3
print(b)      # 4.6
print(c)      # dog
```

Output

```
(3, 4.6, 'dog')
3
4.6
dog
```

Creating a tuple with one element is a bit tricky.

Having one element within parentheses is not enough. We will need a trailing comma to indicate that it is, in fact, a tuple.

```
my_tuple = ("hello")
print(type(my_tuple))  # <class 'str'>

# Creating a tuple having one element
my_tuple = ("hello",)
print(type(my_tuple))  # <class 'tuple'>

# Parentheses is optional
my_tuple = "hello",
print(type(my_tuple))  # <class 'tuple'>
```

Output

```
<class 'str'>
<class 'tuple'>
<class 'tuple'>
```

Access Tuple Elements

There are various ways in which we can access the elements of a tuple.

1. Indexing

We can use the index operator `[]` to access an item in a tuple, where the index starts from 0.

So, a tuple having 6 elements will have indices from 0 to 5. Trying to access an index outside of the tuple index range(6,7,... in this example) will raise an `IndexError`.

The index must be an integer, so we cannot use float or other types. This will result in `TypeError`.

Likewise, nested tuples are accessed using nested indexing, as shown in the example below.

```
# Accessing tuple elements using indexing
my_tuple = ('p','e','r','m','i','t')

print(my_tuple[0])   # 'p'
print(my_tuple[5])   # 't'

# IndexError: list index out of range
# print(my_tuple[6])

# Index must be an integer
# TypeError: list indices must be integers, not float
# my_tuple[2.0]

# nested tuple
n_tuple = ("mouse", [8, 4, 6], (1, 2, 3))

# nested index
print(n_tuple[0][3])       # 's'
print(n_tuple[1][1])       # 4
```

Output

```
p
t
s
4
```

---

2. Negative Indexing

Python allows negative indexing for its sequences.

The index of -1 refers to the last item, -2 to the second last item and so on.

```
# Negative indexing for accessing tuple elements
my_tuple = ('p', 'e', 'r', 'm', 'i', 't')

# Output: 't'
print(my_tuple[-1])

# Output: 'p'
print(my_tuple[-6])
```

Output

```
t
p
```

---

3. Slicing

We can access a range of items in a tuple by using the slicing operator colon `:`.

```
# Accessing tuple elements using slicing
my_tuple = ('p','r','o','g','r','a','m','i','z')

# elements 2nd to 4th
# Output: ('r', 'o', 'g')
print(my_tuple[1:4])

# elements beginning to 2nd
# Output: ('p', 'r')
print(my_tuple[:-7])

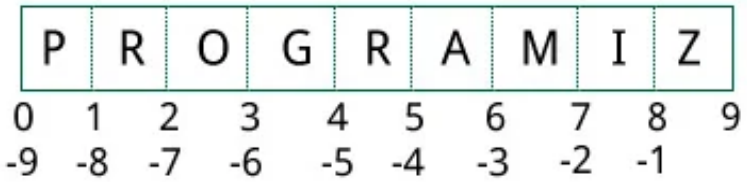
# elements 8th to end
# Output: ('i', 'z')
print(my_tuple[7:])

# elements beginning to end
# Output: ('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')
print(my_tuple[:])
```

Output

```
('r', 'o', 'g')
('p', 'r')
('i', 'z')
('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')
```

Slicing can be best visualized by considering the index to be between the elements as shown below. So if we want to access a range, we need the index that will slice the portion from the tuple.



Element Slicing in Python

Changing a Tuple

Unlike lists, tuples are immutable.

This means that elements of a tuple cannot be changed once they have been assigned. But, if the element is itself a mutable data type like a list, its nested items can be changed.

We can also assign a tuple to different values (reassignment).

```
# Changing tuple values
my_tuple = (4, 2, 3, [6, 5])


# TypeError: 'tuple' object does not support item assignment
# my_tuple[1] = 9

# However, item of mutable element can be changed
my_tuple[3][0] = 9    # Output: (4, 2, 3, [9, 5])
print(my_tuple)

# Tuples can be reassigned
my_tuple = ('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')

# Output: ('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')
print(my_tuple)
```

Output

```
(4, 2, 3, [9, 5])
('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')
```

We can use `+` operator to combine two tuples. This is called **concatenation**.

We can also **repeat** the elements in a tuple for a given number of times using the `*` operator.

Both `+` and `*` operations result in a new tuple.

```
# Concatenation
# Output: (1, 2, 3, 4, 5, 6)
print((1, 2, 3) + (4, 5, 6))

# Repeat
# Output: ('Repeat', 'Repeat', 'Repeat')
print(("Repeat",) * 3)
```

Output

```
(1, 2, 3, 4, 5, 6)
('Repeat', 'Repeat', 'Repeat')
```

Deleting a Tuple

As discussed above, we cannot change the elements in a tuple. It means that we cannot delete or remove items from a tuple.

Deleting a tuple entirely, however, is possible using the keyword del.

```
# Deleting tuples
my_tuple = ('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')

# can't delete items
# TypeError: 'tuple' object doesn't support item deletion
# del my_tuple[3]

# Can delete an entire tuple
del my_tuple

# NameError: name 'my_tuple' is not defined
print(my_tuple)
```

Output

```
Traceback (most recent call last):
  File "<string>", line 12, in <module>
NameError: name 'my_tuple' is not defined
```

Tuple Methods

Methods that add items or remove items are not available with tuple. Only the following two methods are available.

Some examples of Python tuple methods:

```
my_tuple = ('a', 'p', 'p', 'l', 'e',)

print(my_tuple.count('p'))  # Output: 2
print(my_tuple.index('l'))  # Output: 3
```

Output

```
2
3
```

Other Tuple Operations

1. Tuple Membership Test

We can test if an item exists in a tuple or not, using the keyword `in`.

```
# Membership test in tuple
my_tuple = ('a', 'p', 'p', 'l', 'e',)

# In operation
print('a' in my_tuple)
print('b' in my_tuple)

# Not in operation
print('g' not in my_tuple)
```

Output

```
True
False
True
```

2. Iterating Through a Tuple

We can use a `for` loop to iterate through each item in a tuple.

```
# Using a for loop to iterate through a tuple
for name in ('John', 'Kate'):
    print("Hello", name)
```

Output

```
Hello John
Hello Kate
```

Advantages of Tuple over List

Since tuples are quite similar to lists, both of them are used in similar situations. However, there are certain advantages of implementing a tuple over a list. Below listed are some of the main advantages:

- We generally use tuples for heterogeneous (different) data types and lists for homogeneous (similar) data types.
- Since tuples are immutable, iterating through a tuple is faster than with list. So there is a slight performance boost.
- Tuples that contain immutable elements can be used as a key for a dictionary. With lists, this is not possible.
- If you have data that doesn't change, implementing it as tuple will guarantee that it remains write-protected.

Python Strings

programiz.com/python-programming/string

In this tutorial you will learn to create, format, modify and delete strings in Python. Also, you will be introduced to various string operations and functions.

What is String in Python?

A string is a sequence of characters.

A character is simply a symbol. For example, the English language has 26 characters.

Computers do not deal with characters, they deal with numbers (binary). Even though you may see characters on your screen, internally it is stored and manipulated as a combination of 0s and 1s.

This conversion of character to a number is called encoding, and the reverse process is decoding. ASCII and Unicode are some of the popular encodings used.

In Python, a string is a sequence of Unicode characters. Unicode was introduced to include every character in all languages and bring uniformity in encoding. You can learn about Unicode from Python Unicode.

How to create a string in Python?

Strings can be created by enclosing characters inside a single quote or double-quotes. Even triple quotes can be used in Python but generally used to represent multiline strings and docstrings.

```
# defining strings in Python
# all of the following are equivalent
my_string = 'Hello'
print(my_string)

my_string = "Hello"
print(my_string)

my_string = '''Hello'''
print(my_string)

# triple quotes string can extend multiple lines
my_string = """Hello, welcome to
           the world of Python"""
print(my_string)
```

When you run the program, the output will be:

```
Hello
Hello
Hello
Hello, welcome to
           the world of Python
```

How to access characters in a string?

We can access individual characters using indexing and a range of characters using slicing. Index starts from 0. Trying to access a character out of index range will raise an `IndexError`. The index must be an integer. We can't use floats or other types, this will result into `TypeError`.

Python allows negative indexing for its sequences.

The index of `-1` refers to the last item, `-2` to the second last item and so on. We can access a range of items in a string by using the slicing operator `:` (colon).

```
#Accessing string characters in Python
str = 'programiz'
print('str = ', str)

#first character
print('str[0] = ', str[0])

#last character
print('str[-1] = ', str[-1])

#slicing 2nd to 5th character
print('str[1:5] = ', str[1:5])

#slicing 6th to 2nd last character
print('str[5:-2] = ', str[5:-2])
```

When we run the above program, we get the following output:

```
str =  programiz
str[0] =  p
str[-1] =  z
str[1:5] =  rogr
str[5:-2] =  am
```

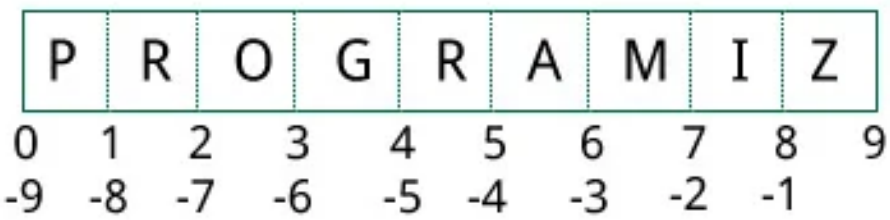
If we try to access an index out of the range or use numbers other than an integer, we will get errors.

```
# index must be in range
>>> my_string[15]
...
IndexError: string index out of range

# index must be an integer
>>> my_string[1.5]
...
TypeError: string indices must be integers
```

Slicing can be best visualized by considering the index to be between the elements as shown below.

If we want to access a range, we need the index that will slice the portion from the string.



String Slicing in Python

How to change or delete a string?

Strings are immutable. This means that elements of a string cannot be changed once they have been assigned. We can simply reassign different strings to the same name.

```
>>> my_string = 'programiz'
>>> my_string[5] = 'a'
...
TypeError: 'str' object does not support item assignment
>>> my_string = 'Python'
>>> my_string
'Python'
```

We cannot delete or remove characters from a string. But deleting the string entirely is possible using the `del` keyword.

```
>>> del my_string[1]
...
TypeError: 'str' object doesn't support item deletion
>>> del my_string
>>> my_string
...
NameError: name 'my_string' is not defined
```

Python String Operations

There are many operations that can be performed with strings which makes it one of the most used data types in Python.

To learn more about the data types available in Python visit: Python Data Types

Concatenation of Two or More Strings

Joining of two or more strings into a single one is called concatenation.

The + operator does this in Python. Simply writing two string literals together also concatenates them.

The * operator can be used to repeat the string for a given number of times.

```
# Python String Operations
str1 = 'Hello'
str2 ='World!'

# using +
print('str1 + str2 = ', str1 + str2)

# using *
print('str1 * 3 =', str1 * 3)
```

When we run the above program, we get the following output:

```
str1 + str2 =  HelloWorld!
str1 * 3 = HelloHelloHello
```

Writing two string literals together also concatenates them like + operator.

If we want to concatenate strings in different lines, we can use parentheses.

```
>>> # two string literals together
>>> 'Hello ''World!'
'Hello World!'

>>> # using parentheses
>>> s = ('Hello '
...      'World')
>>> s
'Hello World'
```

Iterating Through a string

We can iterate through a string using a for loop. Here is an example to count the number of 'l's in a string.

```
# Iterating through a string
count = 0
for letter in 'Hello World':
    if(letter == 'l'):
        count += 1
print(count,'letters found')
```

When we run the above program, we get the following output:

```
3 letters found
```

String Membership Test

We can test if a substring exists within a string or not, using the keyword `in`.

```
>>> 'a' in 'program'
True
>>> 'at' not in 'battle'
False
```

Built-in functions to Work with Python

Various built-in functions that work with sequence work with strings as well.

Some of the commonly used ones are `enumerate()` and `len()`. The `enumerate()` function returns an enumerate object. It contains the index and value of all the items in the string as pairs. This can be useful for iteration.

Similarly, `len()` returns the length (number of characters) of the string.

```
str = 'cold'

# enumerate()
list_enumerate = list(enumerate(str))
print('list(enumerate(str) = ', list_enumerate)

#character count
print('len(str) = ', len(str))
```

When we run the above program, we get the following output:

```
list(enumerate(str) =  [(0, 'c'), (1, 'o'), (2, 'l'), (3, 'd')]
len(str) =  4
```

Python String Formatting

Escape Sequence

If we want to print a text like `He said, "What's there?"`, we can neither use single quotes nor double quotes. This will result in a `SyntaxError` as the text itself contains both single and double quotes.

```
>>> print("He said, "What's there?"")
...
SyntaxError: invalid syntax
>>> print('He said, "What's there?"')
...
SyntaxError: invalid syntax
```

One way to get around this problem is to use triple quotes. Alternatively, we can use escape sequences.

An escape sequence starts with a backslash and is interpreted differently. If we use a single quote to represent a string, all the single quotes inside the string must be escaped. Similar is the case with double quotes. Here is how it can be done to represent the above text.

```
# using triple quotes
print('''He said, "What's there?"''')

# escaping single quotes
print('He said, "What\'s there?"')

# escaping double quotes
print("He said, \"What's there?\"")
```

When we run the above program, we get the following output:

```
He said, "What's there?"
He said, "What's there?"
He said, "What's there?"
```

Here is a list of all the escape sequences supported by Python.

Escape Sequence	Description
\newline	Backslash and newline ignored
\\	Backslash
\'	Single quote
\"	Double quote
\a	ASCII Bell
\b	ASCII Backspace
\f	ASCII Formfeed
\n	ASCII Linefeed
\r	ASCII Carriage Return
\t	ASCII Horizontal Tab
\v	ASCII Vertical Tab
\ooo	Character with octal value ooo
\xHH	Character with hexadecimal value HH

Here are some examples

```
>>> print("C:\\Python32\\Lib")
C:\Python32\Lib

>>> print("This is printed\nin two lines")
This is printed
in two lines

>>> print("This is \x48\x45\x58 representation")
This is HEX representation
```

Raw String to ignore escape sequence

Sometimes we may wish to ignore the escape sequences inside a string. To do this we can place `r` or `R` in front of the string. This will imply that it is a raw string and any escape sequence inside it will be ignored.

```
>>> print("This is \x61 \ngood example")
This is a
good example
>>> print(r"This is \x61 \ngood example")
This is \x61 \ngood example
```

The format() Method for Formatting Strings

The `format()` method that is available with the string object is very versatile and powerful in formatting strings. Format strings contain curly braces `{}` as placeholders or replacement fields which get replaced.

We can use positional arguments or keyword arguments to specify the order.

```
# Python string format() method

# default(implicit) order
default_order = "{}, {} and {}".format('John','Bill','Sean')
print('\n--- Default Order ---')
print(default_order)

# order using positional argument
positional_order = "{1}, {0} and {2}".format('John','Bill','Sean')
print('\n--- Positional Order ---')
print(positional_order)

# order using keyword argument
keyword_order = "{s}, {b} and {j}".format(j='John',b='Bill',s='Sean')
print('\n--- Keyword Order ---')
print(keyword_order)
```

When we run the above program, we get the following output:

```
--- Default Order ---
John, Bill and Sean

--- Positional Order ---
Bill, John and Sean

--- Keyword Order ---
Sean, Bill and John
```

The `format()` method can have optional format specifications. They are separated from the field name using colon. For example, we can left-justify `<`, right-justify `>` or center `^` a string in the given space.

We can also format integers as binary, hexadecimal, etc. and floats can be rounded or displayed in the exponent format. There are tons of formatting you can use. Visit here for all the string formatting available with the `format()` method.

```
>>> # formatting integers
>>> "Binary representation of {0} is {0:b}".format(12)
'Binary representation of 12 is 1100'

>>> # formatting floats
>>> "Exponent representation: {0:e}".format(1566.345)
'Exponent representation: 1.566345e+03'

>>> # round off
>>> "One third is: {0:.3f}".format(1/3)
'One third is: 0.333'

>>> # string alignment
>>> "|{:<10}|{:^10}|{:>10}|".format('butter','bread','ham')
'|butter    |  bread   |       ham|'
```

Old style formatting

We can even format strings like the old `sprintf()` style used in C programming language. We use the `%` operator to accomplish this.

```
>>> x = 12.3456789
>>> print('The value of x is %3.2f' %x)
The value of x is 12.35
>>> print('The value of x is %3.4f' %x)
The value of x is 12.3457
```

Common Python String Methods

There are numerous methods available with the string object. The `format()` method that we mentioned above is one of them. Some of the commonly used methods are `lower()`, `upper()`, `join()`, `split()`, `find()`, `replace()` etc. Here is a complete list of all the built-in methods to work with strings in Python.

```
>>> "PrOgRaMiZ".lower()
'programiz'
>>> "PrOgRaMiZ".upper()
'PROGRAMIZ'
>>> "This will split all words into a list".split()
['This', 'will', 'split', 'all', 'words', 'into', 'a', 'list']
>>> ' '.join(['This', 'will', 'join', 'all', 'words', 'into', 'a', 'string'])
'This will join all words into a string'
>>> 'Happy New Year'.find('ew')
7
>>> 'Happy New Year'.replace('Happy','Brilliant')
'Brilliant New Year'
```

Python Sets

programiz.com/python-programming/set

In this tutorial, you'll learn everything about Python sets; how they are created, adding or removing elements from them, and all operations performed on sets in Python.

A set is an unordered collection of items. Every set element is unique (no duplicates) and must be immutable (cannot be changed).

However, a set itself is mutable. We can add or remove items from it.

Sets can also be used to perform mathematical set operations like union, intersection, symmetric difference, etc.

Creating Python Sets

A set is created by placing all the items (elements) inside curly braces `{}`, separated by comma, or by using the built-in `set()` function.

It can have any number of items and they may be of different types (integer, float, tuple, string etc.). But a set cannot have mutable elements like lists, sets or dictionaries as its elements.

```
# Different types of sets in Python
# set of integers
my_set = {1, 2, 3}
print(my_set)

# set of mixed datatypes
my_set = {1.0, "Hello", (1, 2, 3)}
print(my_set)
```

Output

```
{1, 2, 3}
{1.0, (1, 2, 3), 'Hello'}
```

Try the following examples as well.

```
# set cannot have duplicates
# Output: {1, 2, 3, 4}
my_set = {1, 2, 3, 4, 3, 2}
print(my_set)

# we can make set from a list
# Output: {1, 2, 3}
my_set = set([1, 2, 3, 2])
print(my_set)

# set cannot have mutable items
# here [3, 4] is a mutable list
# this will cause an error.

my_set = {1, 2, [3, 4]}
```

Output

```
{1, 2, 3, 4}
{1, 2, 3}
Traceback (most recent call last):
  File "<string>", line 15, in <module>
    my_set = {1, 2, [3, 4]}
TypeError: unhashable type: 'list'
```

Creating an empty set is a bit tricky.

Empty curly braces `{}` will make an empty dictionary in Python. To make a set without any elements, we use the `set()` function without any argument.

```
# Distinguish set and dictionary while creating empty set

# initialize a with {}
a = {}

# check data type of a
print(type(a))

# initialize a with set()
a = set()

# check data type of a
print(type(a))
```

Output

```
<class 'dict'>
<class 'set'>
```

Modifying a set in Python

Sets are mutable. However, since they are unordered, indexing has no meaning.

We cannot access or change an element of a set using indexing or slicing. Set data type does not support it.

We can add a single element using the `add()` method, and multiple elements using the `update()` method. The `update()` method can take tuples, lists, strings or other sets as its argument. In all cases, duplicates are avoided.

```
# initialize my_set
my_set = {1, 3}
print(my_set)

# my_set[0]
# if you uncomment the above line
# you will get an error
# TypeError: 'set' object does not support indexing

# add an element
# Output: {1, 2, 3}
my_set.add(2)
print(my_set)

# add multiple elements
# Output: {1, 2, 3, 4}
my_set.update([2, 3, 4])
print(my_set)

# add list and set
# Output: {1, 2, 3, 4, 5, 6, 8}
my_set.update([4, 5], {1, 6, 8})
print(my_set)
```

Output

```
{1, 3}
{1, 2, 3}
{1, 2, 3, 4}
{1, 2, 3, 4, 5, 6, 8}
```

Removing elements from a set

A particular item can be removed from a set using the methods `discard()` and `remove()`.

The only difference between the two is that the `discard()` function leaves a set unchanged if the element is not present in the set. On the other hand, the `remove()` function will raise an error in such a condition (if element is not present in the set).

The following example will illustrate this.

```
# Difference between discard() and remove()

# initialize my_set
my_set = {1, 3, 4, 5, 6}
print(my_set)

# discard an element
# Output: {1, 3, 5, 6}
my_set.discard(4)
print(my_set)

# remove an element
# Output: {1, 3, 5}
my_set.remove(6)
print(my_set)

# discard an element
# not present in my_set
# Output: {1, 3, 5}
my_set.discard(2)
print(my_set)

# remove an element
# not present in my_set
# you will get an error.
# Output: KeyError

my_set.remove(2)
```

Output

```
{1, 3, 4, 5, 6}
{1, 3, 5, 6}
{1, 3, 5}
{1, 3, 5}
Traceback (most recent call last):
  File "<string>", line 28, in <module>
KeyError: 2
```

Similarly, we can remove and return an item using the `pop()` method.

Since set is an unordered data type, there is no way of determining which item will be popped. It is completely arbitrary.

We can also remove all the items from a set using the `clear()` method.

```
# initialize my_set
# Output: set of unique elements
my_set = set("HelloWorld")
print(my_set)

# pop an element
# Output: random element
print(my_set.pop())

# pop another element
my_set.pop()
print(my_set)

# clear my_set
# Output: set()
my_set.clear()
print(my_set)

print(my_set)
```

Output

```
{'H', 'l', 'r', 'W', 'o', 'd', 'e'}
H
{'r', 'W', 'o', 'd', 'e'}
set()
```

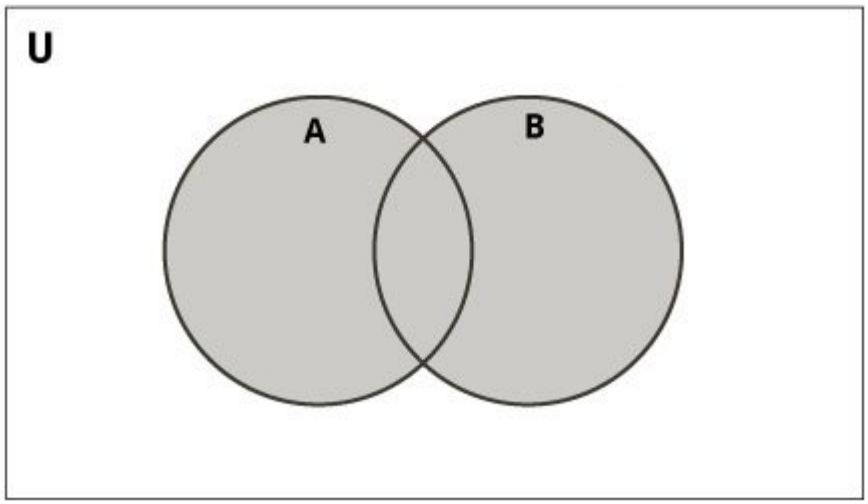
Python Set Operations

Sets can be used to carry out mathematical set operations like union, intersection, difference and symmetric difference. We can do this with operators or methods.

Let us consider the following two sets for the following operations.

```
>>> A = {1, 2, 3, 4, 5}
>>> B = {4, 5, 6, 7, 8}
```

Set Union



Set Union in Python

Union of *A* and *B* is a set of all elements from both sets.

Union is performed using `|` operator. Same can be accomplished using the `union()` method.

```
# Set union method
# initialize A and B
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# use | operator
# Output: {1, 2, 3, 4, 5, 6, 7, 8}
print(A | B)
```

Output

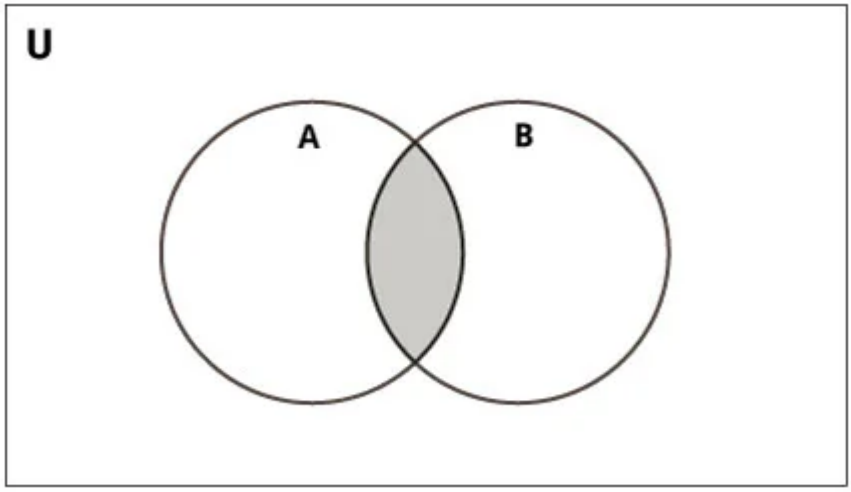
```
{1, 2, 3, 4, 5, 6, 7, 8}
```

Try the following examples on Python shell.

```
# use union function
>>> A.union(B)
{1, 2, 3, 4, 5, 6, 7, 8}

# use union function on B
>>> B.union(A)
{1, 2, 3, 4, 5, 6, 7, 8}
```

Set Intersection



Set Intersection in Python

Intersection of *A* and *B* is a set of elements that are common in both the sets.

Intersection is performed using `&` operator. Same can be accomplished using the `intersection()` method.

```
# Intersection of sets
# initialize A and B
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# use & operator
# Output: {4, 5}
print(A & B)
```

Output

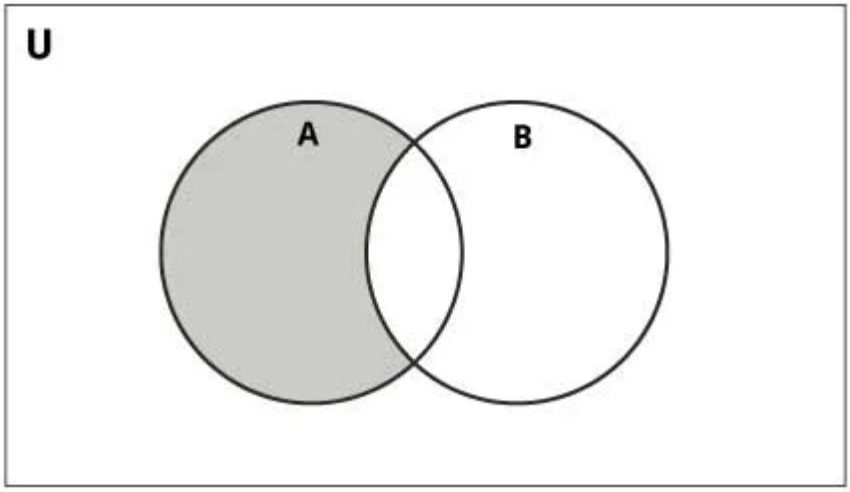
```
{4, 5}
```

Try the following examples on Python shell.

```
# use intersection function on A
>>> A.intersection(B)
{4, 5}

# use intersection function on B
>>> B.intersection(A)
{4, 5}
```

Set Difference



Set Difference in Python

Difference of the set *B* from set *A*(*A* - *B*) is a set of elements that are only in *A* but not in *B*. Similarly, *B* - *A* is a set of elements in *B* but not in *A*.

Difference is performed using `-` operator. Same can be accomplished using the `difference()` method.

```
# Difference of two sets
# initialize A and B
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# use - operator on A
# Output: {1, 2, 3}
print(A - B)
```

Output

```
{1, 2, 3}
```

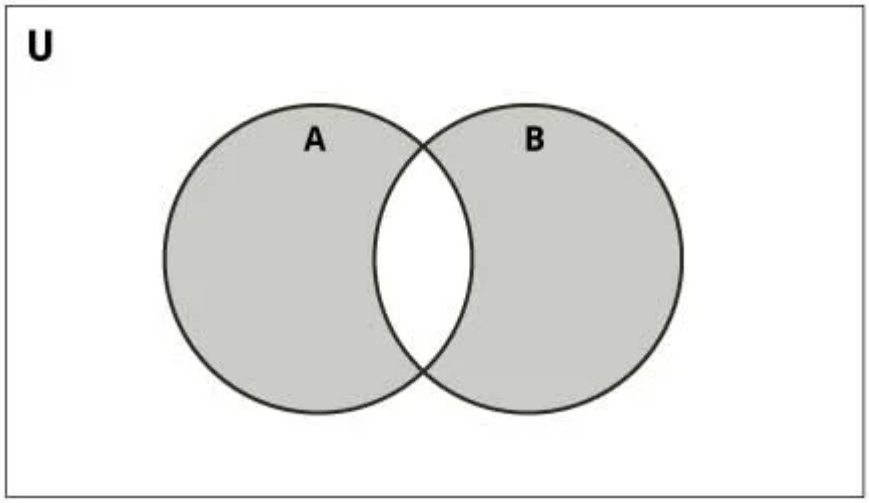
Try the following examples on Python shell.

```
# use difference function on A
>>> A.difference(B)
{1, 2, 3}

# use - operator on B
>>> B - A
{8, 6, 7}

# use difference function on B
>>> B.difference(A)
{8, 6, 7}
```

Set Symmetric Difference



Set Symmetric Difference in Python

Symmetric Difference of *A* and *B* is a set of elements in *A* and *B* but not in both (excluding the intersection).

Symmetric difference is performed using `^` operator. Same can be accomplished using the method `symmetric_difference()`.

```
# Symmetric difference of two sets
# initialize A and B
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# use ^ operator
# Output: {1, 2, 3, 6, 7, 8}
print(A ^ B)
```

Output

```
{1, 2, 3, 6, 7, 8}
```

Try the following examples on Python shell.

```
# use symmetric_difference function on A
>>> A.symmetric_difference(B)
{1, 2, 3, 6, 7, 8}

# use symmetric_difference function on B
>>> B.symmetric_difference(A)
{1, 2, 3, 6, 7, 8}
```

Other Python Set Methods

There are many set methods, some of which we have already used above. Here is a list of all the methods that are available with the set objects:

Method	Description

add()	Adds an element to the set
clear()	Removes all elements from the set
copy()	Returns a copy of the set
difference()	Returns the difference of two or more sets as a new set
difference_update()	Removes all elements of another set from this set
discard()	Removes an element from the set if it is a member. (Do nothing if the element is not in set)
intersection()	Returns the intersection of two sets as a new set
intersection_update()	Updates the set with the intersection of itself and another
isdisjoint()	Returns `True` if two sets have a null intersection
issubset()	Returns `True` if another set contains this set
issuperset()	Returns `True` if this set contains another set
pop()	Removes and returns an arbitrary set element. Raises `KeyError` if the set is empty
remove()	Removes an element from the set. If the element is not a member, raises a `KeyError`
symmetric_difference()	Returns the symmetric difference of two sets as a new set
symmetric_difference_update()	Updates a set with the symmetric difference of itself and another
union()	Returns the union of sets in a new set
update()	Updates the set with the union of itself and others

Other Set Operations

Set Membership Test

We can test if an item exists in a set or not, using the `in` keyword.

```
# in keyword in a set
# initialize my_set
my_set = set("apple")

# check if 'a' is present
# Output: True
print('a' in my_set)

# check if 'p' is present
# Output: False
print('p' not in my_set)
```

Output

```
True
False
```

Iterating Through a Set

We can iterate through each item in a set using a `for` loop.

```
>>> for letter in set("apple"):
...     print(letter)
...
a
p
e
l
```

Built-in Functions with Set

Built-in functions like `all()`, `any()`, `enumerate()`, `len()`, `max()`, `min()`, `sorted()`, `sum()` etc. are commonly used with sets to perform different tasks.

Function	Description
all()	Returns `True` if all elements of the set are true (or if the set is empty).
any()	Returns `True` if any element of the set is true. If the set is empty, returns `False`.
enumerate()	Returns an enumerate object. It contains the index and value for all the items of the set as a pair.
len()	Returns the length (the number of items) in the set.
max()	Returns the largest item in the set.
min()	Returns the smallest item in the set.
sorted()	Returns a new sorted list from elements in the set(does not sort the set itself).

---

sum()	Returns the sum of all elements in the set.

Python Frozenset

Frozenset is a new class that has the characteristics of a set, but its elements cannot be changed once assigned. While tuples are immutable lists, frozensets are immutable sets.

Sets being mutable are unhashable, so they can't be used as dictionary keys. On the other hand, frozensets are hashable and can be used as keys to a dictionary.

Frozensets can be created using the frozenset() function.

This data type supports methods like `copy()`, `difference()`, `intersection()`, `isdisjoint()`, `issubset()`, `issuperset()`, `symmetric_difference()` and `union()`. Being immutable, it does not have methods that add or remove elements.

```
# Frozensets
# initialize A and B
A = frozenset([1, 2, 3, 4])
B = frozenset([3, 4, 5, 6])
```

Try these examples on Python shell.

```
>>> A.isdisjoint(B)
False
>>> A.difference(B)
frozenset({1, 2})
>>> A | B
frozenset({1, 2, 3, 4, 5, 6})
>>> A.add(3)
...
AttributeError: 'frozenset' object has no attribute 'add'
```

Python Dictionary

programiz.com/python-programming/dictionary

In this tutorial, you'll learn everything about Python dictionaries; how they are created, accessing, adding, removing elements from them and various built-in methods.

Python dictionary is an unordered collection of items. Each item of a dictionary has a `key/value` pair.

Dictionaries are optimized to retrieve values when the key is known.

Creating Python Dictionary

Creating a dictionary is as simple as placing items inside curly braces `{}` separated by commas.

An item has a `key` and a corresponding `value` that is expressed as a pair (**key: value**).

While the values can be of any data type and can repeat, keys must be of immutable type (string, number or tuple with immutable elements) and must be unique.

```
# empty dictionary
my_dict = {}

# dictionary with integer keys
my_dict = {1: 'apple', 2: 'ball'}

# dictionary with mixed keys
my_dict = {'name': 'John', 1: [2, 4, 3]}

# using dict()
my_dict = dict({1:'apple', 2:'ball'})

# from sequence having each item as a pair
my_dict = dict([(1,'apple'), (2,'ball')])
```

As you can see from above, we can also create a dictionary using the built-in `dict()` function.

Accessing Elements from Dictionary

While indexing is used with other data types to access values, a dictionary uses `keys`. Keys can be used either inside square brackets `[]` or with the `get()` method.

If we use the square brackets `[]`, `KeyError` is raised in case a key is not found in the dictionary. On the other hand, the `get()` method returns `None` if the key is not found.

```
# get vs [] for retrieving elements
my_dict = {'name': 'Jack', 'age': 26}

# Output: Jack
print(my_dict['name'])

# Output: 26
print(my_dict.get('age'))

# Trying to access keys which doesn't exist throws error
# Output None
print(my_dict.get('address'))

# KeyError
print(my_dict['address'])
```

Output

```
Jack
26
None
Traceback (most recent call last):
  File "<string>", line 15, in <module>
    print(my_dict['address'])
KeyError: 'address'
```

Changing and Adding Dictionary elements

Dictionaries are mutable. We can add new items or change the value of existing items using an assignment operator.

If the key is already present, then the existing value gets updated. In case the key is not present, a new (**key: value**) pair is added to the dictionary.

```
# Changing and adding Dictionary Elements
my_dict = {'name': 'Jack', 'age': 26}

# update value
my_dict['age'] = 27

#Output: {'age': 27, 'name': 'Jack'}
print(my_dict)

# add item
my_dict['address'] = 'Downtown'

# Output: {'address': 'Downtown', 'age': 27, 'name': 'Jack'}
print(my_dict)
```

Output

```
{'name': 'Jack', 'age': 27}
{'name': 'Jack', 'age': 27, 'address': 'Downtown'}
```

Removing elements from Dictionary

We can remove a particular item in a dictionary by using the `pop()` method. This method removes an item with the provided `key` and returns the `value`.

The `popitem()` method can be used to remove and return an arbitrary `(key, value)` item pair from the dictionary. All the items can be removed at once, using the `clear()` method.

We can also use the `del` keyword to remove individual items or the entire dictionary itself.

```
# Removing elements from a dictionary

# create a dictionary
squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# remove a particular item, returns its value
# Output: 16
print(squares.pop(4))

# Output: {1: 1, 2: 4, 3: 9, 5: 25}
print(squares)

# remove an arbitrary item, return (key,value)
# Output: (5, 25)
print(squares.popitem())

# Output: {1: 1, 2: 4, 3: 9}
print(squares)

# remove all items
squares.clear()

# Output: {}
print(squares)

# delete the dictionary itself
del squares

# Throws Error
print(squares)
```

Output

```
16
{1: 1, 2: 4, 3: 9, 5: 25}
(5, 25)
{1: 1, 2: 4, 3: 9}
{}
Traceback (most recent call last):
  File "<string>", line 30, in <module>
    print(squares)
NameError: name 'squares' is not defined
```

Python Dictionary Methods

Methods that are available with a dictionary are tabulated below. Some of them have already been used in the above examples.

Method	Description
clear()	Removes all items from the dictionary.
copy()	Returns a shallow copy of the dictionary.
fromkeys(seq[, v])	Returns a new dictionary with keys from *seq* and value equal to *v* (defaults to `None`).
get(key[,d])	Returns the value of the *key*. If the *key* does not exist, returns *d* (defaults to `None`).
items()	Return a new object of the dictionary's items in (key, value) format.
keys()	Returns a new object of the dictionary's keys.
pop(key[,d])	Removes the item with the *key* and returns its value or *d* if *key* is not found. If *d* is not provided and the *key* is not found, it raises `KeyError`.
popitem()	Removes and returns an arbitrary item (**key, value**). Raises `KeyError` if the dictionary is empty.
setdefault(key[,d])	Returns the corresponding value if the *key* is in the dictionary. If not, inserts the *key* with a value of *d* and returns *d* (defaults to `None`).
update([other])	Updates the dictionary with the key/value pairs from *other*, overwriting existing keys.
values()	Returns a new object of the dictionary's values

Here are a few example use cases of these methods.

```
# Dictionary Methods
marks = {}.fromkeys(['Math', 'English', 'Science'], 0)

# Output: {'English': 0, 'Math': 0, 'Science': 0}
print(marks)

for item in marks.items():
    print(item)

# Output: ['English', 'Math', 'Science']
print(list(sorted(marks.keys())))
```

Output

```
{'Math': 0, 'English': 0, 'Science': 0}
('Math', 0)
('English', 0)
('Science', 0)
['English', 'Math', 'Science']
```

Python Dictionary Comprehension

Dictionary comprehension is an elegant and concise way to create a new dictionary from an iterable in Python.

Dictionary comprehension consists of an expression pair (**key: value**) followed by a `for` statement inside curly braces `{}`.

Here is an example to make a dictionary with each item being a pair of a number and its square.

```
# Dictionary Comprehension
squares = {x: x*x for x in range(6)}

print(squares)
```

Output

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

This code is equivalent to

```
squares = {}
for x in range(6):
    squares[x] = x*x
print(squares)
```

Output

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

A dictionary comprehension can optionally contain more for or if statements.

An optional `if` statement can filter out items to form the new dictionary.

Here are some examples to make a dictionary with only odd items.

```
# Dictionary Comprehension with if conditional
odd_squares = {x: x*x for x in range(11) if x % 2 == 1}

print(odd_squares)
```

Output

```
{1: 1, 3: 9, 5: 25, 7: 49, 9: 81}
```

To learn more dictionary comprehensions, visit Python Dictionary Comprehension.

Other Dictionary Operations

Dictionary Membership Test

We can test if a `key` is in a dictionary or not using the keyword `in`. Notice that the membership test is only for the `keys` and not for the `values`.

```
# Membership Test for Dictionary Keys
squares = {1: 1, 3: 9, 5: 25, 7: 49, 9: 81}

# Output: True
print(1 in squares)

# Output: True
print(2 not in squares)

# membership tests for key only not value
# Output: False
print(49 in squares)
```

Output

```
True
True
False
```

Iterating Through a Dictionary

We can iterate through each key in a dictionary using a `for` loop.

```
# Iterating through a Dictionary
squares = {1: 1, 3: 9, 5: 25, 7: 49, 9: 81}
for i in squares:
    print(squares[i])
```

Output

```
1
9
25
49
81
```

Dictionary Built-in Functions

Built-in functions like `all()`, `any()`, `len()`, `cmp()`, `sorted()`, etc. are commonly used with dictionaries to perform different tasks.

Function	Description
all()	Return `True` if all keys of the dictionary are True (or if the dictionary is empty).

any()	Return `True` if any key of the dictionary is true. If the dictionary is empty, return `False`.
len()	Return the length (the number of items) in the dictionary.
cmp()	Compares items of two dictionaries. (Not available in Python 3)
sorted()	Return a new sorted list of keys in the dictionary.

Here are some examples that use built-in functions to work with a dictionary.

```
# Dictionary Built-in Functions
squares = {0: 0, 1: 1, 3: 9, 5: 25, 7: 49, 9: 81}

# Output: False
print(all(squares))

# Output: True
print(any(squares))

# Output: 6
print(len(squares))

# Output: [0, 1, 3, 5, 7, 9]
print(sorted(squares))
```

Output

```
False
True
6
[0, 1, 3, 5, 7, 9]
```

Python File I/O

programiz.com/python-programming/file-operation

In this tutorial, you'll learn about Python file operations. More specifically, opening a file, reading from it, writing into it, closing it, and various file methods that you should be aware of.

Files

Files are named locations on disk to store related information. They are used to permanently store data in a non-volatile memory (e.g. hard disk).

Since Random Access Memory (RAM) is volatile (which loses its data when the computer is turned off), we use files for future use of the data by permanently storing them.

When we want to read from or write to a file, we need to open it first. When we are done, it needs to be closed so that the resources that are tied with the file are freed.

Hence, in Python, a file operation takes place in the following order:

1. Open a file
2. Read or write (perform operation)
3. Close the file

Opening Files in Python

Python has a built-in `open()` function to open a file. This function returns a file object, also called a handle, as it is used to read or modify the file accordingly.

```
>>> f = open("test.txt")    # open file in current directory
>>> f = open("C:/Python38/README.txt")  # specifying full path
```

We can specify the mode while opening a file. In mode, we specify whether we want to read `r`, write `w` or append `a` to the file. We can also specify if we want to open the file in text mode or binary mode.

The default is reading in text mode. In this mode, we get strings when reading from the file.

On the other hand, binary mode returns bytes and this is the mode to be used when dealing with non-text files like images or executable files.

Mode	Description
`r`	Opens a file for reading. (default)

`w`	Opens a file for writing. Creates a new file if it does not exist or truncates the file if it exists.
`x`	Opens a file for exclusive creation. If the file already exists, the operation fails.
`a`	Opens a file for appending at the end of the file without truncating it. Creates a new file if it does not exist.
`t`	Opens in text mode. (default)
`b`	Opens in binary mode.
`+`	Opens a file for updating (reading and writing)

```
f = open("test.txt")      # equivalent to 'r' or 'rt'
f = open("test.txt",'w')  # write in text mode
f = open("img.bmp",'r+b') # read and write in binary mode
```

Unlike other languages, the character `a` does not imply the number 97 until it is encoded using `ASCII` (or other equivalent encodings).

Moreover, the default encoding is platform dependent. In windows, it is `cp1252` but `utf-8` in Linux.

So, we must not also rely on the default encoding or else our code will behave differently in different platforms.

Hence, when working with files in text mode, it is highly recommended to specify the encoding type.

```
f = open("test.txt", mode='r', encoding='utf-8')
```

Closing Files in Python

When we are done with performing operations on the file, we need to properly close the file.

Closing a file will free up the resources that were tied with the file. It is done using the `close()` method available in Python.

Python has a garbage collector to clean up unreferenced objects but we must not rely on it to close the file.

```
f = open("test.txt", encoding = 'utf-8')
# perform file operations
f.close()
```

This method is not entirely safe. If an exception occurs when we are performing some operation with the file, the code exits without closing the file.

A safer way is to use a try...finally block.

```
try:
   f = open("test.txt", encoding = 'utf-8')
   # perform file operations
finally:
   f.close()
```

This way, we are guaranteeing that the file is properly closed even if an exception is raised that causes program flow to stop.

The best way to close a file is by using the `with` statement. This ensures that the file is closed when the block inside the `with` statement is exited.

We don't need to explicitly call the `close()` method. It is done internally.

```
with open("test.txt", encoding = 'utf-8') as f:
   # perform file operations
```

Writing to Files in Python

In order to write into a file in Python, we need to open it in write `w`, append `a` or exclusive creation `x` mode.

We need to be careful with the `w` mode, as it will overwrite into the file if it already exists. Due to this, all the previous data are erased.

Writing a string or sequence of bytes (for binary files) is done using the `write()` method. This method returns the number of characters written to the file.

```
with open("test.txt",'w',encoding = 'utf-8') as f:
   f.write("my first file\n")
   f.write("This file\n\n")
   f.write("contains three lines\n")
```

This program will create a new file named `test.txt` in the current directory if it does not exist. If it does exist, it is overwritten.

We must include the newline characters ourselves to distinguish the different lines.

Reading Files in Python

To read a file in Python, we must open the file in reading `r` mode.

There are various methods available for this purpose. We can use the `read(size)` method to read in the *size* number of data. If the *size* parameter is not specified, it reads and returns up to the end of the file.

We can read the `text.txt` file we wrote in the above section in the following way:

```
>>> f = open("test.txt",'r',encoding = 'utf-8')
>>> f.read(4)    # read the first 4 data
'This'

>>> f.read(4)    # read the next 4 data
' is '

>>> f.read()     # read in the rest till end of file
'my first file\nThis file\ncontains three lines\n'

>>> f.read()  # further reading returns empty sting
''
```

We can see that the `read()` method returns a newline as `'\n'`. Once the end of the file is reached, we get an empty string on further reading.

We can change our current file cursor (position) using the `seek()` method. Similarly, the `tell()` method returns our current position (in number of bytes).

```
>>> f.tell()    # get the current file position
56

>>> f.seek(0)   # bring file cursor to initial position
0

>>> print(f.read())  # read the entire file
This is my first file
This file
contains three lines
```

We can read a file line-by-line using a for loop. This is both efficient and fast.

```
>>> for line in f:
...     print(line, end = '')
...
This is my first file
This file
contains three lines
```

In this program, the lines in the file itself include a newline character `\n`. So, we use the end parameter of the `print()` function to avoid two newlines when printing.

Alternatively, we can use the `readline()` method to read individual lines of a file. This method reads a file till the newline, including the newline character.

```
>>> f.readline()
'This is my first file\n'

>>> f.readline()
'This file\n'

>>> f.readline()
'contains three lines\n'

>>> f.readline()
''
```

Lastly, the `readlines()` method returns a list of remaining lines of the entire file. All these reading methods return empty values when the end of file (EOF) is reached.

```
>>> f.readlines()
['This is my first file\n', 'This file\n', 'contains three lines\n']
```

Python File Methods

There are various methods available with the file object. Some of them have been used in the above examples.

Here is the complete list of methods in text mode with a brief description:

Method	Description
close()	Closes an opened file. It has no effect if the file is already closed.
detach()	Separates the underlying binary buffer from the `TextIOBase` and returns it.
fileno()	Returns an integer number (file descriptor) of the file.
flush()	Flushes the write buffer of the file stream.
isatty()	Returns `True` if the file stream is interactive.
read(*n*)	Reads at most *n* characters from the file. Reads till end of file if it is negative or `None`.
readable()	Returns `True` if the file stream can be read from.
readline(*n*=-1)	Reads and returns one line from the file. Reads in at most *n* bytes if specified.
readlines(*n*=-1)	Reads and returns a list of lines from the file. Reads in at most *n* bytes/characters if specified.
seek(*offset*,*from*=`SEEK_SET`)	Changes the file position to *offset* bytes, in reference to *from* (start, current, end).
seekable()	Returns `True` if the file stream supports random access.
tell()	Returns the current file location.
truncate(*size*=`None`)	Resizes the file stream to *size* bytes. If *size* is not specified, resizes to current location.
writable()	Returns `True` if the file stream can be written to.
write(*s*)	Writes the string *s* to the file and returns the number of characters written.

writelines(*lines*)	Writes a list of *lines* to the file.

Python Directory and Files Management

programiz.com/python-programming/directory

In this tutorial, you'll learn about file and directory management in Python, i.e. creating a directory, renaming it, listing all directories, and working with them.

Python Directory

If there are a large number of files to handle in our Python program, we can arrange our code within different directories to make things more manageable.

A directory or folder is a collection of files and subdirectories. Python has the `os` module that provides us with many useful methods to work with directories (and files as well).

Get Current Directory

We can get the present working directory using the `getcwd()` method of the `os` module.

This method returns the current working directory in the form of a string. We can also use the `getcwdb()` method to get it as bytes object.

```
>>> import os

>>> os.getcwd()
'C:\\Program Files\\PyScripter'

>>> os.getcwdb()
b'C:\\Program Files\\PyScripter'
```

The extra backslash implies an escape sequence. The `print()` function will render this properly.

```
>>> print(os.getcwd())
C:\Program Files\PyScripter
```

Changing Directory

We can change the current working directory by using the `chdir()` method.

The new path that we want to change into must be supplied as a string to this method. We can use both the forward-slash `/` or the backward-slash `\` to separate the path elements.

It is safer to use an escape sequence when using the backward slash.

```
>>> os.chdir('C:\\Python33')

>>> print(os.getcwd())
C:\Python33
```

List Directories and Files

All files and sub-directories inside a directory can be retrieved using the `listdir()` method.

This method takes in a path and returns a list of subdirectories and files in that path. If no path is specified, it returns the list of subdirectories and files from the current working directory.

```
>>> print(os.getcwd())
C:\Python33

>>> os.listdir()
['DLLs',
'Doc',
'include',
'Lib',
'libs',
'LICENSE.txt',
'NEWS.txt',
'python.exe',
'pythonw.exe',
'README.txt',
'Scripts',
'tcl',
'Tools']

>>> os.listdir('G:\\')
['$RECYCLE.BIN',
'Movies',
'Music',
'Photos',
'Series',
'System Volume Information']
```

Making a New Directory

We can make a new directory using the `mkdir()` method.

This method takes in the path of the new directory. If the full path is not specified, the new directory is created in the current working directory.

```
>>> os.mkdir('test')

>>> os.listdir()
['test']
```

Renaming a Directory or a File

The `rename()` method can rename a directory or a file.

For renaming any directory or file, the `rename()` method takes in two basic arguments: the old name as the first argument and the new name as the second argument.

```
>>> os.listdir()
['test']

>>> os.rename('test','new_one')

>>> os.listdir()
['new_one']
```

Removing Directory or File

A file can be removed (deleted) using the `remove()` method.

Similarly, the `rmdir()` method removes an empty directory.

```
>>> os.listdir()
['new_one', 'old.txt']

>>> os.remove('old.txt')
>>> os.listdir()
['new_one']

>>> os.rmdir('new_one')
>>> os.listdir()
[]
```

Note: The `rmdir()` method can only remove empty directories.

In order to remove a non-empty directory, we can use the `rmtree()` method inside the `shutil` module.

```
>>> os.listdir()
['test']

>>> os.rmdir('test')
Traceback (most recent call last):
...
OSError: [WinError 145] The directory is not empty: 'test'

>>> import shutil

>>> shutil.rmtree('test')
>>> os.listdir()
[]
```

Python Errors and Built-in Exceptions

programiz.com/python-programming/exceptions

In this tutorial, you will learn about different types of errors and exceptions that are built-in to Python. They are raised whenever the Python interpreter encounters errors.

We can make certain mistakes while writing a program that lead to errors when we try to run it. A python program terminates as soon as it encounters an unhandled error. These errors can be broadly classified into two classes:

1. Syntax errors
2. Logical errors (Exceptions)

Python Syntax Errors

Error caused by not following the proper structure (syntax) of the language is called **syntax error** or **parsing error**.

Let's look at one example:

```
>>> if a < 3
  File "<interactive input>", line 1
    if a < 3
           ^
SyntaxError: invalid syntax
```

As shown in the example, an arrow indicates where the parser ran into the syntax error.

We can notice here that a colon `:` is missing in the `if` statement.

Python Logical Errors (Exceptions)

Errors that occur at runtime (after passing the syntax test) are called **exceptions** or **logical errors**.

For instance, they occur when we try to open a file(for reading) that does not exist (`FileNotFoundError`), try to divide a number by zero (`ZeroDivisionError`), or try to import a module that does not exist (`ImportError`).

Whenever these types of runtime errors occur, Python creates an exception object. If not handled properly, it prints a traceback to that error along with some details about why that error occurred.

Let's look at how Python treats these errors:

```
>>> 1 / 0
Traceback (most recent call last):
 File "<string>", line 301, in runcode
 File "<interactive input>", line 1, in <module>
ZeroDivisionError: division by zero

>>> open("imaginary.txt")
Traceback (most recent call last):
 File "<string>", line 301, in runcode
 File "<interactive input>", line 1, in <module>
FileNotFoundError: [Errno 2] No such file or directory: 'imaginary.txt'
```

Python Built-in Exceptions

Illegal operations can raise exceptions. There are plenty of built-in exceptions in Python that are raised when corresponding errors occur. We can view all the built-in exceptions using the built-in `local()` function as follows:

```
print(dir(locals()['__builtins__']))
```

`locals()['__builtins__']` will return a module of built-in exceptions, functions, and attributes. `dir` allows us to list these attributes as strings.

Some of the common built-in exceptions in Python programming along with the error that cause them are listed below:

Exception	Cause of Error
`AssertionError`	Raised when an `assert` statement fails.
`AttributeError`	Raised when attribute assignment or reference fails.
`EOFError`	Raised when the `input()` function hits end-of-file condition.
`FloatingPointError`	Raised when a floating point operation fails.
`GeneratorExit`	Raise when a generator's `close()` method is called.
`ImportError`	Raised when the imported module is not found.
`IndexError`	Raised when the index of a sequence is out of range.
`KeyError`	Raised when a key is not found in a dictionary.
`KeyboardInterrupt`	Raised when the user hits the interrupt key (`Ctrl+C` or `Delete`).
`MemoryError`	Raised when an operation runs out of memory.
`NameError`	Raised when a variable is not found in local or global scope.

`NotImplementedError`	Raised by abstract methods.
`OSError`	Raised when system operation causes system related error.
`OverflowError`	Raised when the result of an arithmetic operation is too large to be represented.
`ReferenceError`	Raised when a weak reference proxy is used to access a garbage collected referent.
`RuntimeError`	Raised when an error does not fall under any other category.
`StopIteration`	Raised by `next()` function to indicate that there is no further item to be returned by iterator.
`SyntaxError`	Raised by parser when syntax error is encountered.
`IndentationError`	Raised when there is incorrect indentation.
`TabError`	Raised when indentation consists of inconsistent tabs and spaces.
`SystemError`	Raised when interpreter detects internal error.
`SystemExit`	Raised by `sys.exit()` function.
`TypeError`	Raised when a function or operation is applied to an object of incorrect type.
`UnboundLocalError`	Raised when a reference is made to a local variable in a function or method, but no value has been bound to that variable.
`UnicodeError`	Raised when a Unicode-related encoding or decoding error occurs.
`UnicodeEncodeError`	Raised when a Unicode-related error occurs during encoding.
`UnicodeDecodeError`	Raised when a Unicode-related error occurs during decoding.
`UnicodeTranslateError`	Raised when a Unicode-related error occurs during translating.
`ValueError`	Raised when a function gets an argument of correct type but improper value.
`ZeroDivisionError`	Raised when the second operand of division or modulo operation is zero.

If required, we can also define our own exceptions in Python. To learn more about them, visit Python User-defined Exceptions.

We can handle these built-in and user-defined exceptions in Python using `try`, `except` and `finally` statements. To learn more about them, visit Python try, except and finally statements.

Python Exception Handling Using try, except and finally statement

programiz.com/python-programming/exception-handling

In this tutorial, you'll learn how to handle exceptions in your Python program using try, except and finally statements with the help of examples.

Exceptions in Python

Python has many built-in exceptions that are raised when your program encounters an error (something in the program goes wrong).

When these exceptions occur, the Python interpreter stops the current process and passes it to the calling process until it is handled. If not handled, the program will crash.

For example, let us consider a program where we have a function `A` that calls function `B`, which in turn calls function `C`. If an exception occurs in function `C` but is not handled in `C`, the exception passes to `B` and then to `A`.

If never handled, an error message is displayed and our program comes to a sudden unexpected halt.

Catching Exceptions in Python

In Python, exceptions can be handled using a `try` statement.

The critical operation which can raise an exception is placed inside the `try` clause. The code that handles the exceptions is written in the `except` clause.

We can thus choose what operations to perform once we have caught the exception. Here is a simple example.

```
# import module sys to get the type of exception
import sys

randomList = ['a', 0, 2]

for entry in randomList:
    try:
        print("The entry is", entry)
        r = 1/int(entry)
        break
    except:
        print("Oops!", sys.exc_info()[0], "occurred.")
        print("Next entry.")
        print()
print("The reciprocal of", entry, "is", r)
```

Output

```
The entry is a
Oops! <class 'ValueError'> occurred.
Next entry.

The entry is 0
Oops! <class 'ZeroDivisionError'> occured.
Next entry.

The entry is 2
The reciprocal of 2 is 0.5
```

In this program, we loop through the values of the *randomList* list. As previously mentioned, the portion that can cause an exception is placed inside the `try` block.

If no exception occurs, the `except` block is skipped and normal flow continues(for last value). But if any exception occurs, it is caught by the `except` block (first and second values).

Here, we print the name of the exception using the `exc_info()` function inside `sys` module. We can see that `a` causes `ValueError` and `0` causes `ZeroDivisionError`.

Since every exception in Python inherits from the base `Exception` class, we can also perform the above task in the following way:

```
# import module sys to get the type of exception
import sys

randomList = ['a', 0, 2]

for entry in randomList:
    try:
        print("The entry is", entry)
        r = 1/int(entry)
        break
    except Exception as e:
        print("Oops!", e.__class__, "occurred.")
        print("Next entry.")
        print()
print("The reciprocal of", entry, "is", r)
```

This program has the same output as the above program.

Catching Specific Exceptions in Python

In the above example, we did not mention any specific exception in the `except` clause.

This is not a good programming practice as it will catch all exceptions and handle every case in the same way. We can specify which exceptions an `except` clause should catch.

A `try` clause can have any number of `except` clauses to handle different exceptions, however, only one will be executed in case an exception occurs.

We can use a tuple of values to specify multiple exceptions in an except clause. Here is an example pseudo code.

```
try:
   # do something
   pass

except ValueError:
   # handle ValueError exception
   pass

except (TypeError, ZeroDivisionError):
   # handle multiple exceptions
   # TypeError and ZeroDivisionError
   pass

except:
   # handle all other exceptions
   pass
```

Raising Exceptions in Python

In Python programming, exceptions are raised when errors occur at runtime. We can also manually raise exceptions using the `raise` keyword.

We can optionally pass values to the exception to clarify why that exception was raised.

```
>>> raise KeyboardInterrupt
Traceback (most recent call last):
...
KeyboardInterrupt

>>> raise MemoryError("This is an argument")
Traceback (most recent call last):
...
MemoryError: This is an argument

>>> try:
...     a = int(input("Enter a positive integer: "))
...     if a <= 0:
...         raise ValueError("That is not a positive number!")
... except ValueError as ve:
...     print(ve)
...
Enter a positive integer: -2
That is not a positive number!
```

Python try with else clause

In some situations, you might want to run a certain block of code if the code block inside `try` ran without any errors. For these cases, you can use the optional `else` keyword with the `try` statement.

Note: Exceptions in the else clause are not handled by the preceding except clauses.

Let's look at an example:

```
# program to print the reciprocal of even numbers

try:
    num = int(input("Enter a number: "))
    assert num % 2 == 0
except:
    print("Not an even number!")
else:
    reciprocal = 1/num
    print(reciprocal)
```

Output

If we pass an odd number:

```
Enter a number: 1
Not an even number!
```

If we pass an even number, the reciprocal is computed and displayed.

```
Enter a number: 4
0.25
```

However, if we pass 0, we get `ZeroDivisionError` as the code block inside `else` is not handled by preceding `except`.

```
Enter a number: 0
Traceback (most recent call last):
  File "<string>", line 7, in <module>
    reciprocal = 1/num
ZeroDivisionError: division by zero
```

Python try...finally

The `try` statement in Python can have an optional `finally` clause. This clause is executed no matter what, and is generally used to release external resources.

For example, we may be connected to a remote data center through the network or working with a file or a Graphical User Interface (GUI).

In all these circumstances, we must clean up the resource before the program comes to a halt whether it successfully ran or not. These actions (closing a file, GUI or disconnecting from network) are performed in the `finally` clause to guarantee the execution.

Here is an example of file operations to illustrate this.

```
try:
   f = open("test.txt",encoding = 'utf-8')
   # perform file operations
finally:
   f.close()
```

This type of construct makes sure that the file is closed even if an exception occurs during the program execution.

Python Custom Exceptions

programiz.com/python-programming/user-defined-exception

In this tutorial, you will learn how to define custom exceptions depending upon your requirements with the help of examples.

Python has numerous built-in exceptions that force your program to output an error when something in the program goes wrong.

However, sometimes you may need to create your own custom exceptions that serve your purpose.

Creating Custom Exceptions

In Python, users can define custom exceptions by creating a new class. This exception class has to be derived, either directly or indirectly, from the built-in `Exception` class. Most of the built-in exceptions are also derived from this class.

```
>>> class CustomError(Exception):
...     pass
...

>>> raise CustomError
Traceback (most recent call last):
...
__main__.CustomError

>>> raise CustomError("An error occurred")
Traceback (most recent call last):
...
__main__.CustomError: An error occurred
```

Here, we have created a user-defined exception called `CustomError` which inherits from the `Exception` class. This new exception, like other exceptions, can be raised using the `raise` statement with an optional error message.

When we are developing a large Python program, it is a good practice to place all the user-defined exceptions that our program raises in a separate file. Many standard modules do this. They define their exceptions separately as `exceptions.py` or `errors.py` (generally but not always).

User-defined exception class can implement everything a normal class can do, but we generally make them simple and concise. Most implementations declare a custom base class and derive others exception classes from this base class. This concept is made clearer in the following example.

Example: User-Defined Exception in Python

In this example, we will illustrate how user-defined exceptions can be used in a program to raise and catch errors.

This program will ask the user to enter a number until they guess a stored number correctly. To help them figure it out, a hint is provided whether their guess is greater than or less than the stored number.

```
# define Python user-defined exceptions
class Error(Exception):
    """Base class for other exceptions"""
    pass


class ValueTooSmallError(Error):
    """Raised when the input value is too small"""
    pass


class ValueTooLargeError(Error):
    """Raised when the input value is too large"""
    pass


# you need to guess this number
number = 10

# user guesses a number until he/she gets it right
while True:
    try:
        i_num = int(input("Enter a number: "))
        if i_num < number:
            raise ValueTooSmallError
        elif i_num > number:
            raise ValueTooLargeError
        break
    except ValueTooSmallError:
        print("This value is too small, try again!")
        print()
    except ValueTooLargeError:
        print("This value is too large, try again!")
        print()

print("Congratulations! You guessed it correctly.")
```

Here is a sample run of this program.

```
Enter a number: 12
This value is too large, try again!

Enter a number: 0
This value is too small, try again!

Enter a number: 8
This value is too small, try again!

Enter a number: 10
Congratulations! You guessed it correctly.
```

We have defined a base class called `Error`.

The other two exceptions (`ValueTooSmallError` and `ValueTooLargeError`) that are actually raised by our program are derived from this class. This is the standard way to define user-defined exceptions in Python programming, but you are not limited to this way only.

Customizing Exception Classes

We can further customize this class to accept other arguments as per our needs.

To learn about customizing the Exception classes, you need to have the basic knowledge of Object-Oriented programming.

Visit Python Object Oriented Programming to start learning about Object-Oriented programming in Python.

Let's look at one example:

```
class SalaryNotInRangeError(Exception):
    """Exception raised for errors in the input salary.

    Attributes:
        salary -- input salary which caused the error
        message -- explanation of the error
    """

    def __init__(self, salary, message="Salary is not in (5000, 15000) range"):
        self.salary = salary
        self.message = message
        super().__init__(self.message)


salary = int(input("Enter salary amount: "))
if not 5000 < salary < 15000:
    raise SalaryNotInRangeError(salary)
```

Output

```
Enter salary amount: 2000
Traceback (most recent call last):
  File "<string>", line 17, in <module>
    raise SalaryNotInRangeError(salary)
__main__.SalaryNotInRangeError: Salary is not in (5000, 15000) range
```

Here, we have overridden the constructor of the `Exception` class to accept our own custom arguments `salary` and `message`. Then, the constructor of the parent `Exception` class is called manually with the `self.message` argument using `super()`.

The custom `self.salary` attribute is defined to be used later.

The inherited `__str__` method of the `Exception` class is then used to display the corresponding message when `SalaryNotInRangeError` is raised.

We can also customize the `__str__` method itself by overriding it.

```
class SalaryNotInRangeError(Exception):
    """Exception raised for errors in the input salary.

    Attributes:
        salary -- input salary which caused the error
        message -- explanation of the error
    """

    def __init__(self, salary, message="Salary is not in (5000, 15000) range"):
        self.salary = salary
        self.message = message
        super().__init__(self.message)

    def __str__(self):
        return f'{self.salary} -> {self.message}'


salary = int(input("Enter salary amount: "))
if not 5000 < salary < 15000:
    raise SalaryNotInRangeError(salary)
```

Output

```
Enter salary amount: 2000
Traceback (most recent call last):
  File "/home/bsoyuj/Desktop/Untitled-1.py", line 20, in <module>
    raise SalaryNotInRangeError(salary)
__main__.SalaryNotInRangeError: 2000 -> Salary is not in (5000, 15000) range
```

To learn more about how you can handle exceptions in Python, visit Python Exception Handling.

Python Object Oriented Programming

programiz.com/python-programming/object-oriented-programming

In this tutorial, you'll learn about Object-Oriented Programming (OOP) in Python and its fundamental concept with the help of examples.

Object Oriented Programming

Python is a multi-paradigm programming language. It supports different programming approaches.

One of the popular approaches to solve a programming problem is by creating objects. This is known as Object-Oriented Programming (OOP).

An object has two characteristics:

- attributes
- behavior

Let's take an example:

A parrot is an object, as it has the following properties:

- name, age, color as attributes
- singing, dancing as behavior

The concept of OOP in Python focuses on creating reusable code. This concept is also known as DRY (Don't Repeat Yourself).

In Python, the concept of OOP follows some basic principles:

Class

A class is a blueprint for the object.

We can think of class as a sketch of a parrot with labels. It contains all the details about the name, colors, size etc. Based on these descriptions, we can study about the parrot. Here, a parrot is an object.

The example for class of parrot can be :

```
class Parrot:
    pass
```

Here, we use the `class` keyword to define an empty class *Parrot*. From class, we construct instances. An instance is a specific object created from a particular class.

Object

An object (instance) is an instantiation of a class. When class is defined, only the description for the object is defined. Therefore, no memory or storage is allocated.

The example for object of parrot class can be:

```
obj = Parrot()
```

Here, *obj* is an object of class `Parrot`.

Suppose we have details of parrots. Now, we are going to show how to build the class and objects of parrots.

Example 1: Creating Class and Object in Python

```
class Parrot:

    # class attribute
    species = "bird"

    # instance attribute
    def __init__(self, name, age):
        self.name = name
        self.age = age

# instantiate the Parrot class
blu = Parrot("Blu", 10)
woo = Parrot("Woo", 15)

# access the class attributes
print("Blu is a {}".format(blu.__class__.species))
print("Woo is also a {}".format(woo.__class__.species))

# access the instance attributes
print("{} is {} years old".format( blu.name, blu.age))
print("{} is {} years old".format( woo.name, woo.age))
```

Output

```
Blu is a bird
Woo is also a bird
Blu is 10 years old
Woo is 15 years old
```

In the above program, we created a class with the name *Parrot*. Then, we define attributes. The attributes are a characteristic of an object.

These attributes are defined inside the `__init__` method of the class. It is the initializer method that is first run as soon as the object is created.

Then, we create instances of the *Parrot* class. Here, *blu* and *woo* are references (value) to our new objects.

We can access the class attribute using `__class__.species`. Class attributes are the same for all instances of a class. Similarly, we access the instance attributes using `blu.name` and `blu.age`. However, instance attributes are different for every instance of a class.

To learn more about classes and objects, go to Python Classes and Objects

---

Methods

Methods are functions defined inside the body of a class. They are used to define the behaviors of an object.

Example 2 : Creating Methods in Python

```
class Parrot:

    # instance attributes
    def __init__(self, name, age):
        self.name = name
        self.age = age

    # instance method
    def sing(self, song):
        return "{} sings {}".format(self.name, song)

    def dance(self):
        return "{} is now dancing".format(self.name)

# instantiate the object
blu = Parrot("Blu", 10)

# call our instance methods
print(blu.sing("'Happy'"))
print(blu.dance())
```

Output

```
Blu sings 'Happy'
Blu is now dancing
```

In the above program, we define two methods i.e `sing()` and `dance()`. These are called instance methods because they are called on an instance object i.e `blu`.

Inheritance

Inheritance is a way of creating a new class for using details of an existing class without modifying it. The newly formed class is a derived class (or child class). Similarly, the existing class is a base class (or parent class).

Example 3: Use of Inheritance in Python

```
# parent class
class Bird:

    def __init__(self):
        print("Bird is ready")

    def whoisThis(self):
        print("Bird")

    def swim(self):
        print("Swim faster")

# child class
class Penguin(Bird):

    def __init__(self):
        # call super() function
        super().__init__()
        print("Penguin is ready")

    def whoisThis(self):
        print("Penguin")

    def run(self):
        print("Run faster")

peggy = Penguin()
peggy.whoisThis()
peggy.swim()
peggy.run()
```

Output

```
Bird is ready
Penguin is ready
Penguin
Swim faster
Run faster
```

In the above program, we created two classes i.e. *Bird* (parent class) and *Penguin* (child class). The child class inherits the functions of parent class. We can see this from the `swim()` method.

Again, the child class modified the behavior of the parent class. We can see this from the `whoisThis()` method. Furthermore, we extend the functions of the parent class, by creating a new `run()` method.

Additionally, we use the `super()` function inside the `__init__()` method. This allows us to run the `__init__()` method of the parent class inside the child class.

Encapsulation

Using OOP in Python, we can restrict access to methods and variables. This prevents data from direct modification which is called encapsulation. In Python, we denote private attributes using underscore as the prefix i.e single `_` or double `__`.

Example 4: Data Encapsulation in Python

```
class Computer:

    def __init__(self):
        self.__maxprice = 900

    def sell(self):
        print("Selling Price: {}".format(self.__maxprice))

    def setMaxPrice(self, price):
        self.__maxprice = price

c = Computer()
c.sell()

# change the price
c.__maxprice = 1000
c.sell()

# using setter function
c.setMaxPrice(1000)
c.sell()
```

Output

```
Selling Price: 900
Selling Price: 900
Selling Price: 1000
```

In the above program, we defined a *Computer* class.

We used `__init__()` method to store the maximum selling price of `Computer`. Here, notice the code

```
c.__maxprice = 1000
```

Here, we have tried to modify the value of *__maxprice* outside of the class. However, since *__maxprice* is a private variable, this modification is not seen on the output.

As shown, to change the value, we have to use a setter function i.e `setMaxPrice()` which takes price as a parameter.

Polymorphism

Polymorphism is an ability (in OOP) to use a common interface for multiple forms (data types).

Suppose, we need to color a shape, there are multiple shape options (rectangle, square, circle). However we could use the same method to color any shape. This concept is called Polymorphism.

Example 5: Using Polymorphism in Python

```
class Parrot:

    def fly(self):
        print("Parrot can fly")

    def swim(self):
        print("Parrot can't swim")

class Penguin:

    def fly(self):
        print("Penguin can't fly")

    def swim(self):
        print("Penguin can swim")

# common interface
def flying_test(bird):
    bird.fly()

#instantiate objects
blu = Parrot()
peggy = Penguin()

# passing the object
flying_test(blu)
flying_test(peggy)
```

Output

```
Parrot can fly
Penguin can't fly
```

In the above program, we defined two classes *Parrot* and *Penguin*. Each of them have a common `fly()` method. However, their functions are different.

To use polymorphism, we created a common interface i.e `flying_test()` function that takes any object and calls the object's `fly()` method. Thus, when we passed the *blu* and *peggy* objects in the `flying_test()` function, it ran effectively.

Key Points to Remember:

- Object-Oriented Programming makes the program easy to understand as well as efficient.
- Since the class is sharable, the code can be reused.
- Data is safe and secure with data abstraction.

- Polymorphism allows the same interface for different objects, so programmers can write efficient code.

Python Objects and Classes

programiz.com/python-programming/class

In this tutorial, you will learn about the core functionality of Python objects and classes. You'll learn what a class is, how to create it and use it in your program.

Python is an object-oriented programming language. Unlike procedure-oriented programming, where the main emphasis is on functions, object-oriented programming stresses on objects.

An object is simply a collection of data (variables) and methods (functions) that act on those data. Similarly, a class is a blueprint for that object.

We can think of a class as a sketch (prototype) of a house. It contains all the details about the floors, doors, windows, etc. Based on these descriptions we build the house. House is the object.

As many houses can be made from a house's blueprint, we can create many objects from a class. An object is also called an instance of a class and the process of creating this object is called **instantiation**.

Defining a Class in Python

Like function definitions begin with the def keyword in Python, class definitions begin with a class keyword.

The first string inside the class is called docstring and has a brief description of the class. Although not mandatory, this is highly recommended.

Here is a simple class definition.

```
class MyNewClass:
    '''This is a docstring. I have created a new class'''
    pass
```

A class creates a new local namespace where all its attributes are defined. Attributes may be data or functions.

There are also special attributes in it that begins with double underscores `__`. For example, `__doc__` gives us the docstring of that class.

As soon as we define a class, a new class object is created with the same name. This class object allows us to access the different attributes as well as to instantiate new objects of that class.

```
class Person:
    "This is a person class"
    age = 10

    def greet(self):
        print('Hello')


# Output: 10
print(Person.age)

# Output: <function Person.greet>
print(Person.greet)

# Output: "This is a person class"
print(Person.__doc__)
```

Output

```
10
<function Person.greet at 0x7fc78c6e8160>
This is a person class
```

Creating an Object in Python

We saw that the class object could be used to access different attributes.

It can also be used to create new object instances (instantiation) of that class. The procedure to create an object is similar to a function call.

```
>>> harry = Person()
```

This will create a new object instance named *harry*. We can access the attributes of objects using the object name prefix.

Attributes may be data or method. Methods of an object are corresponding functions of that class.

This means to say, since `Person.greet` is a function object (attribute of class), `Person.greet` will be a method object.

```
class Person:
    "This is a person class"
    age = 10

    def greet(self):
        print('Hello')


# create a new object of Person class
harry = Person()

# Output: <function Person.greet>
print(Person.greet)

# Output: <bound method Person.greet of <__main__.Person object>>
print(harry.greet)

# Calling object's greet() method
# Output: Hello
harry.greet()
```

Output

```
<function Person.greet at 0x7fd288e4e160>
<bound method Person.greet of <__main__.Person object at 0x7fd288e9fa30>>
Hello
```

You may have noticed the `self` parameter in function definition inside the class but we called the method simply as `harry.greet()` without any arguments. It still worked.

This is because, whenever an object calls its method, the object itself is passed as the first argument. So, `harry.greet()` translates into `Person.greet(harry)`.

In general, calling a method with a list of n arguments is equivalent to calling the corresponding function with an argument list that is created by inserting the method's object before the first argument.

For these reasons, the first argument of the function in class must be the object itself. This is conventionally called *self*. It can be named otherwise but we highly recommend to follow the convention.

Now you must be familiar with class object, instance object, function object, method object and their differences.

Constructors in Python

Class functions that begin with double underscore `__` are called special functions as they have special meaning.

Of one particular interest is the `__init__()` function. This special function gets called whenever a new object of that class is instantiated.

This type of function is also called constructors in Object Oriented Programming (OOP). We normally use it to initialize all the variables.

```
class ComplexNumber:
    def __init__(self, r=0, i=0):
        self.real = r
        self.imag = i

    def get_data(self):
        print(f'{self.real}+{self.imag}j')


# Create a new ComplexNumber object
num1 = ComplexNumber(2, 3)

# Call get_data() method
# Output: 2+3j
num1.get_data()

# Create another ComplexNumber object
# and create a new attribute 'attr'
num2 = ComplexNumber(5)
num2.attr = 10

# Output: (5, 0, 10)
print((num2.real, num2.imag, num2.attr))

# but c1 object doesn't have attribute 'attr'
# AttributeError: 'ComplexNumber' object has no attribute 'attr'
print(num1.attr)
```

Output

```
2+3j
(5, 0, 10)
Traceback (most recent call last):
  File "<string>", line 27, in <module>
    print(num1.attr)
AttributeError: 'ComplexNumber' object has no attribute 'attr'
```

In the above example, we defined a new class to represent complex numbers. It has two functions, `__init__()` to initialize the variables (defaults to zero) and `get_data()` to display the number properly.

An interesting thing to note in the above step is that attributes of an object can be created on the fly. We created a new attribute *attr* for object *num2* and read it as well. But this does not create that attribute for object *num1*.

Deleting Attributes and Objects

Any attribute of an object can be deleted anytime, using the `del` statement. Try the following on the Python shell to see the output.

```
>>> num1 = ComplexNumber(2,3)
>>> del num1.imag
>>> num1.get_data()
Traceback (most recent call last):
...
AttributeError: 'ComplexNumber' object has no attribute 'imag'

>>> del ComplexNumber.get_data
>>> num1.get_data()
Traceback (most recent call last):
...
AttributeError: 'ComplexNumber' object has no attribute 'get_data'
```

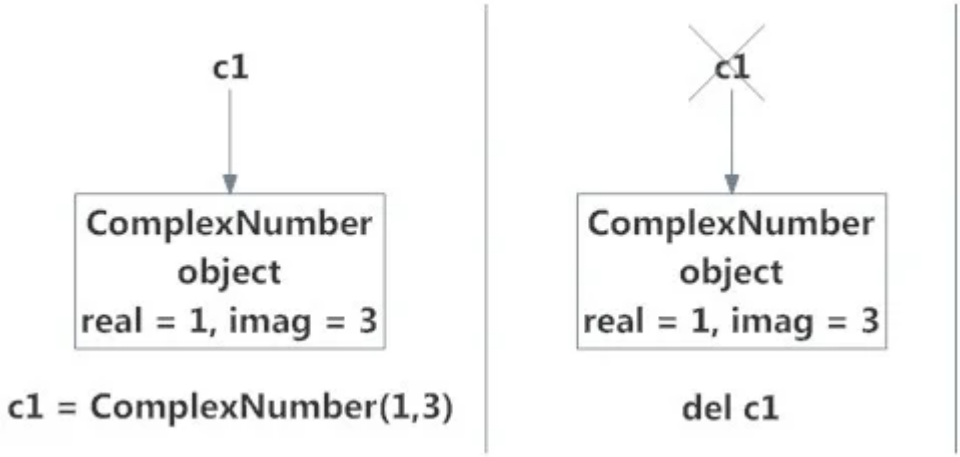
We can even delete the object itself, using the del statement.

```
>>> c1 = ComplexNumber(1,3)
>>> del c1
>>> c1
Traceback (most recent call last):
...
NameError: name 'c1' is not defined
```

Actually, it is more complicated than that. When we do `c1 = ComplexNumber(1,3)`, a new instance object is created in memory and the name *c1* binds with it.

On the command `del c1`, this binding is removed and the name *c1* is deleted from the corresponding namespace. The object however continues to exist in memory and if no other name is bound to it, it is later automatically destroyed.

This automatic destruction of unreferenced objects in Python is also called garbage collection.



Deleting objects in Python removes the name binding

Python Inheritance

programiz.com/python-programming/inheritance

Inheritance enables us to define a class that takes all the functionality from a parent class and allows us to add more. In this tutorial, you will learn to use inheritance in Python.

Inheritance in Python

Inheritance is a powerful feature in object oriented programming.

It refers to defining a new class with little or no modification to an existing class. The new class is called **derived (or child) class** and the one from which it inherits is called the **base (or parent) class**.

Python Inheritance Syntax

```
class BaseClass:
  Body of base class
class DerivedClass(BaseClass):
  Body of derived class
```

Derived class inherits features from the base class where new features can be added to it. This results in re-usability of code.

Example of Inheritance in Python

To demonstrate the use of inheritance, let us take an example.

A polygon is a closed figure with 3 or more sides. Say, we have a class called `Polygon` defined as follows.

```
class Polygon:
    def __init__(self, no_of_sides):
        self.n = no_of_sides
        self.sides = [0 for i in range(no_of_sides)]

    def inputSides(self):
        self.sides = [float(input("Enter side "+str(i+1)+" : ")) for i in
range(self.n)]

    def dispSides(self):
        for i in range(self.n):
            print("Side",i+1,"is",self.sides[i])
```

This class has data attributes to store the number of sides *n* and magnitude of each side as a list called *sides*.

The `inputSides()` method takes in the magnitude of each side and `dispSides()` displays these side lengths.

A triangle is a polygon with 3 sides. So, we can create a class called *Triangle* which inherits from *Polygon*. This makes all the attributes of *Polygon* class available to the *Triangle* class.

We don't need to define them again (code reusability). *Triangle* can be defined as follows.

```
class Triangle(Polygon):
    def __init__(self):
        Polygon.__init__(self,3)

    def findArea(self):
        a, b, c = self.sides
        # calculate the semi-perimeter
        s = (a + b + c) / 2
        area = (s*(s-a)*(s-b)*(s-c)) ** 0.5
        print('The area of the triangle is %0.2f' %area)
```

However, class `Triangle` has a new method `findArea()` to find and print the area of the triangle. Here is a sample run.

```
>>> t = Triangle()

>>> t.inputSides()
Enter side 1 : 3
Enter side 2 : 5
Enter side 3 : 4

>>> t.dispSides()
Side 1 is 3.0
Side 2 is 5.0
Side 3 is 4.0

>>> t.findArea()
The area of the triangle is 6.00
```

We can see that even though we did not define methods like `inputSides()` or `dispSides()` for class `Triangle` separately, we were able to use them.

If an attribute is not found in the class itself, the search continues to the base class. This repeats recursively, if the base class is itself derived from other classes.

Method Overriding in Python

In the above example, notice that `__init__()` method was defined in both classes, *Triangle* as well *Polygon*. When this happens, the method in the derived class overrides that in the base class. This is to say, `__init__()` in *Triangle* gets preference over the `__init__` in *Polygon*.

Generally when overriding a base method, we tend to extend the definition rather than simply replace it. The same is being done by calling the method in base class from the one in derived class (calling `Polygon.__init__()` from `__init__()` in `Triangle`).

A better option would be to use the built-in function `super()`. So, `super().__init__(3)` is equivalent to `Polygon.__init__(self,3)` and is preferred. To learn more about the `super()` function in Python, visit Python super() function.

Two built-in functions `isinstance()` and `issubclass()` are used to check inheritances.

The function `isinstance()` returns `True` if the object is an instance of the class or other classes derived from it. Each and every class in Python inherits from the base class `object`.

```
>>> isinstance(t,Triangle)
True

>>> isinstance(t,Polygon)
True

>>> isinstance(t,int)
False

>>> isinstance(t,object)
True
```

Similarly, `issubclass()` is used to check for class inheritance.

```
>>> issubclass(Polygon,Triangle)
False

>>> issubclass(Triangle,Polygon)
True

>>> issubclass(bool,int)
True
```

Python Multiple Inheritance

programiz.com/python-programming/multiple-inheritance

In this tutorial, you'll learn about multiple inheritance in Python and how to use it in your program. You'll also learn about multi-level inheritance and the method resolution order.

A class can be derived from more than one base class in Python, similar to C++. This is called multiple inheritance.

In multiple inheritance, the features of all the base classes are inherited into the derived class. The syntax for multiple inheritance is similar to single inheritance.

Example

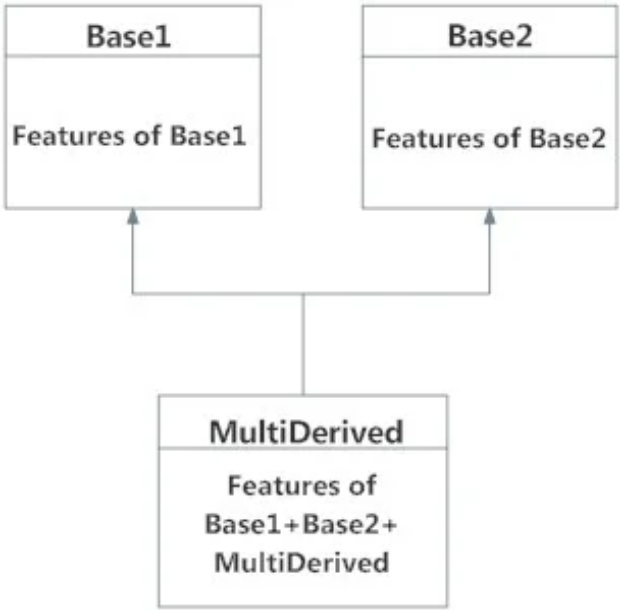
```
class Base1:
    pass

class Base2:
    pass

class MultiDerived(Base1, Base2):
    pass
```

Here, the *MultiDerived* class is derived from *Base1* and *Base2* classes.

The *MultiDerived* class inherits from both *Base1* and *Base2* classes.



Multiple Inheritance in Python

Python Multilevel Inheritance

We can also inherit from a derived class. This is called multilevel inheritance. It can be of any depth in Python.

In multilevel inheritance, features of the base class and the derived class are inherited into the new derived class.

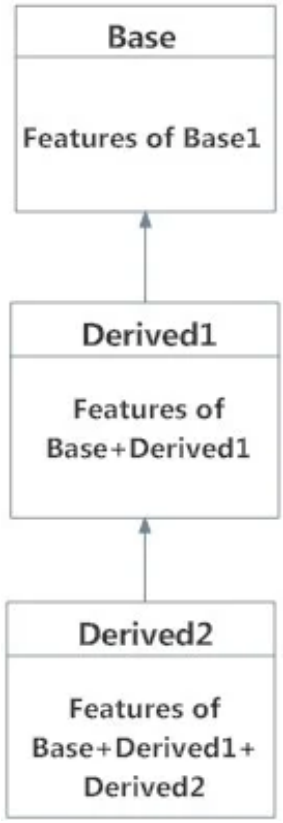
An example with corresponding visualization is given below.

```
class Base:
    pass

class Derived1(Base):
    pass

class Derived2(Derived1):
    pass
```

Here, the *Derived1* class is derived from the *Base* class, and the *Derived2* class is derived from the *Derived1* class.



Multilevel Inheritance in Python

Method Resolution Order in Python

Every class in Python is derived from the `object` class. It is the most base type in Python.

So technically, all other classes, either built-in or user-defined, are derived classes and all objects are instances of the `object` class.

```
# Output: True
print(issubclass(list,object))

# Output: True
print(isinstance(5.5,object))

# Output: True
print(isinstance("Hello",object))
```

In the multiple inheritance scenario, any specified attribute is searched first in the current class. If not found, the search continues into parent classes in depth-first, left-right fashion without searching the same class twice.

So, in the above example of `MultiDerived` class the search order is [`MultiDerived` , `Base1` , `Base2` , `object`]. This order is also called linearization of `MultiDerived` class and the set of rules used to find this order is called **Method Resolution Order (MRO)**.

MRO must prevent local precedence ordering and also provide monotonicity. It ensures that a class always appears before its parents. In case of multiple parents, the order is the same as tuples of base classes.

MRO of a class can be viewed as the `__mro__` attribute or the `mro()` method. The former returns a tuple while the latter returns a list.

```
>>> MultiDerived.__mro__
(<class '__main__.MultiDerived'>,
 <class '__main__.Base1'>,
 <class '__main__.Base2'>,
 <class 'object'>)

>>> MultiDerived.mro()
[<class '__main__.MultiDerived'>,
 <class '__main__.Base1'>,
 <class '__main__.Base2'>,
 <class 'object'>]
```

Here is a little more complex multiple inheritance example and its visualization along with the MRO.

```
# Demonstration of MRO

class X:
    pass


class Y:
    pass


class Z:
    pass

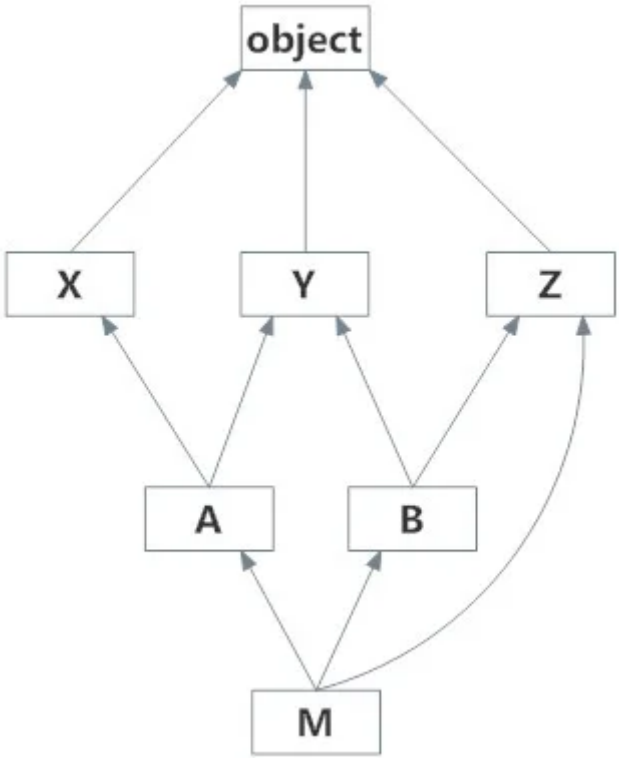

class A(X, Y):
    pass


class B(Y, Z):
    pass


class M(B, A, Z):
    pass

# Output:
# [<class '__main__.M'>, <class
'__main__.B'>,
#  <class '__main__.A'>, <class
'__main__.X'>,
#  <class '__main__.Y'>, <class
'__main__.Z'>,
#  <class 'object'>]

print(M.mro())
```



Visualizing Multiple Inheritance in Python

Output

```
[<class '__main__.M'>, <class '__main__.B'>, <class '__main__.A'>, <class
'__main__.X'>, <class '__main__.Y'>, <class '__main__.Z'>, <class 'object'>]
```

To know the actual algorithm on how MRO is calculated, visit Discussion on MRO.

Python Operator Overloading

programiz.com/python-programming/operator-overloading

You can change the meaning of an operator in Python depending upon the operands used. In this tutorial, you will learn how to use operator overloading in Python Object Oriented Programming.

Python operators work for built-in classes. But the same operator behaves differently with different types. For example, the `+` operator will perform arithmetic addition on two numbers, merge two lists, or concatenate two strings.

This feature in Python that allows the same operator to have different meaning according to the context is called operator overloading.

So what happens when we use them with objects of a user-defined class? Let us consider the following class, which tries to simulate a point in 2-D coordinate system.

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y


p1 = Point(1, 2)
p2 = Point(2, 3)
print(p1+p2)
```

Output

```
Traceback (most recent call last):
  File "<string>", line 9, in <module>
    print(p1+p2)
TypeError: unsupported operand type(s) for +: 'Point' and 'Point'
```

Here, we can see that a `TypeError` was raised, since Python didn't know how to add two `Point` objects together.

However, we can achieve this task in Python through operator overloading. But first, let's get a notion about special functions.

Python Special Functions

Class functions that begin with double underscore `__` are called special functions in Python.

These functions are not the typical functions that we define for a class. The `__init__()` function we defined above is one of them. It gets called every time we create a new object of that class.

There are numerous other special functions in Python. Visit Python Special Functions to learn more about them.

Using special functions, we can make our class compatible with built-in functions.

```
>>> p1 = Point(2,3)
>>> print(p1)
<__main__.Point object at 0x00000000031F8CC0>
```

Suppose we want the `print()` function to print the coordinates of the `Point` object instead of what we got. We can define a `__str__()` method in our class that controls how the object gets printed. Let's look at how we can achieve this:

```
class Point:
    def __init__(self, x = 0, y = 0):
        self.x = x
        self.y = y

    def __str__(self):
        return "({0},{1})".format(self.x,self.y)
```

Now let's try the `print()` function again.

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return "({0}, {1})".format(self.x, self.y)


p1 = Point(2, 3)
print(p1)
```

Output

```
(2, 3)
```

That's better. Turns out, that this same method is invoked when we use the built-in function `str()` or `format()`.

```
>>> str(p1)
'(2,3)'

>>> format(p1)
'(2,3)'
```

So, when you use `str(p1)` or `format(p1)`, Python internally calls the `p1.__str__()` method. Hence the name, special functions.

Now let's go back to operator overloading.

Overloading the + Operator

To overload the `+` operator, we will need to implement `__add__()` function in the class. With great power comes great responsibility. We can do whatever we like, inside this function. But it is more sensible to return a `Point` object of the coordinate sum.

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return "({0},{1})".format(self.x, self.y)

    def __add__(self, other):
        x = self.x + other.x
        y = self.y + other.y
        return Point(x, y)
```

Now let's try the addition operation again:

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return "({0},{1})".format(self.x, self.y)

    def __add__(self, other):
        x = self.x + other.x
        y = self.y + other.y
        return Point(x, y)


p1 = Point(1, 2)
p2 = Point(2, 3)

print(p1+p2)
```

Output

```
(3,5)
```

What actually happens is that, when you use `p1 + p2` , Python calls `p1.__add__(p2)` which in turn is `Point.__add__(p1,p2)` . After this, the addition operation is carried out the way we specified.

Similarly, we can overload other operators as well. The special function that we need to implement is tabulated below.

Operator	Expression	Internally
Addition	`p1 + p2`	`p1.__add__(p2)`
Subtraction	`p1 - p2`	`p1.__sub__(p2)`

Multiplication	`p1 * p2`	`p1.__mul__(p2)`
Power	`p1 ** p2`	`p1.__pow__(p2)`
Division	`p1 / p2`	`p1.__truediv__(p2)`
Floor Division	`p1 // p2`	`p1.__floordiv__(p2)`
Remainder (modulo)	`p1 % p2`	`p1.__mod__(p2)`
Bitwise Left Shift	`p1 << p2`	`p1.__lshift__(p2)`
Bitwise Right Shift	`p1 >> p2`	`p1.__rshift__(p2)`
Bitwise AND	`p1 & p2`	`p1.__and__(p2)`
Bitwise OR	`p1 | p2`	`p1.__or__(p2)`
Bitwise XOR	`p1 ^ p2`	`p1.__xor__(p2)`
Bitwise NOT	`~p1`	`p1.__invert__()`

Overloading Comparison Operators

Python does not limit operator overloading to arithmetic operators only. We can overload comparison operators as well.

Suppose we wanted to implement the less than symbol `<` symbol in our `Point` class.

Let us compare the magnitude of these points from the origin and return the result for this purpose. It can be implemented as follows.

```
# overloading the less than operator
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return "({0},{1})".format(self.x, self.y)

    def __lt__(self, other):
        self_mag = (self.x ** 2) + (self.y ** 2)
        other_mag = (other.x ** 2) + (other.y ** 2)
        return self_mag < other_mag

p1 = Point(1,1)
p2 = Point(-2,-3)
p3 = Point(1,-1)

# use less than
print(p1<p2)
print(p2<p3)
print(p1<p3)
```

Output

```
True
False
False
```

Similarly, the special functions that we need to implement, to overload other comparison operators are tabulated below.

Operator	Expression	Internally
Less than	`p1 < p2`	`p1.__lt__(p2)`
Less than or equal to	`p1 <= p2`	`p1.__le__(p2)`
Equal to	`p1 == p2`	`p1.__eq__(p2)`
Not equal to	`p1 != p2`	`p1.__ne__(p2)`
Greater than	`p1 > p2`	`p1.__gt__(p2)`
Greater than or equal to	`p1 >= p2`	`p1.__ge__(p2)`

Python Decorators

programiz.com/python-programming/decorator

A decorator takes in a function, adds some functionality and returns it. In this tutorial, you will learn how you can create a decorator and why you should use it.

Decorators in Python

Python has an interesting feature called **decorators** to add functionality to an existing code.

This is also called **metaprogramming** because a part of the program tries to modify another part of the program at compile time.

Prerequisites for learning decorators

In order to understand about decorators, we must first know a few basic things in Python.

We must be comfortable with the fact that everything in Python (Yes! Even classes), are objects. Names that we define are simply identifiers bound to these objects. Functions are no exceptions, they are objects too (with attributes). Various different names can be bound to the same function object.

Here is an example.

```
def first(msg):
    print(msg)


first("Hello")

second = first
second("Hello")
```

Output

```
Hello
Hello
```

When you run the code, both functions `first` and `second` give the same output. Here, the names `first` and `second` refer to the same function object.

Now things start getting weirder.

Functions can be passed as arguments to another function.

If you have used functions like `map`, `filter` and `reduce` in Python, then you already know about this.

Such functions that take other functions as arguments are also called **higher order functions**. Here is an example of such a function.

```
def inc(x):
    return x + 1


def dec(x):
    return x - 1


def operate(func, x):
    result = func(x)
    return result
```

We invoke the function as follows.

```
>>> operate(inc,3)
4
>>> operate(dec,3)
2
```

Furthermore, a function can return another function.

```
def is_called():
    def is_returned():
        print("Hello")
    return is_returned


new = is_called()

# Outputs "Hello"
new()
```

Output

```
Hello
```

Here, `is_returned()` is a nested function which is defined and returned each time we call `is_called()`.

Finally, we must know about Closures in Python.

Getting back to Decorators

Functions and methods are called **callable** as they can be called.

In fact, any object which implements the special `__call__()` method is termed callable. So, in the most basic sense, a decorator is a callable that returns a callable.

Basically, a decorator takes in a function, adds some functionality and returns it.

```
def make_pretty(func):
    def inner():
        print("I got decorated")
        func()
    return inner


def ordinary():
    print("I am ordinary")
```

When you run the following codes in shell,

```
>>> ordinary()
I am ordinary

>>> # let's decorate this ordinary function
>>> pretty = make_pretty(ordinary)
>>> pretty()
I got decorated
I am ordinary
```

In the example shown above, `make_pretty()` is a decorator. In the assignment step:

```
pretty = make_pretty(ordinary)
```

The function `ordinary()` got decorated and the returned function was given the name `pretty`.

We can see that the decorator function added some new functionality to the original function. This is similar to packing a gift. The decorator acts as a wrapper. The nature of the object that got decorated (actual gift inside) does not alter. But now, it looks pretty (since it got decorated).

Generally, we decorate a function and reassign it as,

```
ordinary = make_pretty(ordinary).
```

This is a common construct and for this reason, Python has a syntax to simplify this.

We can use the `@` symbol along with the name of the decorator function and place it above the definition of the function to be decorated. For example,

```
@make_pretty
def ordinary():
    print("I am ordinary")
```

is equivalent to

```
def ordinary():
    print("I am ordinary")
ordinary = make_pretty(ordinary)
```

This is just a syntactic sugar to implement decorators.

Decorating Functions with Parameters

The above decorator was simple and it only worked with functions that did not have any parameters. What if we had functions that took in parameters like:

```
def divide(a, b):
    return a/b
```

This function has two parameters, *a* and *b*. We know it will give an error if we pass in *b* as 0.

```
>>> divide(2,5)
0.4
>>> divide(2,0)
Traceback (most recent call last):
...
ZeroDivisionError: division by zero
```

Now let's make a decorator to check for this case that will cause the error.

```
def smart_divide(func):
    def inner(a, b):
        print("I am going to divide", a, "and", b)
        if b == 0:
            print("Whoops! cannot divide")
            return

        return func(a, b)
    return inner


@smart_divide
def divide(a, b):
    print(a/b)
```

This new implementation will return `None` if the error condition arises.

```
>>> divide(2,5)
I am going to divide 2 and 5
0.4

>>> divide(2,0)
I am going to divide 2 and 0
Whoops! cannot divide
```

In this manner, we can decorate functions that take parameters.

A keen observer will notice that parameters of the nested `inner()` function inside the decorator is the same as the parameters of functions it decorates. Taking this into account, now we can make general decorators that work with any number of parameters.

In Python, this magic is done as `function(*args, **kwargs)`. In this way, `args` will be the tuple of positional arguments and `kwargs` will be the dictionary of keyword arguments. An example of such a decorator will be:

```
def works_for_all(func):
    def inner(*args, **kwargs):
        print("I can decorate any function")
        return func(*args, **kwargs)
    return inner
```

Chaining Decorators in Python

Multiple decorators can be chained in Python.

This is to say, a function can be decorated multiple times with different (or same) decorators. We simply place the decorators above the desired function.

```
def star(func):
    def inner(*args, **kwargs):
        print("*" * 30)
        func(*args, **kwargs)
        print("*" * 30)
    return inner


def percent(func):
    def inner(*args, **kwargs):
        print("%" * 30)
        func(*args, **kwargs)
        print("%" * 30)
    return inner


@star
@percent
def printer(msg):
    print(msg)


printer("Hello")
```

Output

```
******************************
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
Hello
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
******************************
```

The above syntax of,

```
@star
@percent
def printer(msg):
    print(msg)
```

is equivalent to

```
def printer(msg):
    print(msg)
printer = star(percent(printer))
```

The order in which we chain decorators matter. If we had reversed the order as,

```
@percent
@star
def printer(msg):
    print(msg)
```

The output would be:

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
******************************
Hello
******************************
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

Python datetime

programiz.com/python-programming/datetime

In this article, you will learn to manipulate date and time in Python with the help of examples.

Python has a module named **datetime** to work with dates and times. Let's create a few simple programs related to date and time before we dig deeper.

Example 1: Get Current Date and Time

```
import datetime

datetime_object = datetime.datetime.now()
print(datetime_object)
```

When you run the program, the output will be something like:

```
2018-12-19 09:26:03.478039
```

Here, we have imported **datetime** module using `import datetime` statement.

One of the classes defined in the `datetime` module is `datetime` class. We then used `now()` method to create a `datetime` object containing the current local date and time.

Example 2: Get Current Date

```
import datetime

date_object = datetime.date.today()
print(date_object)
```

When you run the program, the output will be something like:

```
2018-12-19
```

In this program, we have used `today()` method defined in the `date` class to get a `date` object containing the current local date.

What's inside datetime?

We can use dir() function to get a list containing all attributes of a module.

```
import datetime

print(dir(datetime))
```

When you run the program, the output will be:

```
['MAXYEAR', 'MINYEAR', '__builtins__', '__cached__', '__doc__', '__file__',
'__loader__', '__name__', '__package__', '__spec__', '_divide_and_round', 'date',
'datetime', 'datetime_CAPI', 'time', 'timedelta', 'timezone', 'tzinfo']
```

Commonly used classes in the datetime module are:

- date Class
- time Class
- datetime Class
- timedelta Class

---

datetime.date Class

You can instantiate `date` objects from the `date` class. A date object represents a date (year, month and day).

Example 3: Date object to represent a date

```
import datetime

d = datetime.date(2019, 4, 13)
print(d)
```

When you run the program, the output will be:

```
2019-04-13
```

If you are wondering, `date()` in the above example is a constructor of the `date` class. The constructor takes three arguments: year, month and day.

The variable *a* is a `date` object.

We can only import `date` class from the `datetime` module. Here's how:

```
from datetime import date

a = date(2019, 4, 13)
print(a)
```

Example 4: Get current date

You can create a `date` object containing the current date by using a classmethod named `today()`. Here's how:

```
from datetime import date

today = date.today()

print("Current date =", today)
```

Example 5: Get date from a timestamp

We can also create `date` objects from a timestamp. A Unix timestamp is the number of seconds between a particular date and January 1, 1970 at UTC. You can convert a timestamp to date using `fromtimestamp()` method.

```
from datetime import date

timestamp = date.fromtimestamp(1326244364)
print("Date =", timestamp)
```

When you run the program, the output will be:

```
Date = 2012-01-11
```

Example 6: Print today's year, month and day

We can get year, month, day, day of the week etc. from the date object easily. Here's how:

```
from datetime import date

# date object of today's date
today = date.today()

print("Current year:", today.year)
print("Current month:", today.month)
print("Current day:", today.day)
```

datetime.time

A time object instantiated from the `time` class represents the local time.

Example 7: Time object to represent time

```
from datetime import time

# time(hour = 0, minute = 0, second = 0)
a = time()
print("a =", a)

# time(hour, minute and second)
b = time(11, 34, 56)
print("b =", b)

# time(hour, minute and second)
c = time(hour = 11, minute = 34, second = 56)
print("c =", c)

# time(hour, minute, second, microsecond)
d = time(11, 34, 56, 234566)
print("d =", d)
```

When you run the program, the output will be:

```
a = 00:00:00
b = 11:34:56
c = 11:34:56
d = 11:34:56.234566
```

Example 8: Print hour, minute, second and microsecond

Once you create a `time` object, you can easily print its attributes such as *hour*, *minute* etc.

```
from datetime import time

a = time(11, 34, 56)

print("hour =", a.hour)
print("minute =", a.minute)
print("second =", a.second)
print("microsecond =", a.microsecond)
```

When you run the example, the output will be:

```
hour = 11
minute = 34
second = 56
microsecond = 0
```

Notice that we haven't passed *microsecond* argument. Hence, its default value `0` is printed.

datetime.datetime

The `datetime` module has a class named `dateclass` that can contain information from both **date** and **time** objects.

Example 9: Python datetime object

```
from datetime import datetime

#datetime(year, month, day)
a = datetime(2018, 11, 28)
print(a)

# datetime(year, month, day, hour, minute, second, microsecond)
b = datetime(2017, 11, 28, 23, 55, 59, 342380)
print(b)
```

When you run the program, the output will be:

```
2018-11-28 00:00:00
2017-11-28 23:55:59.342380
```

The first three arguments *year*, *month* and *day* in the `datetime()` constructor are mandatory.

Example 10: Print year, month, hour, minute and timestamp

```
from datetime import datetime

a = datetime(2017, 11, 28, 23, 55, 59, 342380)
print("year =", a.year)
print("month =", a.month)
print("hour =", a.hour)
print("minute =", a.minute)
print("timestamp =", a.timestamp())
```

When you run the program, the output will be:

```
year = 2017
month = 11
day = 28
hour = 23
minute = 55
timestamp = 1511913359.34238
```

datetime.timedelta

A `timedelta` object represents the difference between two dates or times.

Example 11: Difference between two dates and times

```
from datetime import datetime, date

t1 = date(year = 2018, month = 7, day = 12)
t2 = date(year = 2017, month = 12, day = 23)
t3 = t1 - t2
print("t3 =", t3)

t4 = datetime(year = 2018, month = 7, day = 12, hour = 7, minute = 9, second = 33)
t5 = datetime(year = 2019, month = 6, day = 10, hour = 5, minute = 55, second =
13)
t6 = t4 - t5
print("t6 =", t6)

print("type of t3 =", type(t3))
print("type of t6 =", type(t6))
```

When you run the program, the output will be:

```
t3 = 201 days, 0:00:00
t6 = -333 days, 1:14:20
type of t3 = <class 'datetime.timedelta'>
type of t6 = <class 'datetime.timedelta'>
```

Notice, both *t3* and *t6* are of `<class 'datetime.timedelta'>` type.

Example 12: Difference between two timedelta objects

```
from datetime import timedelta

t1 = timedelta(weeks = 2, days = 5, hours = 1, seconds = 33)
t2 = timedelta(days = 4, hours = 11, minutes = 4, seconds = 54)
t3 = t1 - t2

print("t3 =", t3)
```

When you run the program, the output will be:

```
t3 = 14 days, 13:55:39
```

Here, we have created two `timedelta` objects *t1* and *t2*, and their difference is printed on the screen.

Example 13: Printing negative timedelta object

```
from datetime import timedelta

t1 = timedelta(seconds = 33)
t2 = timedelta(seconds = 54)
t3 = t1 - t2

print("t3 =", t3)
print("t3 =", abs(t3))
```

When you run the program, the output will be:

```
t3 = -1 day, 23:59:39
t3 = 0:00:21
```

Example 14: Time duration in seconds

You can get the total number of seconds in a timedelta object using `total_seconds()` method.

```
from datetime import timedelta

t = timedelta(days = 5, hours = 1, seconds = 33, microseconds = 233423)
print("total seconds =", t.total_seconds())
```

When you run the program, the output will be:

```
total seconds = 435633.233423
```

You can also find sum of two dates and times using `+` operator. Also, you can multiply and divide a `timedelta` object by integers and floats.

Python format datetime

The way date and time is represented may be different in different places, organizations etc. It's more common to use `mm/dd/yyyy` in the US, whereas `dd/mm/yyyy` is more common in the UK.

Python has `strftime()` and `strptime()` methods to handle this.

Python strftime() - datetime object to string

The `strftime()` method is defined under classes `date`, `datetime` and `time`. The method creates a formatted string from a given `date`, `datetime` or `time` object.

Example 15: Format date using strftime()

```
from datetime import datetime

# current date and time
now = datetime.now()

t = now.strftime("%H:%M:%S")
print("time:", t)

s1 = now.strftime("%m/%d/%Y, %H:%M:%S")
# mm/dd/YY H:M:S format
print("s1:", s1)

s2 = now.strftime("%d/%m/%Y, %H:%M:%S")
# dd/mm/YY H:M:S format
print("s2:", s2)
```

When you run the program, the output will be something like:

```
time: 04:34:52
s1: 12/26/2018, 04:34:52
s2: 26/12/2018, 04:34:52
```

Here, `%Y` , `%m` , `%d` , `%H` etc. are format codes. The `strftime()` method takes one or more format codes and returns a formatted string based on it.

In the above program, *t*, *s1* and *s2* are strings.

- `%Y` - year [0001,..., 2018, 2019,..., 9999]
- `%m` - month [01, 02, ..., 11, 12]
- `%d` - day [01, 02, ..., 30, 31]
- `%H` - hour [00, 01, ..., 22, 23
- `%M` - minute [00, 01, ..., 58, 59]
- `%S` - second [00, 01, ..., 58, 59]

To learn more about `strftime()` and format codes, visit: Python strftime().

Python strptime() - string to datetime

The `strptime()` method creates a `datetime` object from a given string (representing date and time).

Example 16: strptime()

```
from datetime import datetime

date_string = "21 June, 2018"
print("date_string =", date_string)

date_object = datetime.strptime(date_string, "%d %B, %Y")
print("date_object =", date_object)
```

When you run the program, the output will be:

```
date_string = 21 June, 2018
date_object = 2018-06-21 00:00:00
```

The `strptime()` method takes two arguments:

1. a string representing date and time
2. format code equivalent to the first argument

By the way, `%d` , `%B` and `%Y` format codes are used for *day*, *month*(full name) and *year* respectively.

Visit Python strptime() to learn more.

Handling timezone in Python

Suppose, you are working on a project and need to display date and time based on their timezone. Rather than trying to handle timezone yourself, we suggest you to use a third-party pytZ module.

```
from datetime import datetime
import pytz

local = datetime.now()
print("Local:", local.strftime("%m/%d/%Y, %H:%M:%S"))


tz_NY = pytz.timezone('America/New_York')
datetime_NY = datetime.now(tz_NY)
print("NY:", datetime_NY.strftime("%m/%d/%Y, %H:%M:%S"))

tz_London = pytz.timezone('Europe/London')
datetime_London = datetime.now(tz_London)
print("London:", datetime_London.strftime("%m/%d/%Y, %H:%M:%S"))
```

When you run the program, the output will be something like:

```
Local time: 2018-12-20 13:10:44.260462
America/New_York time: 2018-12-20 13:10:44.260462
Europe/London time: 2018-12-20 13:10:44.260462
```

Here, *datetime_NY* and *datetime_London* are datetime objects containing the current date and time of their respective timezone.

Python strftime()

programiz.com/python-programming/datetime/strftime

In this article, you will learn to convert date, time and datetime objects to its equivalent string (with the help of examples)

The `strftime()` method returns a string representing date and time using date, time or datetime object.

Example 1: datetime to string using strftime()

The program below converts a `datetime` object containing current date and time to different string formats.

```
from datetime import datetime

now = datetime.now() # current date and time

year = now.strftime("%Y")
print("year:", year)

month = now.strftime("%m")
print("month:", month)

day = now.strftime("%d")
print("day:", day)

time = now.strftime("%H:%M:%S")
print("time:", time)

date_time = now.strftime("%m/%d/%Y, %H:%M:%S")
print("date and time:",date_time)
```

When you run the program, the output will something like be:

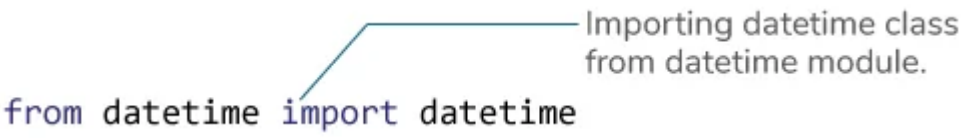
```
year: 2018
month: 12
day: 24
time: 04:59:31
date and time: 12/24/2018, 04:59:31
```

Here, *year*, *day*, *time* and *date_time* are strings, whereas *now* is a `datetime` object.

How strftime() works?

In the above program, `%Y` , `%m` , `%d` etc. are format codes. The `strftime()` method takes one or more format codes as an argument and returns a formatted string based on it.

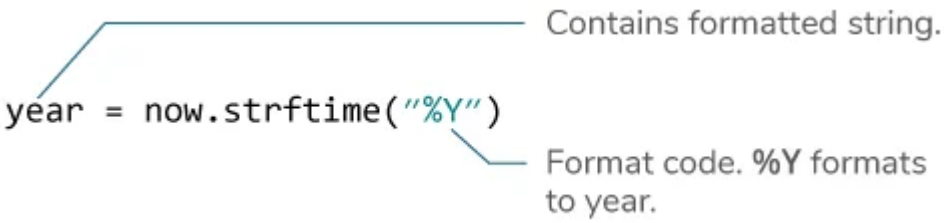
1. We imported `datetime` class from the `datetime` module. It's because the object of `datetime` class can access `strftime()` method.



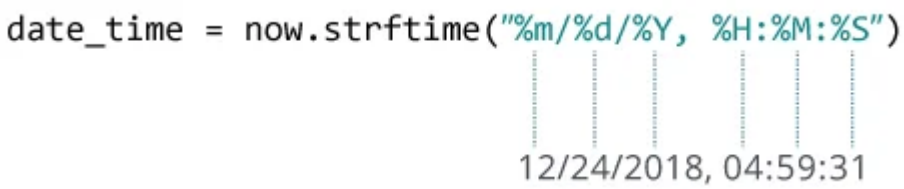
2. The `datetime` object containing current date and time is stored in *now* variable.



3. The `strftime()` method can be used to create formatted strings.



4. The string you pass to the `strftime()` method may contain more than one format codes.



Example 2: Creating string from a timestamp

```
from datetime import datetime

timestamp = 1528797322
date_time = datetime.fromtimestamp(timestamp)

print("Date time object:", date_time)

d = date_time.strftime("%m/%d/%Y, %H:%M:%S")
print("Output 2:", d)

d = date_time.strftime("%d %b, %Y")
print("Output 3:", d)

d = date_time.strftime("%d %B, %Y")
print("Output 4:", d)

d = date_time.strftime("%I%p")
print("Output 5:", d)
```

When you run the program, the output will be:

```
Date time object: 2018-06-12 09:55:22
Output 2: 06/12/2018, 09:55:22
Output 3: 12 Jun, 2018
Output 4: 12 June, 2018
Output 5: 09AM
```

Format Code List

The table below shows all the codes that you can pass to the `strftime()` method.

Directive	Meaning	Example
`%a`	Abbreviated weekday name.	Sun, Mon, ...
`%A`	Full weekday name.	Sunday, Monday, ...
`%w`	Weekday as a decimal number.	0, 1, ..., 6
`%d`	Day of the month as a zero-padded decimal.	01, 02, ..., 31
`%-d`	Day of the month as a decimal number.	1, 2, ..., 30
`%b`	Abbreviated month name.	Jan, Feb, ..., Dec
`%B`	Full month name.	January, February, ...

Directive	Meaning	Example
%m	Month as a zero-padded decimal number.	01, 02, ..., 12
%-m	Month as a decimal number.	1, 2, ..., 12
%y	Year without century as a zero-padded decimal number.	00, 01, ..., 99
%-y	Year without century as a decimal number.	0, 1, ..., 99
%Y	Year with century as a decimal number.	2013, 2019 etc.
%H	Hour (24-hour clock) as a zero-padded decimal number.	00, 01, ..., 23
%-H	Hour (24-hour clock) as a decimal number.	0, 1, ..., 23
%I	Hour (12-hour clock) as a zero-padded decimal number.	01, 02, ..., 12
%-I	Hour (12-hour clock) as a decimal number.	1, 2, ... 12
%p	Locale's AM or PM.	AM, PM
%M	Minute as a zero-padded decimal number.	00, 01, ..., 59
%-M	Minute as a decimal number.	0, 1, ..., 59
%S	Second as a zero-padded decimal number.	00, 01, ..., 59
%-S	Second as a decimal number.	0, 1, ..., 59
%f	Microsecond as a decimal number, zero-padded on the left.	000000 - 999999
%z	UTC offset in the form +HHMM or -HHMM.	
%Z	Time zone name.	
%j	Day of the year as a zero-padded decimal number.	001, 002, ..., 366
%-j	Day of the year as a decimal number.	1, 2, ..., 366
%U	Week number of the year (Sunday as the first day of the week). All days in a new year preceding the first Sunday are considered to be in week 0.	00, 01, ..., 53

Directive	Meaning	Example
`%W`	Week number of the year (Monday as the first day of the week). All days in a new year preceding the first Monday are considered to be in week 0.	00, 01, ..., 53
`%c`	Locale's appropriate date and time representation.	Mon Sep 30 07:06:05 2013
`%x`	Locale's appropriate date representation.	09/30/13
`%X`	Locale's appropriate time representation.	07:06:05
`%%`	A literal '%' character.	%

Example 3: Locale's appropriate date and time

```
from datetime import datetime

timestamp = 1528797322
date_time = datetime.fromtimestamp(timestamp)

d = date_time.strftime("%c")
print("Output 1:", d)

d = date_time.strftime("%x")
print("Output 2:", d)

d = date_time.strftime("%X")
print("Output 3:", d)
```

When you run the program, the output will be:

```
Output 1: Tue Jun 12 09:55:22 2018
Output 2: 06/12/18
Output 3: 09:55:22
```

Format codes `%c` , `%x` and `%X` are used for locale's appropriate date and time representation.

We also recommend you to check Python strptime(). The `strptime()` method creates a `datetime` object from a string.

Python Get Current time

programiz.com/python-programming/datetime/current-time

In this article, you will learn to get current time of your locale as well as different time zones in Python.

There are a number of ways you can take to get current time in Python.

Example 1: Current time using datetime object

```
from datetime import datetime

now = datetime.now()

current_time = now.strftime("%H:%M:%S")
print("Current Time =", current_time)
```

Output

```
Current Time = 07:41:19
```

In the above example, we have imported `datetime` class from the datetime module. Then, we used `now()` method to get a `datetime` object containing current date and time.

Using datetime.strftime() method, we then created a string representing current time.

If you need to create a `time` object containing current time, you can do something like this.

```
from datetime import datetime

now = datetime.now().time() # time object

print("now =", now)
print("type(now) =", type(now))
```

Output

```
now = 07:43:37.457423
type(now) = <class 'datetime.time'>
```

Example 2: Current time using time module

You can also get the current time using time module.

```
import time

t = time.localtime()
current_time = time.strftime("%H:%M:%S", t)
print(current_time)
```

Output

```
07:46:58
```

Example 3: Current time of a timezone

If you need to find current time of a certain timezone, you can use pytZ module.

```
from datetime import datetime
import pytz

tz_NY = pytz.timezone('America/New_York')
datetime_NY = datetime.now(tz_NY)
print("NY time:", datetime_NY.strftime("%H:%M:%S"))

tz_London = pytz.timezone('Europe/London')
datetime_London = datetime.now(tz_London)
print("London time:", datetime_London.strftime("%H:%M:%S"))
```

Output

```
NY time: 03:45:16
London time: 08:45:16
```

Python timestamp to datetime and vice-versa

programiz.com/python-programming/datetime/timestamp-datetime

In this article, you will learn to convert timestamp to datetime object and datetime object to timestamp (with the help of examples).

It's pretty common to store date and time as a timestamp in a database. A Unix timestamp is the number of seconds between a particular date and January 1, 1970 at UTC.

Example 1: Python timestamp to datetime

```
from datetime import datetime

timestamp = 1545730073
dt_object = datetime.fromtimestamp(timestamp)

print("dt_object =", dt_object)
print("type(dt_object) =", type(dt_object))
```

When you run the program, the output will be:

```
dt_object = 2018-12-25 09:27:53
type(dt_object) = <class 'datetime.datetime'>
```

Here, we have imported `datetime` class from the datetime module. Then, we used `datetime.fromtimestamp()` classmethod which returns the local date and time (datetime object). This object is stored in *dt_object* variable.

Note: You can easily create a string representing date and time from a `datetime` object using strftime() method.

Example 2: Python datetime to timestamp

You can get timestamp from a datetime object using `datetime.timestamp()` method.

```
from datetime import datetime

# current date and time
now = datetime.now()

timestamp = datetime.timestamp(now)
print("timestamp =", timestamp)
```

Python time Module

programiz.com/python-programming/time

In this article, we will explore time module in detail. We will learn to use different time-related functions defined in the time module with the help of examples.

Python has a module named `time` to handle time-related tasks. To use functions defined in the module, we need to import the module first. Here's how:

```
import time
```

Here are commonly used time-related functions.

Python time.time()

The `time()` function returns the number of seconds passed since epoch.

For Unix system, `January 1, 1970, 00:00:00` at **UTC** is epoch (the point where time begins).

```
import time
seconds = time.time()
print("Seconds since epoch =", seconds)
```

Python time.ctime()

The `time.ctime()` function takes seconds passed since epoch as an argument and returns a string representing local time.

```
import time

# seconds passed since epoch
seconds = 1545925769.9618232
local_time = time.ctime(seconds)
print("Local time:", local_time)
```

If you run the program, the output will be something like:

```
Local time: Thu Dec 27 15:49:29 2018
```

Python time.sleep()

The `sleep()` function suspends (delays) execution of the current thread for the given number of seconds.

```
import time

print("This is printed immediately.")
time.sleep(2.4)
print("This is printed after 2.4 seconds.")
```

To learn more, visit: Python sleep().

Before we talk about other time-related functions, let's explore `time.struct_time` class in brief.

---

time.struct_time Class

Several functions in the `time` module such as `gmtime()`, `asctime()` etc. either take `time.struct_time` object as an argument or return it.

Here's an example of `time.struct_time` object.

```
time.struct_time(tm_year=2018, tm_mon=12, tm_mday=27,
                 tm_hour=6, tm_min=35, tm_sec=17,
                 tm_wday=3, tm_yday=361, tm_isdst=0)
```

Index	Attribute	Values
0	`tm_year`	0000, ..., 2018, ..., 9999
1	`tm_mon`	1, 2, ..., 12
2	`tm_mday`	1, 2, ..., 31
3	`tm_hour`	0, 1, ..., 23
4	`tm_min`	0, 1, ..., 59
5	`tm_sec`	0, 1, ..., 61
6	`tm_wday`	0, 1, ..., 6; Monday is 0
7	`tm_yday`	1, 2, ..., 366
8	`tm_isdst`	0, 1 or -1

The values (elements) of the `time.struct_time` object are accessible using both indices and attributes.

Python time.localtime()

The `localtime()` function takes the number of seconds passed since epoch as an argument and returns `struct_time` in **local time**.

```
import time

result = time.localtime(1545925769)
print("result:", result)
print("\nyear:", result.tm_year)
print("tm_hour:", result.tm_hour)
```

When you run the program, the output will be something like:

```
result: time.struct_time(tm_year=2018, tm_mon=12, tm_mday=27, tm_hour=15,
tm_min=49, tm_sec=29, tm_wday=3, tm_yday=361, tm_isdst=0)

year: 2018
tm_hour: 15
```

If no argument or `None` is passed to `localtime()`, the value returned by `time()` is used.

Python time.gmtime()

The `gmtime()` function takes the number of seconds passed since epoch as an argument and returns `struct_time` in **UTC**.

```
import time

result = time.gmtime(1545925769)
print("result:", result)
print("\nyear:", result.tm_year)
print("tm_hour:", result.tm_hour)
```

When you run the program, the output will be:

```
result = time.struct_time(tm_year=2018, tm_mon=12, tm_mday=28, tm_hour=8,
tm_min=44, tm_sec=4, tm_wday=4, tm_yday=362, tm_isdst=0)

year = 2018
tm_hour = 8
```

If no argument or `None` is passed to `gmtime()`, the value returned by `time()` is used.

Python time.mktime()

The `mktime()` function takes `struct_time` (or a tuple containing 9 elements corresponding to `struct_time`) as an argument and returns the seconds passed since epoch in local time. Basically, it's the inverse function of `localtime()`.

```
import time

t = (2018, 12, 28, 8, 44, 4, 4, 362, 0)

local_time = time.mktime(t)
print("Local time:", local_time)
```

The example below shows how `mktime()` and `localtime()` are related.

```
import time

seconds = 1545925769

# returns struct_time
t = time.localtime(seconds)
print("t1: ", t)

# returns seconds from struct_time
s = time.mktime(t)
print("\s:", seconds)
```

When you run the program, the output will be something like:

```
t1:  time.struct_time(tm_year=2018, tm_mon=12, tm_mday=27, tm_hour=15, tm_min=49,
tm_sec=29, tm_wday=3, tm_yday=361, tm_isdst=0)

s: 1545925769.0
```

Python time.asctime()

The `asctime()` function takes `struct_time` (or a tuple containing 9 elements corresponding to `struct_time`) as an argument and returns a string representing it. Here's an example:

```
import time

t = (2018, 12, 28, 8, 44, 4, 4, 362, 0)

result = time.asctime(t)
print("Result:", result)
```

When you run the program, the output will be:

```
Result: Fri Dec 28 08:44:04 2018
```

Python time.strftime()

The `strftime()` function takes `struct_time` (or tuple corresponding to it) as an argument and returns a string representing it based on the format code used. For example,

```
import time

named_tuple = time.localtime() # get struct_time
time_string = time.strftime("%m/%d/%Y, %H:%M:%S", named_tuple)

print(time_string)
```

When you run the program, the output will be something like:

```
12/28/2018, 09:47:41
```

Here, `%Y` , `%m` , `%d` , `%H` etc. are format codes.

- `%Y` - year [0001,..., 2018, 2019,..., 9999]
- `%m` - month [01, 02, ..., 11, 12]
- `%d` - day [01, 02, ..., 30, 31]
- `%H` - hour [00, 01, ..., 22, 23
- `%M` - minutes [00, 01, ..., 58, 59]
- `%S` - second [00, 01, ..., 58, 61]

To learn more, visit: time.strftime().

Python time.strptime()

The `strptime()` function parses a string representing time and returns `struct_time` .

```
import time

time_string = "21 June, 2018"
result = time.strptime(time_string, "%d %B, %Y")

print(result)
```

When you run the program, the output will be:

```
time.struct_time(tm_year=2018, tm_mon=6, tm_mday=21, tm_hour=0, tm_min=0,
tm_sec=0, tm_wday=3, tm_yday=172, tm_isdst=-1)
```

Python sleep()

programiz.com/python-programming/time/sleep

The sleep() function suspends (waits) execution of the current thread for a given number of seconds.

Python has a module named time which provides several useful functions to handle time-related tasks. One of the popular functions among them is `sleep()`.

The `sleep()` function suspends execution of the current thread for a given number of seconds.

Example 1: Python sleep()

```
import time

print("Printed immediately.")
time.sleep(2.4)
print("Printed after 2.4 seconds.")
```

Here's how this program works:

- `"Printed immediately"` is printed
- Suspends (Delays) execution for 2.4 seconds.
- `"Printed after 2.4 seconds"` is printed.

As you can see from the above example, `sleep()` takes a floating-point number as an argument.

Before Python 3.5, the actual suspension time may be less than the argument specified to the `time()` function.

Since Python 3.5, the suspension time will be at least the seconds specified.

Example 2: Python create a digital clock

```
import time

while True:
  localtime = time.localtime()
  result = time.strftime("%I:%M:%S %p", localtime)
  print(result)
  time.sleep(1)
```

In the above program, we computed and printed the current local time inside the infinite while loop. Then, the program waits for 1 second. Again, the current local time is computed and printed. This process goes on.

When you run the program, the output will be something like:

```
02:10:50 PM
02:10:51 PM
02:10:52 PM
02:10:53 PM
02:10:54 PM
... .. ...
```

Here is a slightly modified better version of the above program.

```
import time

while True:
  localtime = time.localtime()
  result = time.strftime("%I:%M:%S %p", localtime)
  print(result, end="", flush=True)
  print("\r", end="", flush=True)
  time.sleep(1)
```

To learn more, visit digital clock in Python shell.

Multithreading in Python

Before talking about `sleep()` in multithreaded programs, let's talk about processes and threads.

A computer program is a collection of instructions. A process is the execution of those instructions.

A thread is a subset of the process. A process can have one or more threads.

Example 3: Python multithreading

All the programs above in this article are single-threaded programs. Here's an example of a multithreaded Python program.

```
import threading

def print_hello_three_times():
  for i in range(3):
    print("Hello")

def print_hi_three_times():
    for i in range(3):
      print("Hi")

t1 = threading.Thread(target=print_hello_three_times)
t2 = threading.Thread(target=print_hi_three_times)

t1.start()
t2.start()
```

When you run the program, the output will be something like:

```
Hello
Hello
Hi
Hello
Hi
Hi
```

The above program has two threads *t1* and *t2*. These threads are run using `t1.start()` and `t2.start()` statements.

Note that, *t1* and *t2* run concurrently and you might get different output.

Visit this page to learn more about Multithreading in Python.

---

time.sleep() in multithreaded programs

The `sleep()` function suspends execution of the current thread for a given number of seconds.

In case of single-threaded programs, `sleep()` suspends execution of the thread and process. However, the function suspends a thread rather than the whole process in multithreaded programs.

Example 4: sleep() in a multithreaded program

```
import threading
import time

def print_hello():
  for i in range(4):
    time.sleep(0.5)
    print("Hello")

def print_hi():
    for i in range(4):
      time.sleep(0.7)
      print("Hi")

t1 = threading.Thread(target=print_hello)
t2 = threading.Thread(target=print_hi)
t1.start()
t2.start()
```

The above program has two threads. We have used `time.sleep(0.5)` and `time.sleep(0.75)` to suspend execution of these two threads for 0.5 seconds and 0.7 seconds respectively.

Recommended Reading: Python time.sleep() sleeps thread

Python strptime()

programiz.com/python-programming/datetime/strptime

In this article, you will learn to create a datetime object from a string (with the help of examples).

The `strptime()` method creates a datetime object from the given string.

Note: You cannot create `datetime` object from every string. The string needs to be in a certain format.

Example 1: string to datetime object

```
from datetime import datetime

date_string = "21 June, 2018"

print("date_string =", date_string)
print("type of date_string =", type(date_string))

date_object = datetime.strptime(date_string, "%d %B, %Y")

print("date_object =", date_object)
print("type of date_object =", type(date_object))
```

When you run the program, the output will be:

```
date_string = 21 June, 2018
type of date_string = <class 'str'>
date_object = 2018-06-21 00:00:00
type of date_object = <class 'datetime.datetime'>
```

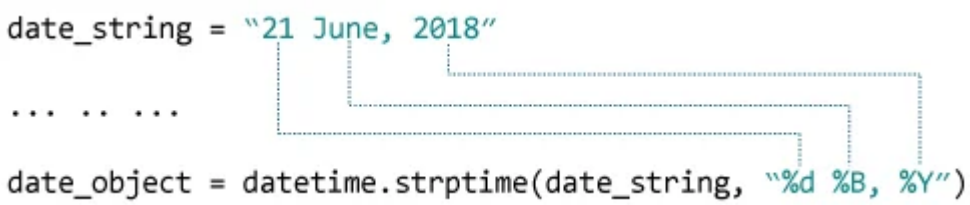
How strptime() works?

The `strptime()` class method takes two arguments:

- string (that be converted to datetime)
- format code

Based on the string and format code used, the method returns its equivalent `datetime` object.

In the above example:



Here,

- `%d` - Represents the day of the month. **Example:** 01, 02, ..., 31
- `%B` - Month's name in full. **Example:** January, February etc.
- `%Y` - Year in four digits. **Example:** 2018, 2019 etc.

Example 2: string to datetime object

```
from datetime import datetime

dt_string = "12/11/2018 09:15:32"

# Considering date is in dd/mm/yyyy format
dt_object1 = datetime.strptime(dt_string, "%d/%m/%Y %H:%M:%S")
print("dt_object1 =", dt_object1)

# Considering date is in mm/dd/yyyy format
dt_object2 = datetime.strptime(dt_string, "%m/%d/%Y %H:%M:%S")
print("dt_object2 =", dt_object2)
```

When you run the program, the output will be:

```
dt_object1 = 2018-11-12 09:15:32
dt_object2 = 2018-12-11 09:15:32
```

Format Code List

The table below shows all the format codes that you can use.

Directive	Meaning	Example
`%a`	Abbreviated weekday name.	Sun, Mon, ...
`%A`	Full weekday name.	Sunday, Monday, ...
`%w`	Weekday as a decimal number.	0, 1, ..., 6
`%d`	Day of the month as a zero-padded decimal.	01, 02, ..., 31

Directive	Meaning	Example
`%-d`	Day of the month as a decimal number.	1, 2, ..., 30
`%b`	Abbreviated month name.	Jan, Feb, ..., Dec
`%B`	Full month name.	January, February, ...
`%m`	Month as a zero-padded decimal number.	01, 02, ..., 12
`%-m`	Month as a decimal number.	1, 2, ..., 12
`%y`	Year without century as a zero-padded decimal number.	00, 01, ..., 99
`%-y`	Year without century as a decimal number.	0, 1, ..., 99
`%Y`	Year with century as a decimal number.	2013, 2019 etc.
`%H`	Hour (24-hour clock) as a zero-padded decimal number.	00, 01, ..., 23
`%-H`	Hour (24-hour clock) as a decimal number.	0, 1, ..., 23
`%I`	Hour (12-hour clock) as a zero-padded decimal number.	01, 02, ..., 12
`%-I`	Hour (12-hour clock) as a decimal number.	1, 2, ... 12
`%p`	Locale's AM or PM.	AM, PM
`%M`	Minute as a zero-padded decimal number.	00, 01, ..., 59
`%-M`	Minute as a decimal number.	0, 1, ..., 59
`%S`	Second as a zero-padded decimal number.	00, 01, ..., 59
`%-S`	Second as a decimal number.	0, 1, ..., 59
`%f`	Microsecond as a decimal number, zero-padded on the left.	000000 - 999999
`%z`	UTC offset in the form +HHMM or -HHMM.	
`%Z`	Time zone name.	
`%j`	Day of the year as a zero-padded decimal number.	001, 002, ..., 366

Directive	Meaning	Example
`%-j`	Day of the year as a decimal number.	1, 2, ..., 366
`%U`	Week number of the year (Sunday as the first day of the week). All days in a new year preceding the first Sunday are considered to be in week 0.	00, 01, ..., 53
`%W`	Week number of the year (Monday as the first day of the week). All days in a new year preceding the first Monday are considered to be in week 0.	00, 01, ..., 53
`%c`	Locale's appropriate date and time representation.	Mon Sep 30 07:06:05 2013
`%x`	Locale's appropriate date representation.	09/30/13
`%X`	Locale's appropriate time representation.	07:06:05
`%%`	A literal '%' character.	%

ValueError in strptime()

If the string (first argument) and the format code (second argument) passed to the `strptime()` doesn't match, you will get `ValueError`. For example:

```
from datetime import datetime

date_string = "12/11/2018"
date_object = datetime.strptime(date_string, "%d %m %Y")

print("date_object =", date_object)
```

If you run this program, you will get an error.

```
ValueError: time data '12/11/2018' does not match format '%d %m %Y'
```

Recommended Readings: Python strftime()

Python Examples

programiz.com/python-programming/examples

The best way to learn Python is by practicing examples. This page contains examples on basic concepts of Python. We encourage you to try these examples on your own before looking at the solution.

All the programs on this page are tested and should work on all platforms.

1. Python Program to Print Hello world!
2. Python Program to Add Two Numbers
3. Python Program to Find the Square Root
4. Python Program to Calculate the Area of a Triangle
5. Python Program to Solve Quadratic Equation
6. Python Program to Swap Two Variables
7. Python Program to Generate a Random Number
8. Python Program to Convert Kilometers to Miles
9. Python Program to Convert Celsius To Fahrenheit
10. Python Program to Check if a Number is Positive, Negative or 0
11. Python Program to Check if a Number is Odd or Even
12. Python Program to Check Leap Year
13. Python Program to Find the Largest Among Three Numbers
14. Python Program to Check Prime Number
15. Python Program to Print all Prime Numbers in an Interval
16. Python Program to Find the Factorial of a Number
17. Python Program to Display the multiplication Table
18. Python Program to Print the Fibonacci sequence
19. Python Program to Check Armstrong Number
20. Python Program to Find Armstrong Number in an Interval
21. Python Program to Find the Sum of Natural Numbers
22. Python Program to Display Powers of 2 Using Anonymous Function
23. Python Program to Find Numbers Divisible by Another Number
24. Python Program to Convert Decimal to Binary, Octal and Hexadecimal
25. Python Program to Find ASCII Value of Character
26. Python Program to Find HCF or GCD
27. Python Program to Find LCM
28. Python Program to Find the Factors of a Number
29. Python Program to Make a Simple Calculator
30. Python Program to Shuffle Deck of Cards
31. Python Program to Display Calendar
32. Python Program to Display Fibonacci Sequence Using Recursion
33. Python Program to Find Sum of Natural Numbers Using Recursion
34. Python Program to Find Factorial of Number Using Recursion

35. Python Program to Convert Decimal to Binary Using Recursion
36. Python Program to Add Two Matrices
37. Python Program to Transpose a Matrix
38. Python Program to Multiply Two Matrices
39. Python Program to Check Whether a String is Palindrome or Not
40. Python Program to Remove Punctuations From a String
41. Python Program to Sort Words in Alphabetic Order
42. Python Program to Illustrate Different Set Operations
43. Python Program to Count the Number of Each Vowel
44. Python Program to Merge Mails
45. Python Program to Find the Size (Resolution) of a Image
46. Python Program to Find Hash of File
47. Python Program to Create Pyramid Patterns
48. Python Program to Merge Two Dictionaries
49. Python Program to Safely Create a Nested Directory
50. Python Program to Access Index of a List Using for Loop
51. Python Program to Flatten a Nested List
52. Python Program to Slice Lists
53. Python Program to Iterate Over Dictionaries Using for Loop
54. Python Program to Sort a Dictionary by Value
55. Python Program to Check If a List is Empty
56. Python Program to Catch Multiple Exceptions in One Line
57. Python Program to Copy a File
58. Python Program to Concatenate Two Lists
59. Python Program to Check if a Key is Already Present in a Dictionary
60. Python Program to Split a List Into Evenly Sized Chunks
61. Python Program to Parse a String to a Float or Int
62. Python Program to Print Colored Text to the Terminal
63. Python Program to Convert String to Datetime
64. Python Program to Get the Last Element of the List
65. Python Program to Get a Substring of a String
66. Python Program to Print Output Without a Newline
67. Python Program Read a File Line by Line Into a List
68. Python Program to Randomly Select an Element From the List
69. Python Program to Check If a String Is a Number (Float)
70. Python Program to Count the Occurrence of an Item in a List
71. Python Program to Append to a File
72. Python Program to Delete an Element From a Dictionary
73. Python Program to Create a Long Multiline String
74. Python Program to Extract Extension From the File Name
75. Python Program to Measure the Elapsed Time in Python
76. Python Program to Get the Class Name of an Instance
77. Python Program to Convert Two Lists Into a Dictionary
78. Python Program to Differentiate Between type() and isinstance()
79. Python Program to Trim Whitespace From a String

80. Python Program to Get the File Name From the File Path
81. Python Program to Represent enum
82. Python Program to Return Multiple Values From a Function
83. Python Program to Get Line Count of a File
84. Python Program to Find All File with .txt Extension Present Inside a Directory
85. Python Program to Get File Creation and Modification Date
86. Python Program to Get the Full Path of the Current Working Directory
87. Python Program to Iterate Through Two Lists in Parallel
88. Python Program to Check the File Size
89. Python Program to Reverse a Number
90. Python Program to Compute the Power of a Number
91. Python Program to Count the Number of Digits Present In a Number
92. Python Program to Check If Two Strings are Anagram
93. Python Program to Capitalize the First Character of a String
94. Python Program to Compute all the Permutation of the String
95. Python Program to Create a Countdown Timer
96. Python Program to Count the Number of Occurrence of a Character in String
97. Python Program to Remove Duplicate Element From a List
98. Python Program to Convert Bytes to a String

© Parewa Labs Pvt. Ltd. All rights reserved.